Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that _____**Prajjwal Pandey**_____ of **D20A** semester **VIII,** have successfully completed necessary experiments in the **ITC801: Blockchain & DLT Lab** under my supervision in **VES Institute of Technology** during the academic year **2025-2026.**

Subject Teacher
Name:Mrs. Kajal Joseph
Signature:

Head of Department
Name: Dr. Mrs. Shalu Chopra
Signature:

Lab Teacher
Name: Kajal Joseph
Signature:

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	**Computing & Emerging Technology:** Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	**Professionalism & Innovation :** Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	**PO**											**PSO**	
	PO 1	**PO 2**	**PO 3**	**PO 4**	**PO 5**	**PO 6**	**PO 7**	**PO 8**	**PO 9**	**PO 10**	**PO 11**	**PSO 1**	**PSO 2**
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1
13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Blockchain Exp - 1

Prajjwal Pandey D20A / Batch A / 37

AIM: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash.

Theory: SHA and Merkle Tree in Blockchain

1. Cryptographic Hash Functions in Blockchain

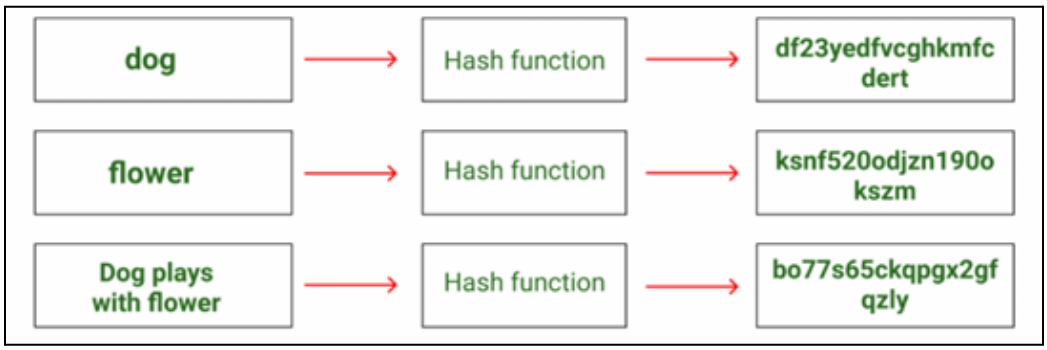
A cryptographic hash function is a fundamental building block of blockchain technology. It accepts input data of arbitrary size and produces a fixed-size output known as a **hash digest**. These functions are designed to be fast to compute but extremely hard to reverse or manipulate.

For a hash function to be secure and effective in blockchain systems, it must satisfy the following properties:

- **Collision Resistance**: It should be computationally infeasible to find two different inputs that generate the same hash value.
- **Preimage Resistance**: Given a hash output, it must be practically impossible to determine the original input.
- **Second Preimage Resistance**: Given an input and its hash, it should be difficult to find another input that produces the same hash.
- **Deterministic Nature**: The same input must always produce the same hash output.
- **Avalanche Effect**: A small change in the input should cause a drastic and unpredictable change in the output hash.
- **Fixed Output Length**: Regardless of input size, the hash output length remains constant.
- **Computational Infeasibility**: Guessing the output without computing the hash is practically impossible.

Role in Blockchain:

- Ensures immutability of transaction data
- Links blocks together through hash pointers
- Used in Proof-of-Work mechanisms
- Forms the basis of Merkle Trees and digital signatures

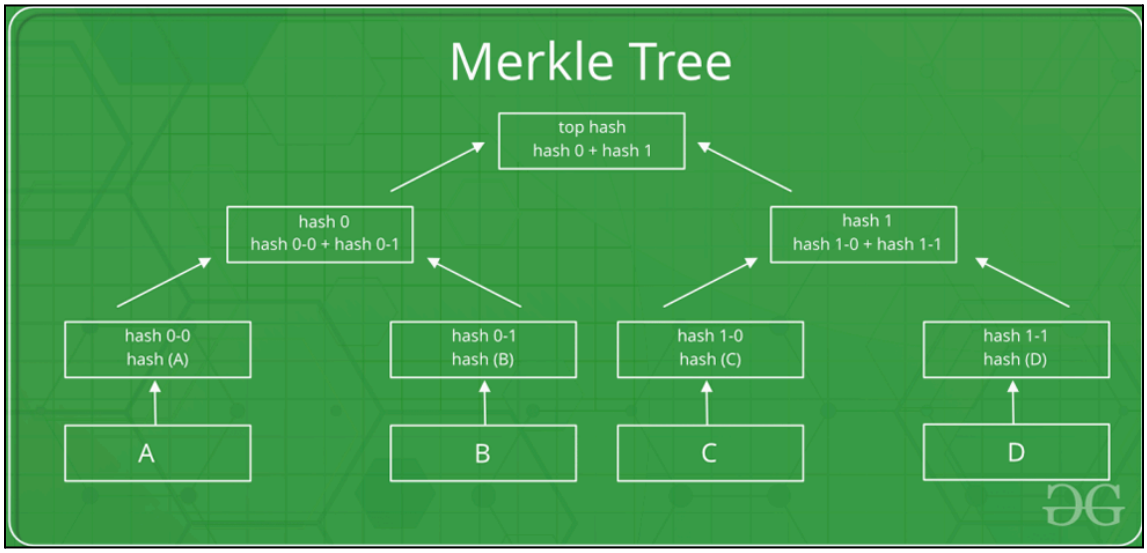


2. Merkle Tree (Hash Tree)

A Merkle Tree, also known as a **hash tree**, is a hierarchical data structure that enables efficient and secure verification of large datasets. It is constructed by recursively hashing pairs of data until a single hash value remains.

Each **leaf node** represents the hash of a data block (transaction), while each **internal node** stores the hash of its child nodes. The final hash at the top is called the **Merkle Root**, representing the integrity of the entire dataset.

Merkle Trees are widely used in blockchains to verify transactions efficiently without requiring access to all stored data.

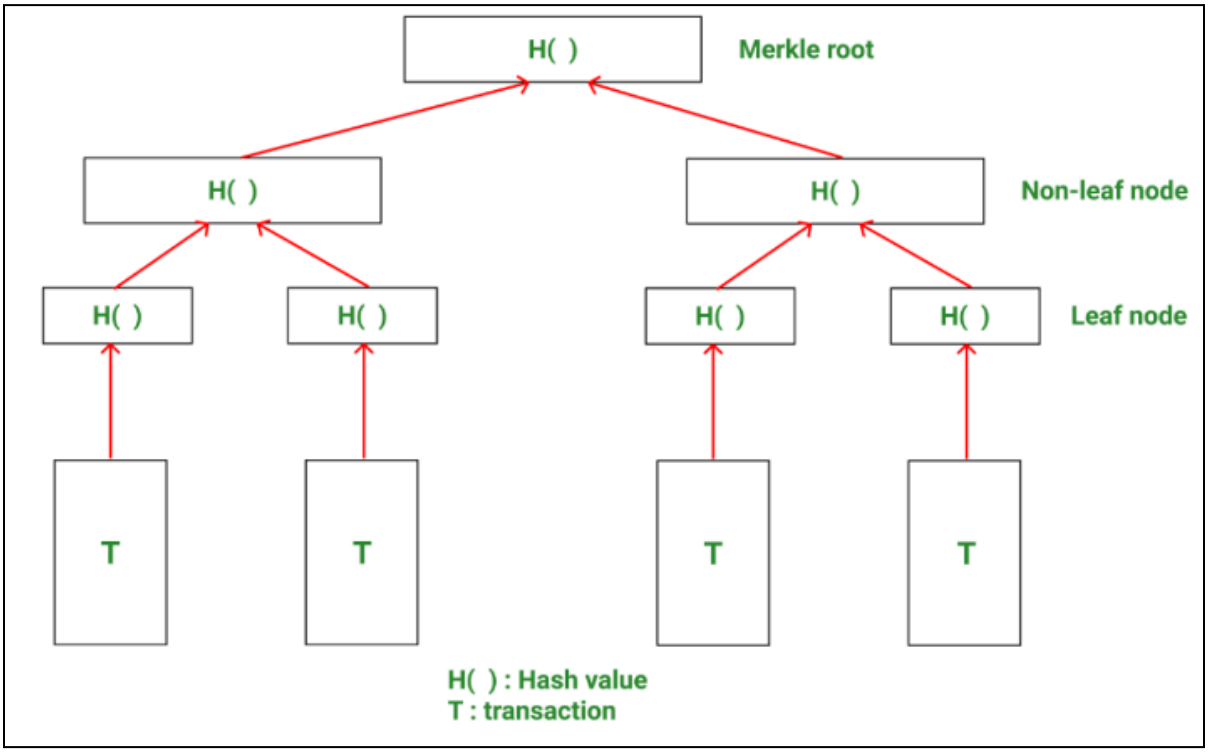


3. Structure of a Merkle Tree

A Merkle Tree consists of the following components:

- **Leaf Nodes**: Store the cryptographic hashes of individual transactions or data blocks.
- **Intermediate Nodes**: Generated by concatenating and hashing the hashes of child nodes.
- **Merkle Root**: The topmost node that acts as a unique digital fingerprint for all underlying data.

If even a single transaction changes, the resulting change propagates upward, altering the Merkle Root and indicating data tampering.



4. Merkle Root

The Merkle Root is the final hash produced after repeatedly hashing all transaction pairs in a Merkle Tree. It compactly represents the entire set of transactions in a block.

Importance of Merkle Root:

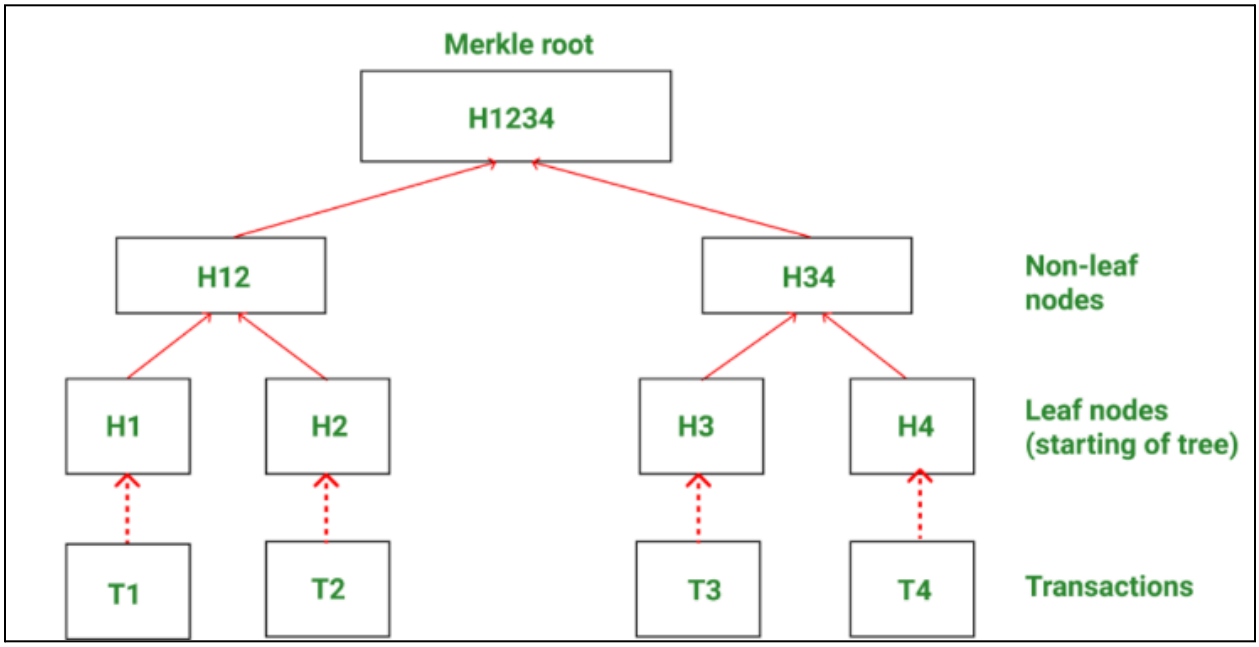
1. Acts as a single integrity check for all transactions
2. Stored in the block header instead of full transaction data
3. Any modification in a transaction changes the Merkle Root
4. Enables fast verification using Merkle Proofs
5. Strengthens security and immutability of the blockchain

5. Working of a Merkle Tree

The construction of a Merkle Tree follows a **bottom-up approach**:

1. Each transaction is hashed individually.
2. Hashes are grouped in pairs and concatenated.
3. The concatenated values are hashed again to form parent nodes.
4. This process repeats until only one hash remains — the Merkle Root.

If the number of nodes at any level is odd, the last hash is duplicated to maintain a complete binary structure.



Example: Consider a block having 4 transactions- T1, T2, T3, T4. These four transactions have to be stored in the Merkle tree and this is done by the following steps

Step 1: The hash of each transaction is computed.

H1 = Hash(T1)

Step 2: The hashes computed are stored in leaf nodes of the Merkle tree.

Step 3: Now non-leaf nodes will be formed. In order to form these nodes, leaf nodes will be paired together from left to right, and the hash of these pairs will be calculated. Firstly hash of H1 and H2 will be computed to form H12. Similarly, H34 is computed. Values H12 and H34 are parent nodes of H1, H2, and H3, H4 respectively. These are non-leaf nodes.

H12 = Hash(H1 + H2)

H34 = Hash(H3 + H4)

Step 4: Finally H1234 is computed by pairing H12 and H34. H1234 is the only hash remaining.

This means we have reached the root node and therefore H1234 is the Merkle root.

H1234 = Hash(H12 + H34)

6. Advantages of Merkle Trees

- **Efficient Verification**: Requires minimal data to verify transaction authenticity
- **Reduced Bandwidth Usage**: Only hash values are transmitted instead of full data
- **Tamper Detection**: Even minor data changes are immediately detectable
- **Storage Optimization**: Occupies less space than storing full datasets
- **Supports Trustless Systems**: Enables verification without centralized authority

7. Role of Merkle Trees in Blockchain

In decentralized systems like blockchain, each node maintains a copy of the ledger. Without Merkle Trees, validating transactions would require sharing entire datasets between nodes.

Merkle Trees solve this problem by:

- Allowing verification using small Merkle Proofs
- Reducing computation and communication overhead
- Enabling nodes to validate transactions independently
- Improving scalability and efficiency of blockchain networks

8. Applications of Merkle Trees

1. **Blockchain Light Clients (SPV)**
   Enables mobile wallets to verify transactions using only block headers and Merkle proofs.
2. **Version Control Systems (Git)**
   Tracks file changes efficiently by hashing directory structures.
3. **Distributed Databases**
   Used in systems like Cassandra for data synchronization and repair.

4. **Peer-to-Peer File Sharing**
   Ensures integrity of downloaded file chunks in BitTorrent and IPFS.
5. **Certificate Transparency**
   Verifies SSL certificates without downloading massive certificate logs.

PROGRAMS & OUTPUT :

Program 1: Python program that uses the hashlib library to create the hash of a given string:

```
import hashlib

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

```
Enter a string: prajjwal
Hash: b2d3977033871cb96477ac0cdc4bee8442df9e14eed9c68a9174ee43cac8267c
```

Program 2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
```

```
Enter a string: PRAJJWAL
Enter the nonce: 1
Hash Code: 3919c66c5bbe2c20311e2324140351f413827d4aabae929c5c6a038927c084d9
```

Program 3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 9
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:",hash_code )
            return nonce

        nonce += 1

# Get user input
input_string = "A"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

```
Hash: 06663975f75f189f1d70bd14d8f22df264cd6a5c7575d6875183d0ec76432fbb
Input String: A
Leading Zeros: 1
Expected Nonce: 9
```

Program 4: Generating Merkle Tree for given set of Transactions

```python
import hashlib

def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    if len(transactions) == 1:
        return transactions[0]

    # Recursive construction of the Merkle Tree
    while len(transactions) > 1:
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []
        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i+1]
            hash_combined = hashlib.sha256(combined.encode('utf-8')).hexdigest()
            new_transactions.append(hash_combined)

        transactions = new_transactions

    return transactions[0]

# Example usage
transactions = ["Transaction 1", "Transaction 2", "Transaction 3", "Transaction 4", "Transaction 5"]

merkle_root = build_merkle_tree(transactions)
print("Merkle Root:", merkle_root)
```

```
Merkle Root: a4a18941de1162b17a46c4f8c87d8a0850b46fad17ac881340061d9233785077
```

Conclusion

This experiment demonstrates how cryptographic hash functions and Merkle Trees form the backbone of blockchain security. Hashing ensures data integrity and immutability, while Merkle Trees enable efficient verification of large transaction sets. Together, these mechanisms make blockchain systems secure, scalable, and resistant to tampering, reinforcing trust in decentralized networks.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain & DLT Lab

Experiment 02

Aim:Create a Blockchain using Python

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Blockchain Exp - 2

AIM: Create a Blockchain using Python.

Theory:

1. What is a Blockchain?

A **Blockchain** is a decentralized, distributed, and immutable digital ledger used to record transactions across multiple computers in a secure and transparent manner. Instead of relying on a central authority, blockchain operates on a **peer-to-peer network**, where each participant (node) maintains a copy of the ledger.

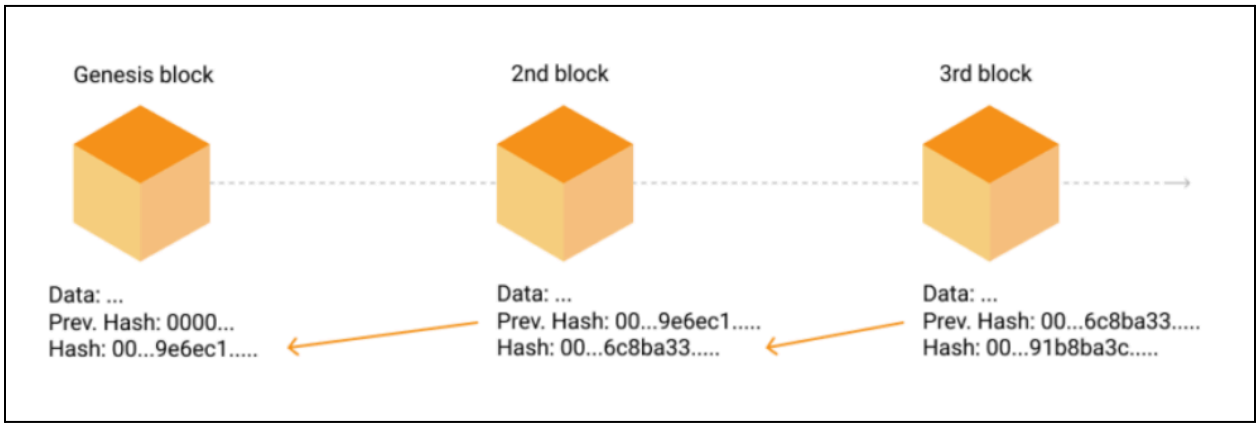
A blockchain consists of a sequence of **blocks**, where each block contains:

- A list of transactions or data
- A timestamp indicating when the block was created
- A cryptographic hash of the previous block
- A proof value generated using a consensus mechanism

Blocks are linked together using cryptographic hashes, forming a chain. Any modification to a block changes its hash, which breaks the chain, making blockchain **tamper-resistant** and **immutable**.

Key characteristics of blockchain include:

- **Decentralization**: No single entity controls the data
- **Immutability**: Once data is recorded, it cannot be altered
- **Transparency**: Transactions can be publicly verified
- **Security**: Cryptographic techniques ensure data integrity



2. Process of Mining

Mining is the process through which new blocks are added to a blockchain in a secure and decentralized manner. It is carried out using a consensus mechanism known as **Proof of Work (PoW)**, which ensures that all participating nodes agree on the state of the blockchain.

The mining process begins when a miner retrieves the **latest block** from the blockchain. This block contains a proof value and a hash of the previous block. Using this information, the miner attempts to compute a new proof value (nonce) that satisfies a predefined difficulty condition.

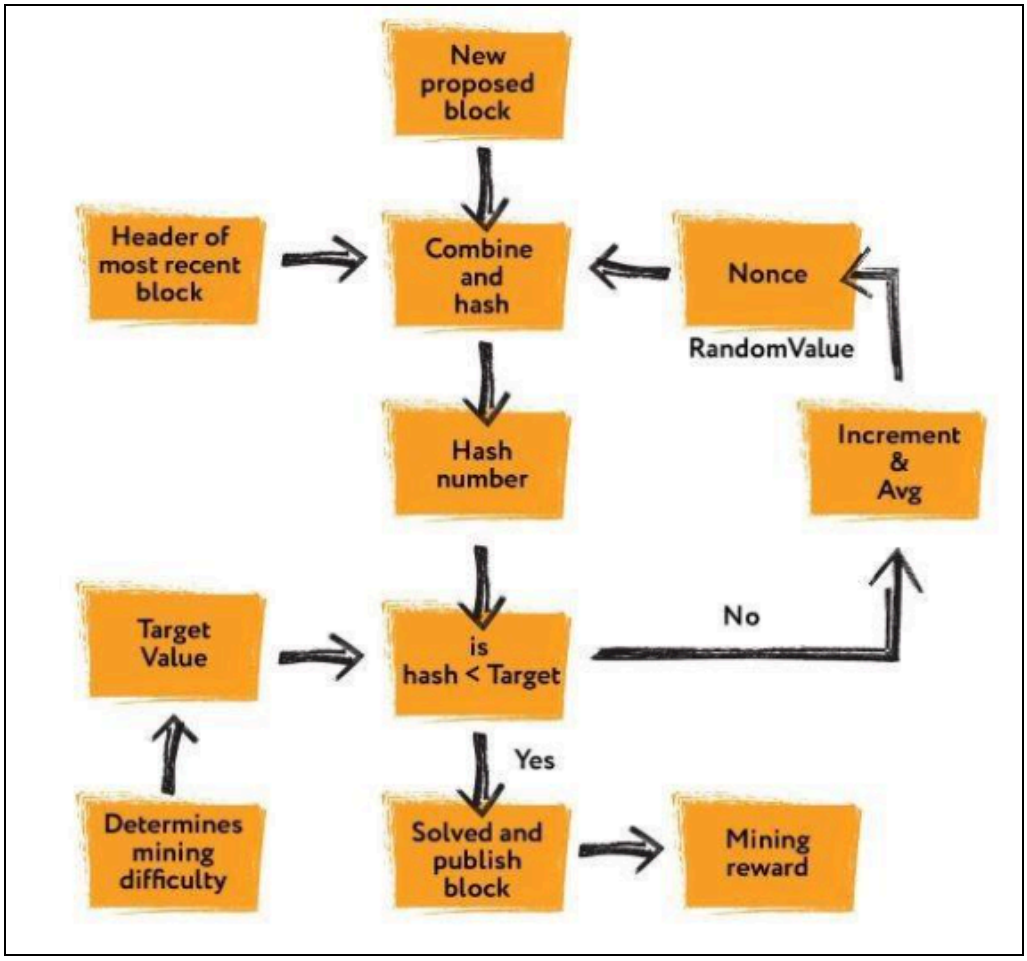
The miner repeatedly performs the following steps:

1. A candidate proof value (nonce) is selected.
2. A mathematical expression involving the new proof and the previous proof is computed.
3. The result is passed through a cryptographic hash function such as **SHA-256**.
4. The generated hash is checked against the difficulty constraint (for example, the hash must start with a certain number of leading zeros).
5. If the condition is not satisfied, the nonce is incremented and the process is repeated.

This iterative process is computationally intensive and requires significant processing power, making it difficult for malicious users to manipulate the blockchain. Once a valid proof is found, the miner creates a new block containing:

- Block index
- Timestamp
- Proof value
- Hash of the previous block

The newly mined block is then added to the blockchain and shared with other nodes for verification. Mining ensures **network security**, **data integrity**, and **consensus**, while also preventing attacks such as double spending.



3. How to Check the Validity of Blocks in a Blockchain

Blockchain validation is the process of verifying that all blocks in the chain are authentic and that the data has not been altered. Validation is crucial for maintaining trust in a decentralized system.

The validity of a blockchain is checked using the following mechanisms:

1. Previous Hash Verification

Each block stores the cryptographic hash of the previous block. During validation, the hash of the previous block is recalculated and compared with the stored `previous_hash` value. If both values match, the linkage between blocks is confirmed. Any mismatch indicates data tampering.

2. Proof of Work Verification

For every block, the proof value is verified by recomputing the hash operation used during mining. The blockchain is considered valid only if the recomputed hash satisfies the difficulty condition (such as leading zeros). This ensures that each block was mined legitimately.

3. Sequential Integrity Check

Blocks must appear in the correct order, starting from the genesis block. Each block must reference only the immediately preceding block, ensuring continuity of the chain.

4. Genesis Block Validation

The first block of the blockchain, known as the genesis block, is checked separately. It must have a fixed previous hash (usually “0”) and a predefined proof value.

If all these conditions are satisfied for every block in the chain, the blockchain is considered **valid and trustworthy**. If any single check fails, the blockchain is declared invalid, indicating possible corruption or unauthorized modification.

Program & Output:

1. **Installing flask**

```
prajj@_pep MINGW64 ~/Desktop/College/sem-8/Blockchain Lab (main)
$ pip install flask==2.2.5
Collecting flask==2.2.5
  Downloading Flask-2.2.5-py3-none-any.whl.metadata (3.9 kB)
Collecting Werkzeug>=2.2.2 (from flask==2.2.5)
  Downloading werkzeug-3.1.5-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: Jinja2>=3.0 in c:\python313\lib\site-packages (from flask==2.2.5) (3.1.6)
Collecting itsdangerous>=2.0 (from flask==2.2.5)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting click>=8.0 (from flask==2.2.5)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: colorama in c:\python313\lib\site-packages (from click>=8.0->flask==2.2.5) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in c:\python313\lib\site-packages (from Jinja2>=3.0->flask==2.2.5) (3.0.3)
Downloading Flask-2.2.5-py3-none-any.whl (101 kB)
Downloading click-8.3.1-py3-none-any.whl (108 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading werkzeug-3.1.5-py3-none-any.whl (225 kB)
Installing collected packages: Werkzeug, itsdangerous, click, flask
Successfully installed Werkzeug-3.1.5 click-8.3.1 flask-2.2.5 itsdangerous-2.2.0
```

2. Running the script

```
prajj@_pep MINGW64 ~/Desktop/College/sem-8/Blockchain Lab (main)
$ python blockchain.py
 * Serving Flask app 'blockchain'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.31.144:5000
Press CTRL+C to quit
127.0.0.1 - - [30/Jan/2026 22:07:50] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 22:08:04] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 22:08:41] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 22:10:39] "GET /mine_chain HTTP/1.1" 404 -
127.0.0.1 - - [30/Jan/2026 22:10:50] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 22:10:55] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 22:11:06] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 22:12:30] "GET /is_valid HTTP/1.1" 200 -
```

3. The python script

```
# ------------------------------------------------------------

# Required installation:(run cmd as administrator)
# pip install flask==2.2.5

# Importing libraries
import datetime      # To generate timestamps for blocks
import hashlib       # To generate SHA256 hashes
import json          # To convert block data into JSON
from flask import Flask, jsonify    # To create API endpoints

# ------------------------------------------------------------
# Part 1 - Building the Blockchain
# ------------------------------------------------------------

class Blockchain:

    def __init__(self):
        # This list will store the entire chain of blocks
        self.chain = []

        # Create the genesis block (first block)
        # proof = 1 → arbitrary
        # previous_hash = '0' → since no previous block exists
        self.create_block(proof=1, previous_hash='0')
```

```
    def create_block(self, proof, previous_hash):
        """
        Creates a new block and adds it to the chain.
        Each block contains:
        - index
        - timestamp
        - proof (PoW output)
        - previous block hash
        """
        block = {
            'index': len(self.chain) + 1,              # Position of block in chain
            'timestamp': str(datetime.datetime.now()), # Current timestamp
            'proof': proof,                            # PoW result
            'previous_hash': previous_hash             # Hash of the previous block
        }

        # Add block to the chain
        self.chain.append(block)
        return block

    def get_previous_block(self):
        """
        Returns the last block in the chain.
        """
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        """
        Simple Proof of Work (PoW) algorithm:
        - Find a number (new_proof)
        - Such that the SHA256 hash of (new_proof^2 - previous_proof^2)
          starts with '0000'

        This is a computational puzzle to secure the network.
        """
        new_proof = 1
        check_proof = False

        while check_proof is False:
            # Cryptographic puzzle: hash difference of squares
            hash_operation = hashlib.sha256(
                str(new_proof**2 - previous_proof**2).encode()
            ).hexdigest()
```

```
            # Check if hash begins with 4 leading zeros
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1

        return new_proof

    def hash(self, block):
        """
        Creates a SHA256 hash of a block.
        - Sort keys to ensure consistent hashing
        """
        encoded_block = json.dumps(block, sort_keys=True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        """
        Validates the blockchain by checking:
        1. The previous_hash field matches the actual hash of the previous block.
        2. The PoW condition (hash starting with '0000') is satisfied for each block.
        """
        previous_block = chain[0]   # Genesis block
        block_index = 1             # Start checking from block 2

        while block_index < len(chain):
            block = chain[block_index]

            # Check previous hash correctness
            if block['previous_hash'] != self.hash(previous_block):
                return False

            # Validate Proof of Work
            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(
                str(proof**2 - previous_proof**2).encode()
            ).hexdigest()

            if hash_operation[:4] != '0000':
                return False

            # Move to next block
            previous_block = block
```

```
            block_index += 1

        return True


# ---------------------------------------------------------
# Part 2 - Mining our Blockchain (Flask API)
# ---------------------------------------------------------

# Initialize Flask web application
app = Flask(__name__)

# Create an instance of the Blockchain
blockchain = Blockchain()

# ---------------------------------------------------------
# Home Route
# ---------------------------------------------------------
@app.route('/', methods=['GET'])
def home():
    response = {
        'message': 'Welcome to the Blockchain API',
        'endpoints': {
            '/mine_block': 'Mine a new block',
            '/get_chain': 'Get full blockchain',
            '/is_valid': 'Check blockchain validity'
        }
    }
    return jsonify(response), 200


# ---------------------------------------------------------
# Favicon handler (optional, avoids 404 log spam)
# ---------------------------------------------------------
@app.route('/favicon.ico')
def favicon():
    return '', 204


# ---------------------------------------------------------
# API Endpoint: Mine a new block
# ---------------------------------------------------------
@app.route('/mine_block', methods=['GET'])
def mine_block():
```

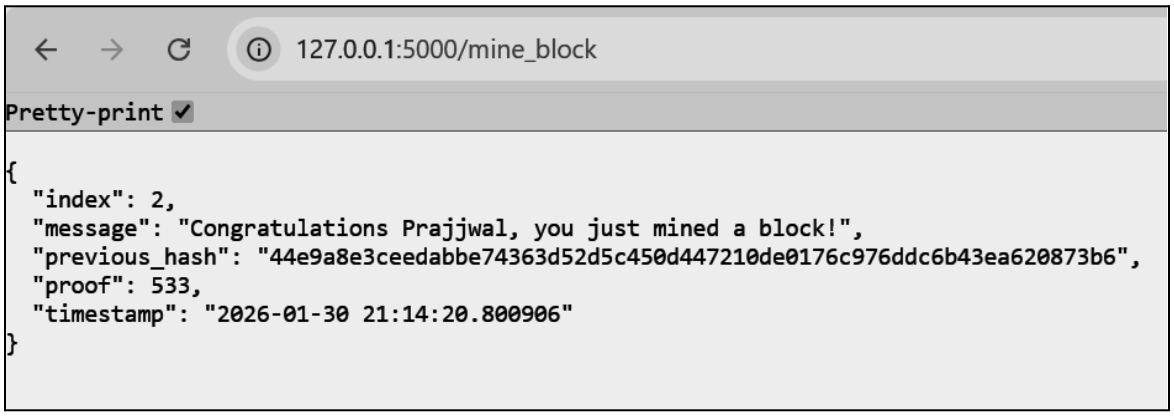
```
    # Get the previous block
    previous_block = blockchain.get_previous_block()

    # Extract previous proof
    previous_proof = previous_block['proof']

    # Run Proof of Work algorithm
    proof = blockchain.proof_of_work(previous_proof)

    # Get hash of previous block
    previous_hash = blockchain.hash(previous_block)

    # Create the new block
    block = blockchain.create_block(proof, previous_hash)

    # Prepare response
    response = {
        'message': 'Congratulations Prajjwal, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash']
    }
    return jsonify(response), 200


# ------------------------------------------------------------
# API Endpoint: Get the full blockchain
# ------------------------------------------------------------
@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }
    return jsonify(response), 200


# ------------------------------------------------------------
# API Endpoint: Check if blockchain is valid
# ------------------------------------------------------------
@app.route('/is_valid', methods=['GET'])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
```

```
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}

    return jsonify(response), 200


# ---------------------------------------------------------
# Run the Flask app
# ---------------------------------------------------------
app.run(host='0.0.0.0', port=5000)
```

4. Output

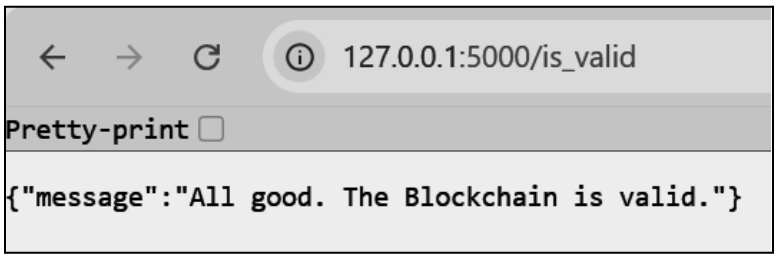
4.1 home url

4.2 /mine_block



4.3 /get_chain

4.4 /is_valid



Conclusion: In this experiment, a basic blockchain was successfully implemented using Python. The creation of blocks, mining using a Proof of Work algorithm, and validation of the blockchain demonstrated the core principles of blockchain technology. The experiment highlighted how cryptographic hashing, consensus mechanisms, and block linking work together to ensure data integrity, security, and immutability. This implementation provides a foundational understanding of how real-world blockchain systems operate.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

Blockchain Exp - 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
   Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

6. Chain Replacement

When /replace_chain is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints
  pip install Flask
- **Requests** – For HTTP communication between nodes
  pip install requests==2.18.4
- **Postman** – For testing API requests
- Python Standard Libraries:
  - datetime
  - jsonify
  - hashlib
  - uuid4
  - urlparse
  - request

Code :

```
# Module 2 - Create a Cryptocurrency

# To be installed:
# Flask==0.12.2: pip install Flask==0.12.2
# Postman HTTP Client: https://www.getpostman.com/
# requests==2.18.4: pip install requests==2.18.4

# Importing the libraries
import datetime
import hashlib
import json
from flask import Flask, jsonify, request
import requests
from uuid import uuid4                                  # Generate a unique id that is in hex
from urllib.parse import urlparse                       # To parse url of the nodes

# Part 1 - Building a Blockchain

class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = []                          # Adding transactions before they are
added to a block
        self.create_block(proof = 1, previous_hash = '0')
        self.nodes = set()                              # Set is used as there is no order to be
maintained as the nodes can be from all around the globe

    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                 'timestamp': str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash': previous_hash,
                 'transactions': self.transactions}     # Adding transactions to make the
blockchain a cryptocurrency
        self.transactions = []                          # The list of transacction should become
empty after they are added to a block
        self.chain.append(block)
        return block
```

```
    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys = True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            if block['previous_hash'] != self.hash(previous_block):
                return False
            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(str(proof**2 - previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] != '0000':
                return False
            previous_block = block
            block_index += 1
        return True

    # This method will add the trqnsaction to the list of trnsactions
    def add_transaction(self, sender, receiver, amount):
        self.transactions.append({'sender': sender,
                                  'receiver': receiver,
                                  'amount': amount})
        previous_block = self.get_previous_block()
```

```
        return previous_block['index'] + 1                          # It will return the block index to
which the transaction should be added

    # This function will add the node containing an address to the set of nodes created in init
function
    def add_node(self, address):
        parsed_url = urlparse(address)                              # urlparse will parse the url from the
address
        self.nodes.add(parsed_url.netloc)                           # Add is used and not append as it's
a set. Netloc will only return '127.0.0.1:5000'

    # Consensus Protocol. This function will replace all the shorter chain with the longer chain in
all the nodes on the network
    def replace_chain(self):
        network = self.nodes                                        # network variable is the set of nodes all
around the globe
        longest_chain = None                                        # It will hold the longest chain when we
scan the network
        max_length = len(self.chain)                                # This will hold the length of the chain
held by the node that runs this function
        for node in network:
            response = requests.get(f'http://{node}/get_chain')     # Use get chain method
already created to  get the length of the chain
            if response.status_code == 200:
                length = response.json()['length']                  # Extract the length of the chain from
get_chain fiunction
                chain = response.json()['chain']
                if length > max_length and self.is_chain_valid(chain):      # We check if the length is
bigger and if the chain is valid then
                    max_length = length                             # We update the max length
                    longest_chain = chain                           # We update the longest chain
        if longest_chain:                                           # If longest_chain is not none that means it
was replaced
            self.chain = longest_chain                              # Replace the chain of the current node
with the longest chain
            return True
        return False                                                # Return false if current chain is the longest
one

# Part 2 - Mining our Blockchain

# Creating a Web App
```

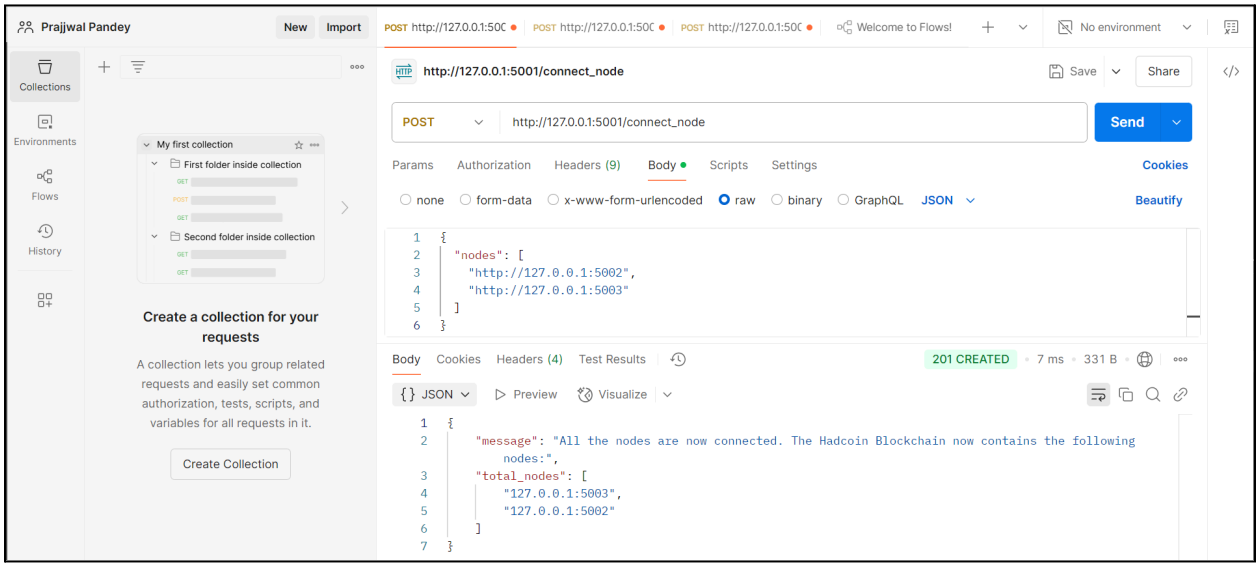
```
app = Flask(__name__)

# Creating an address for the node on Port 5000. We will create some other nodes as well on
different ports
node_address = str(uuid4()).replace('-', '')                    #

# Creating a Blockchain
blockchain = Blockchain()

# Mining a new block
@app.route('/mine_block', methods = ['GET'])
def mine_block():
   previous_block = blockchain.get_previous_block()
   previous_proof = previous_block['proof']
   proof = blockchain.proof_of_work(previous_proof)
   previous_hash = blockchain.hash(previous_block)
   blockchain.add_transaction(sender = node_address, receiver = 'Richard', amount = 1) #
Hadcoins to mine the block (A Reward). So the node gives 1 hadcoin to Abcde for mining the
block
   block = blockchain.create_block(proof, previous_hash)
   response = {'message': 'Congratulations, you just mined a block!',
               'index': block['index'],
               'timestamp': block['timestamp'],
               'proof': block['proof'],
               'previous_hash': block['previous_hash'],
               'transactions': block['transactions']}
   return jsonify(response), 200

# Getting the full Blockchain
@app.route('/get_chain', methods = ['GET'])
def get_chain():
   response = {'chain': blockchain.chain,
               'length': len(blockchain.chain)}
   return jsonify(response), 200

# Checking if the Blockchain is valid
@app.route('/is_valid', methods = ['GET'])
def is_valid():
   is_valid = blockchain.is_chain_valid(blockchain.chain)
   if is_valid:
      response = {'message': 'All good. The Blockchain is valid.'}
   else:
```

```
        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return jsonify(response), 200

# Adding a new transaction to the Blockchain
@app.route('/add_transaction', methods = ['POST'])                    # Post method as we have
to pass something to get something in return
def add_transaction():
    json = request.get_json()                                          # This will get the json file from
postman. In Postman we will create a json file in which we will pass the values for the keys in
the json file
    transaction_keys = ['sender', 'receiver', 'amount']
    if not all(key in json for key in transaction_keys):               # Checking if all keys are
available in json
        return 'Some elements of the transaction are missing', 400
    index = blockchain.add_transaction(json['sender'], json['receiver'], json['amount'])
    response = {'message': f'This transaction will be added to Block {index}'}
    return jsonify(response), 201                                      # Code 201 for creation

# Part 3 - Decentralizing our Blockchain

# Connecting new nodes
@app.route('/connect_node', methods = ['POST'])                        # POST request to register
the new nodes from the json file
def connect_node():
    json = request.get_json()
    nodes = json.get('nodes')                                          # Get the nodes from json file
    if nodes is None:
        return "No node", 400
    for node in nodes:
        blockchain.add_node(node)
    response = {'message': 'All the nodes are now connected. The Hadcoin Blockchain now
contains the following nodes:',
                'total_nodes': list(blockchain.nodes)}
    return jsonify(response), 201

# Replacing the chain by the longest chain if needed
@app.route('/replace_chain', methods = ['GET'])
def replace_chain():
    is_chain_replaced = blockchain.replace_chain()
    if is_chain_replaced:
        response = {'message': 'The nodes had different chains so the chain was replaced by the
longest one.',
```

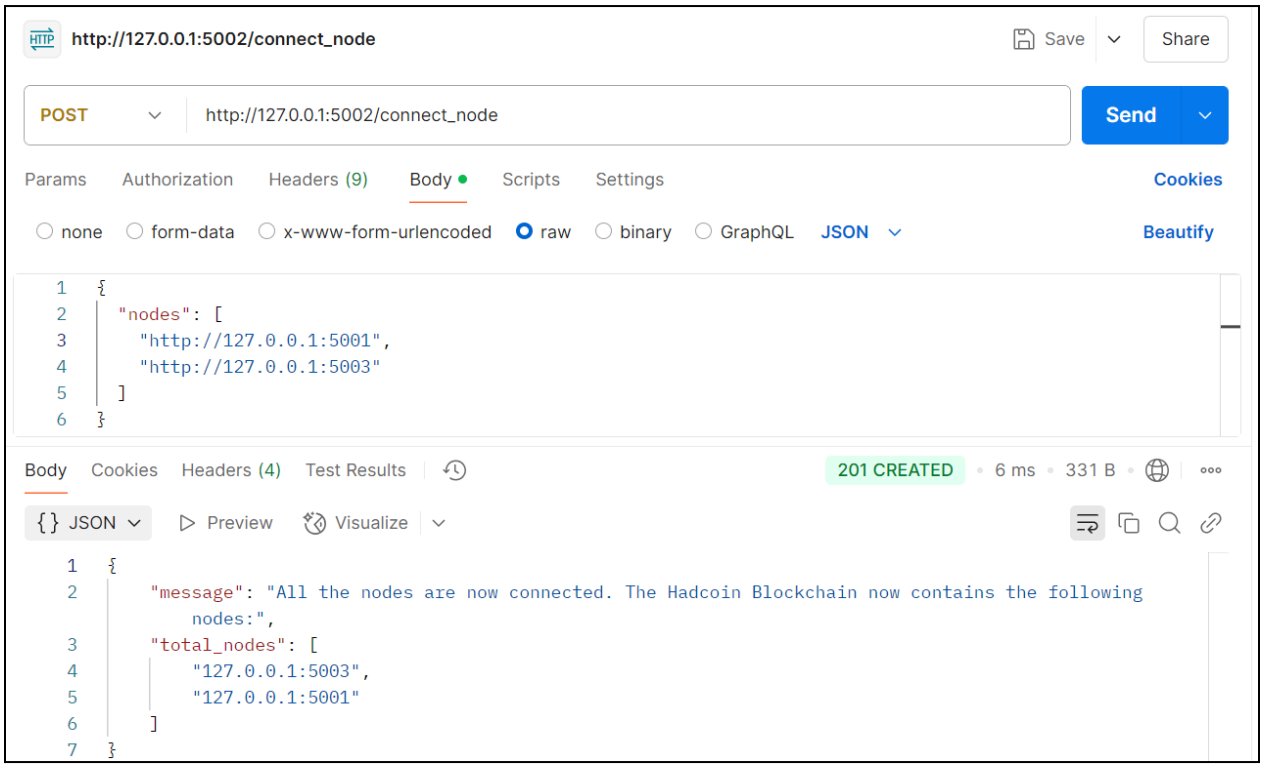
```
            'new_chain': blockchain.chain}
    else:
        response = {'message': 'All good. The chain is the largest one.',
            'actual_chain': blockchain.chain}
    return jsonify(response), 200

# Running the app
app.run(host = '0.0.0.0', port = 500*)
```

Output :

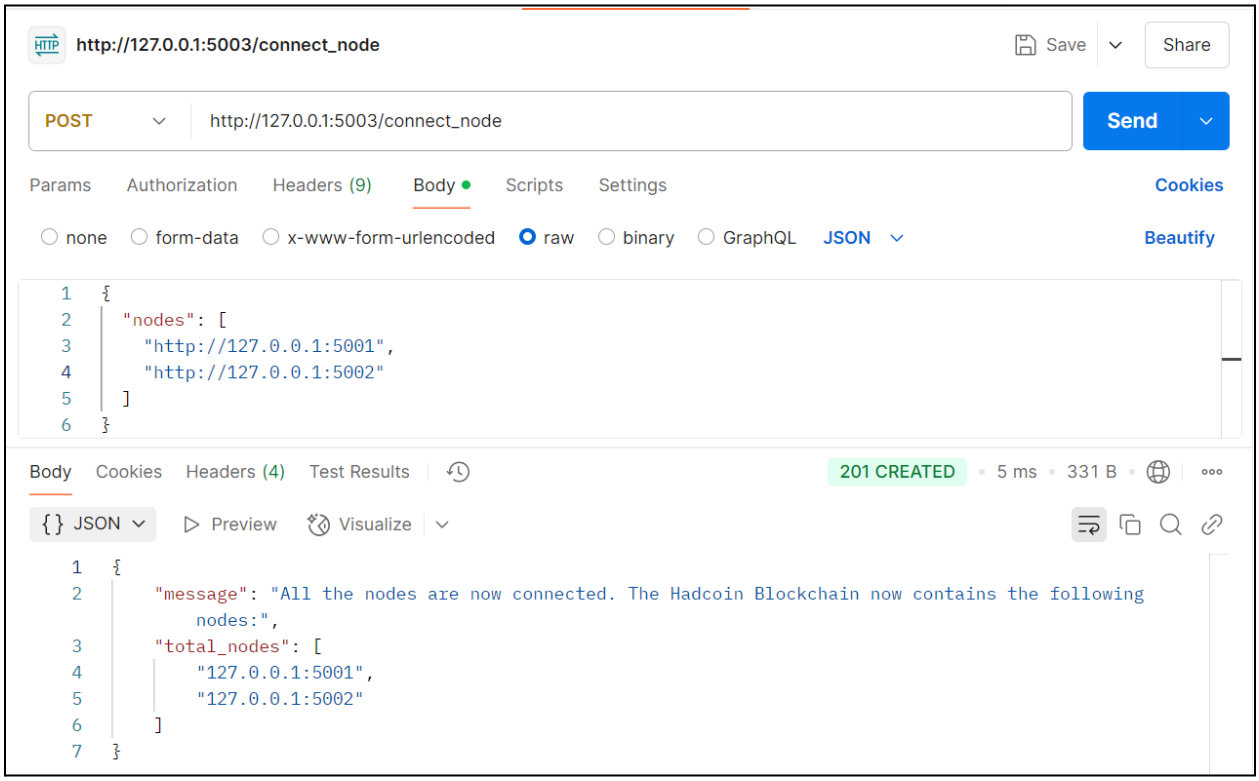
1. Connecting nodes: Node 1 -> 2,3



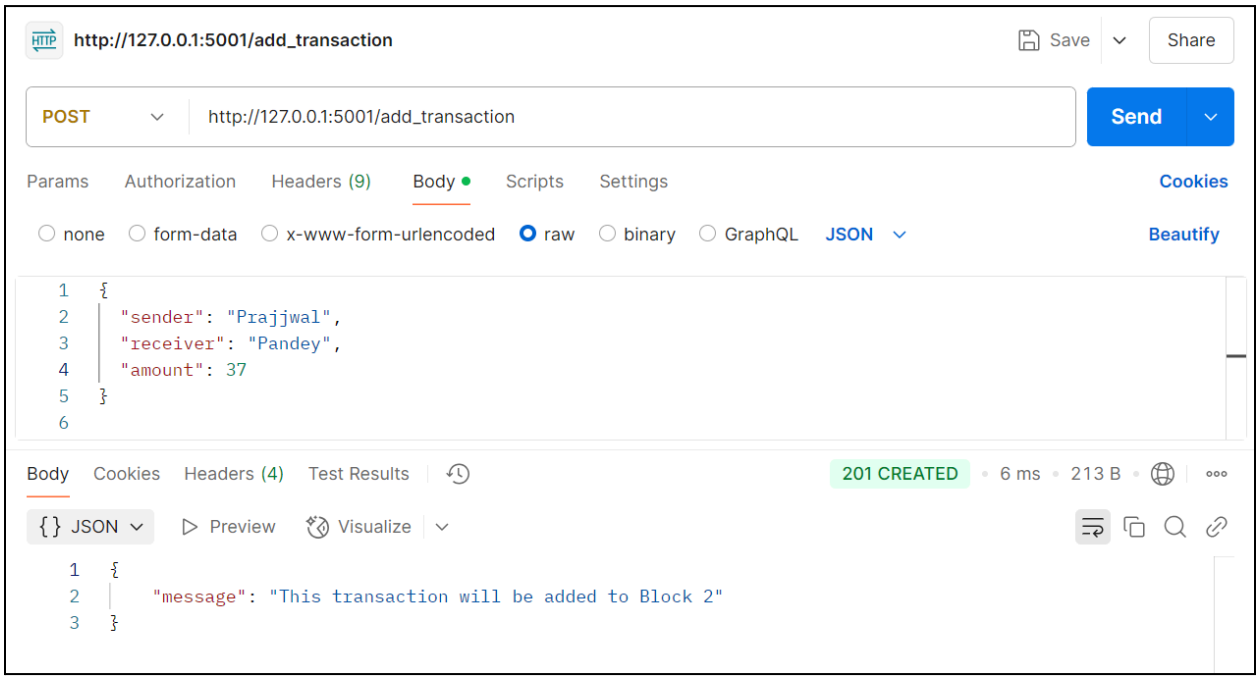
Node 2 -> 1,3



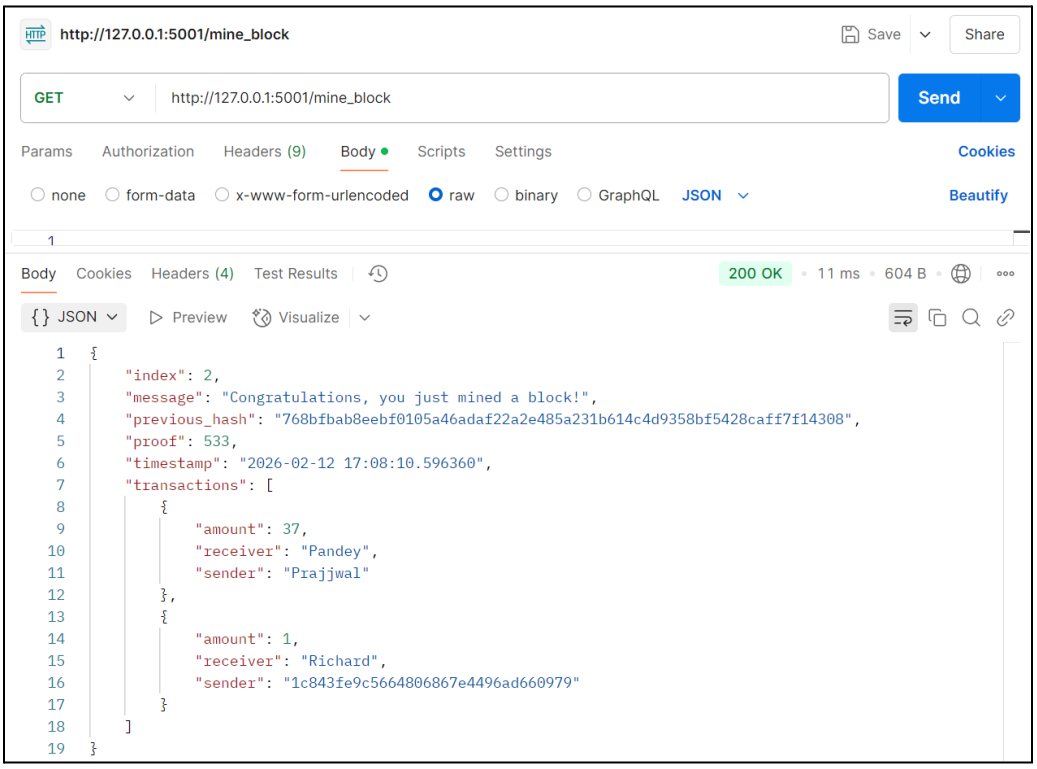
Node 3 -> 1,2



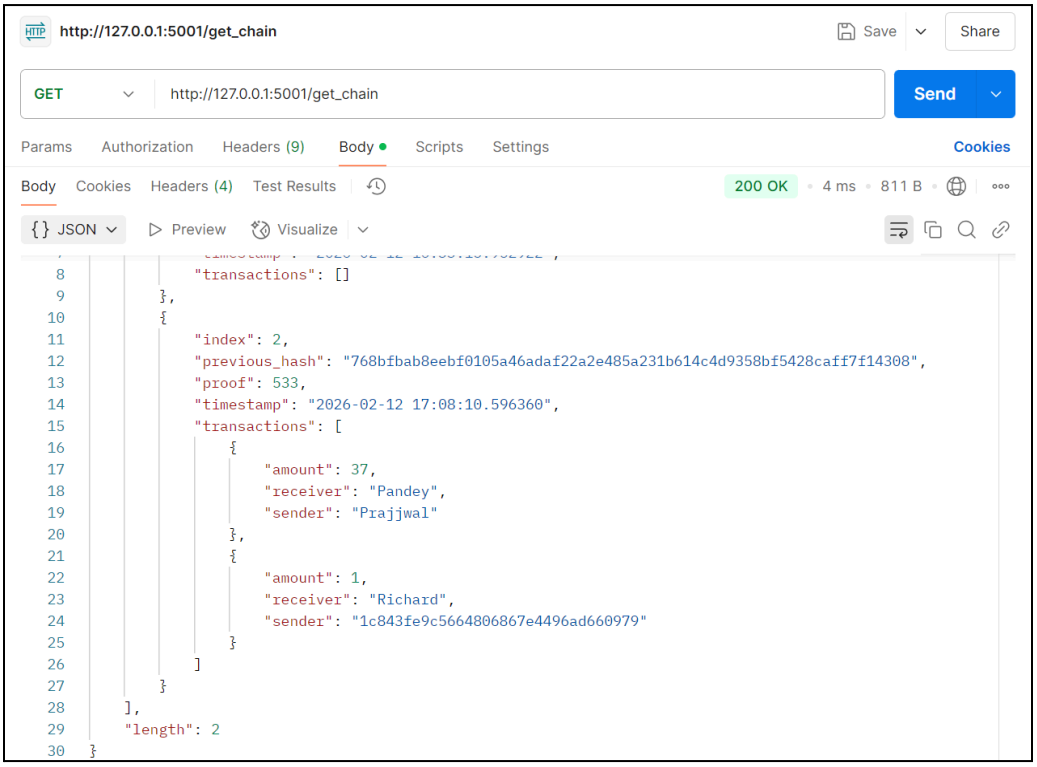
2. Adding transaction from node 1



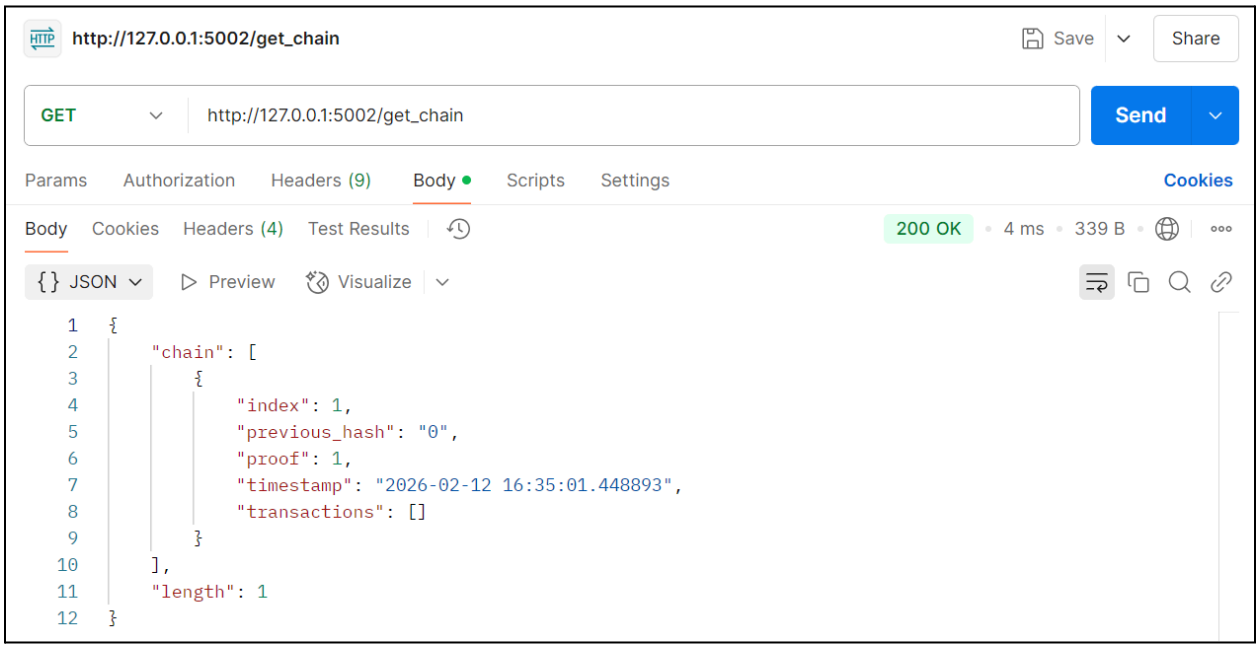
3. Mining a Block (Only on 5001)



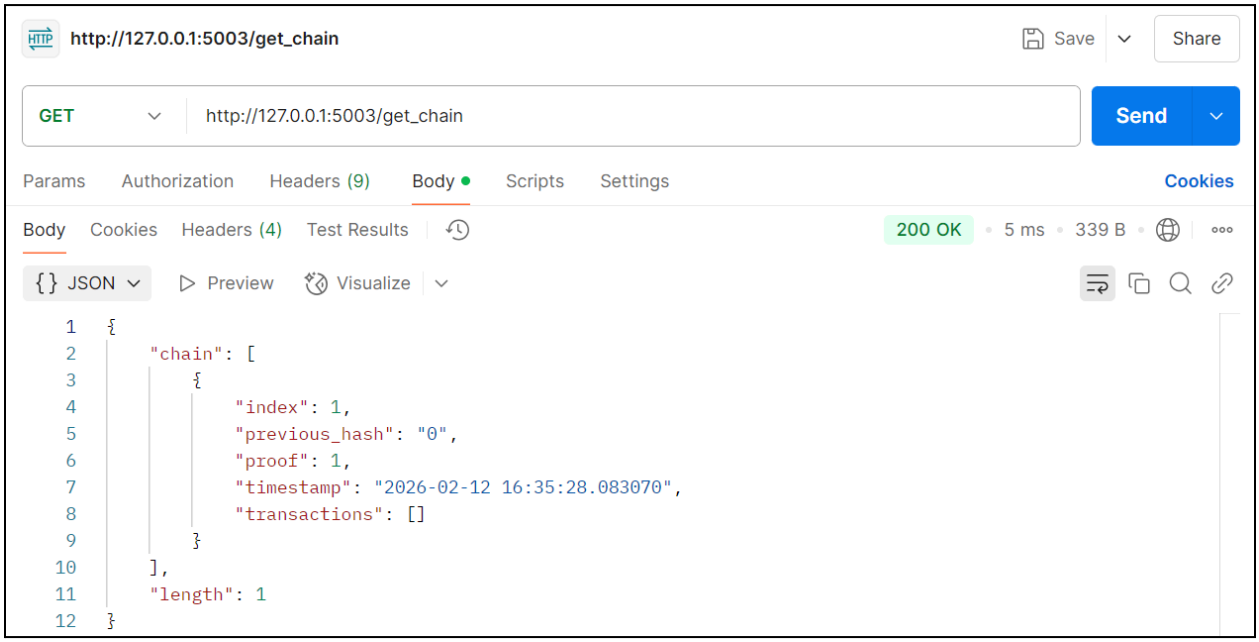
4. Checking chain length difference: Node 1



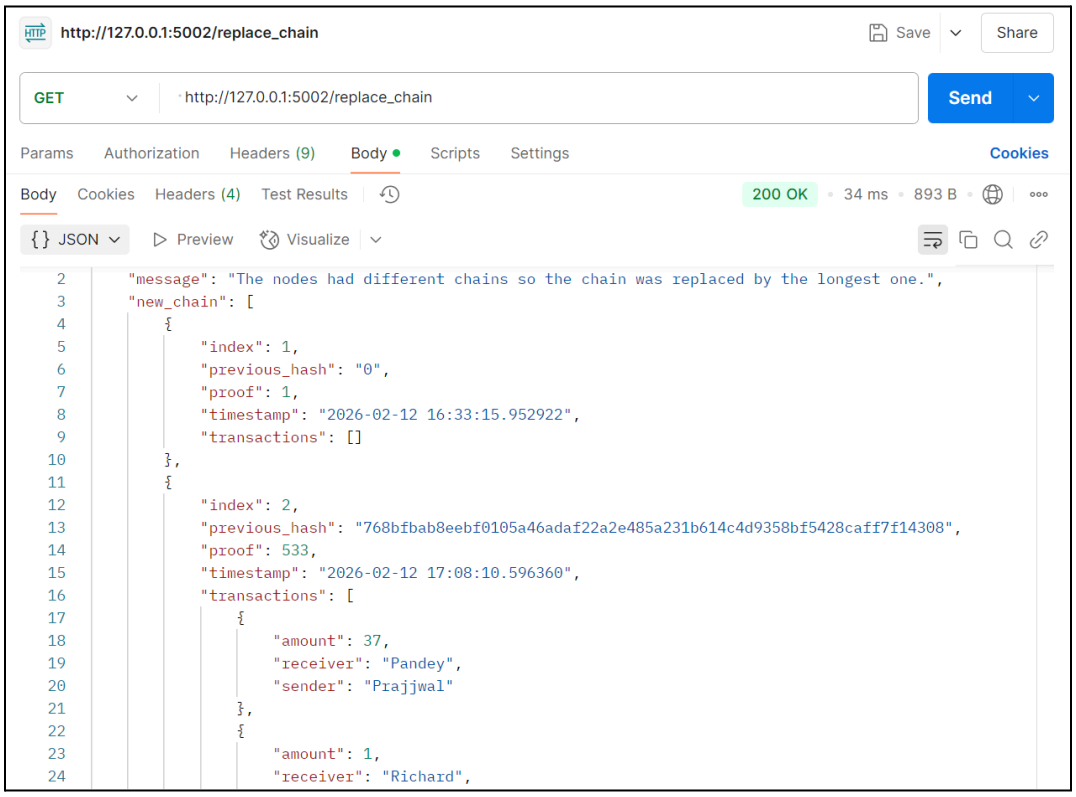
Node 2


http://127.0.0.1:5002/get_chain
Save
Share
GET
http://127.0.0.1:5002/get_chain
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Cookies
Body
Cookies
Headers (4)
Test Results
200 OK
4 ms
339 B
JSON
Preview
Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-02-12 16:35:01.448893",
"transactions": []
}
],
"length": 1
}

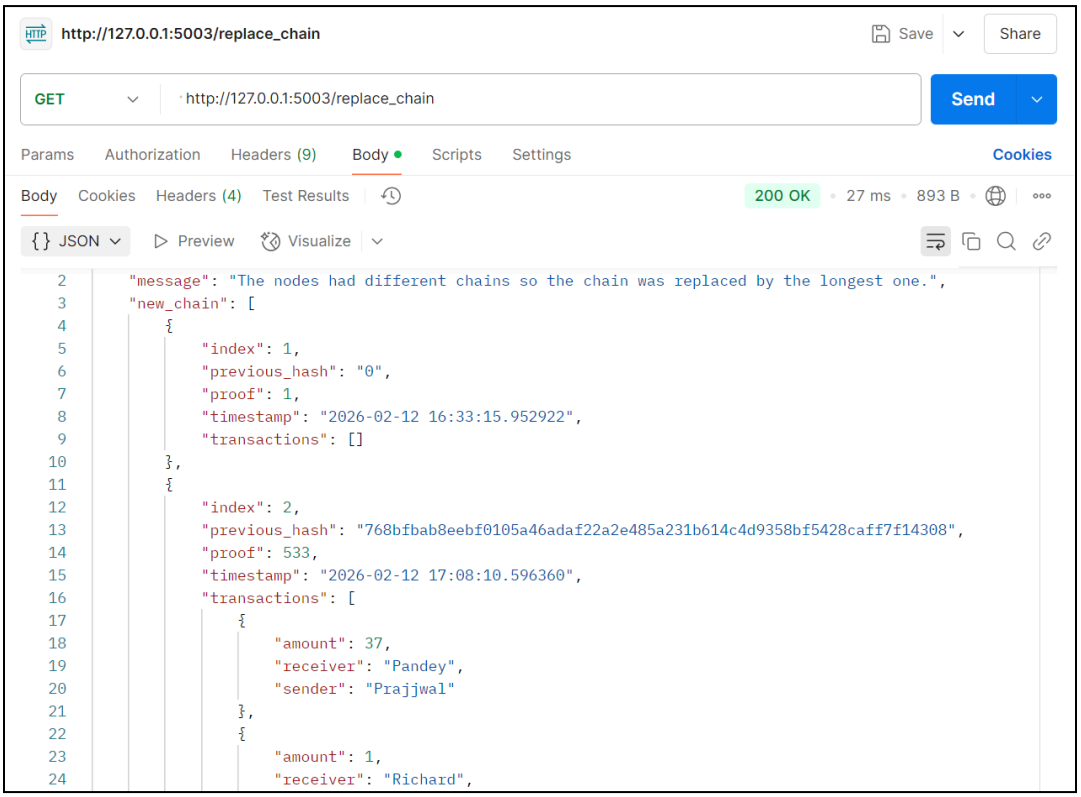
Node 3


http://127.0.0.1:5003/get_chain
Save
Share
GET
http://127.0.0.1:5003/get_chain
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Cookies
Body
Cookies
Headers (4)
Test Results
200 OK
5 ms
339 B
JSON
Preview
Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-02-12 16:35:28.083070",
"transactions": []
}
],
"length": 1
}

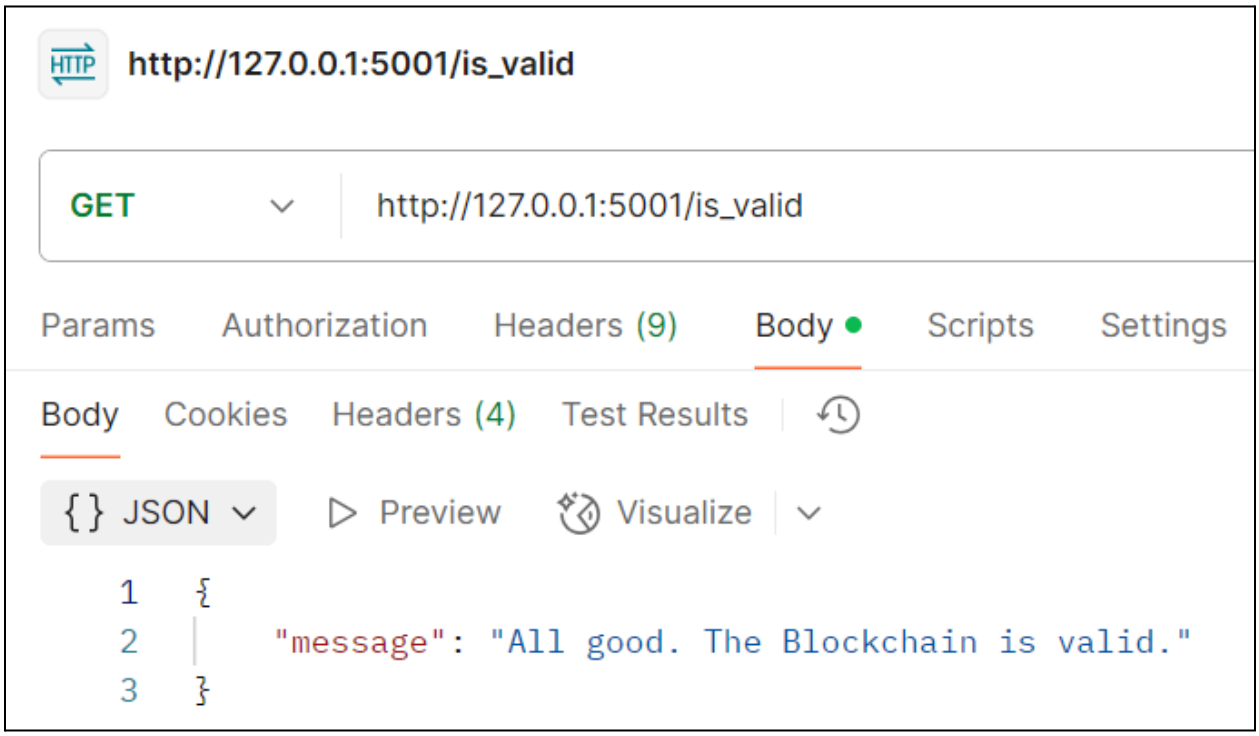
5. Replace Chain (Consensus Mechanism) : Node 2



Node 3



6. Final Validation



7. Terminal Activity

Conclusion: In this experiment, we successfully created a basic cryptocurrency using Python and Flask and implemented mining in a multi-node blockchain network. We demonstrated how transactions are added, stored temporarily, and included in a block during the mining process using a Proof-of-Work mechanism. By running three separate nodes, we simulated a peer-to-peer network and applied the Longest Chain Rule to maintain consensus across all nodes. The experiment helped us understand blockchain structure, decentralized networking, mining rewards, and chain synchronization in a practical manner.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Blockchain Exp - 4

AIM: Hands on Solidity Programming Assignments for creating Smart Contracts.

Theory:

1. **Primitive Data Types, Variables, Functions – pure, view**

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int**: unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool**: represents logical values (true or false).
- **address**: holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string**: store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure**: cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view**: can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. **Inputs and Outputs to Functions**

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:
    - public: available both inside and outside the contract.
    - private: only accessible within the same contract.
    - internal: accessible within the contract and its child contracts.
    - external: can be called only by external accounts or other contracts.

- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays**: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings**: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs**: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums**: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

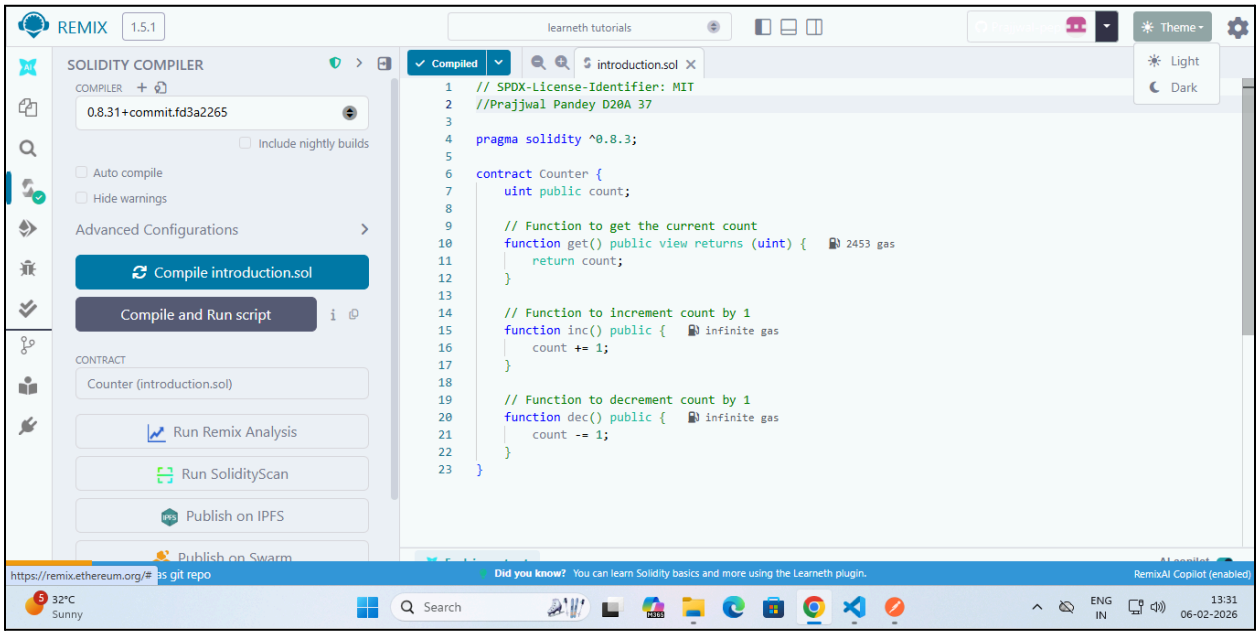
- **storage**: Data stored permanently on the blockchain. Examples: state variables.
- **memory**: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata**: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

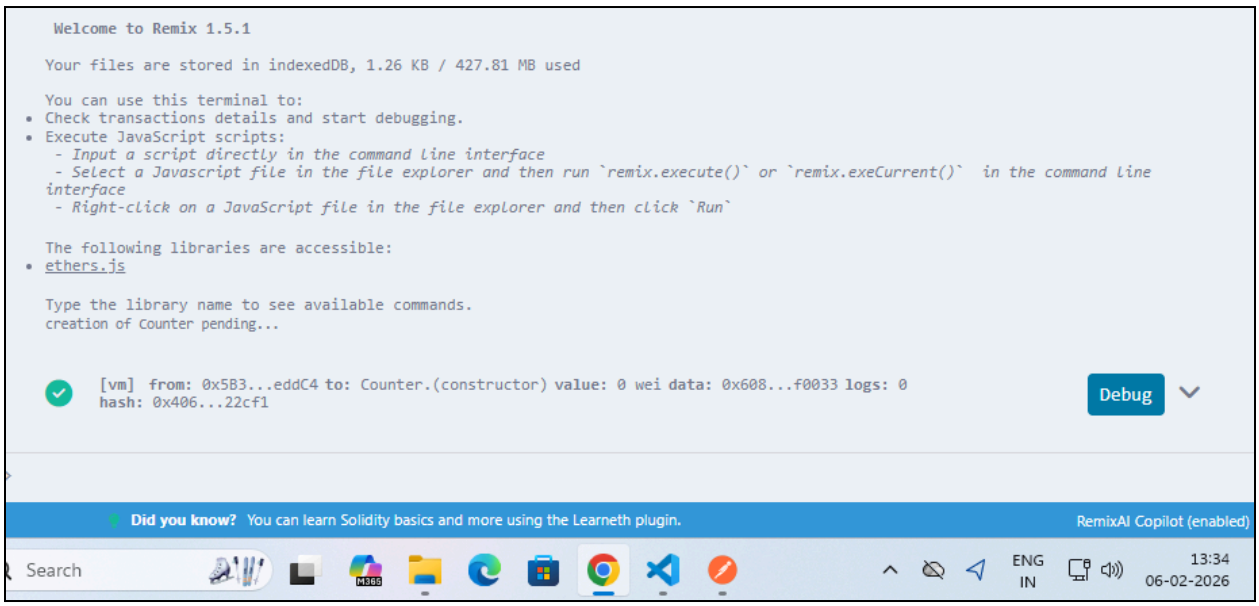
7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

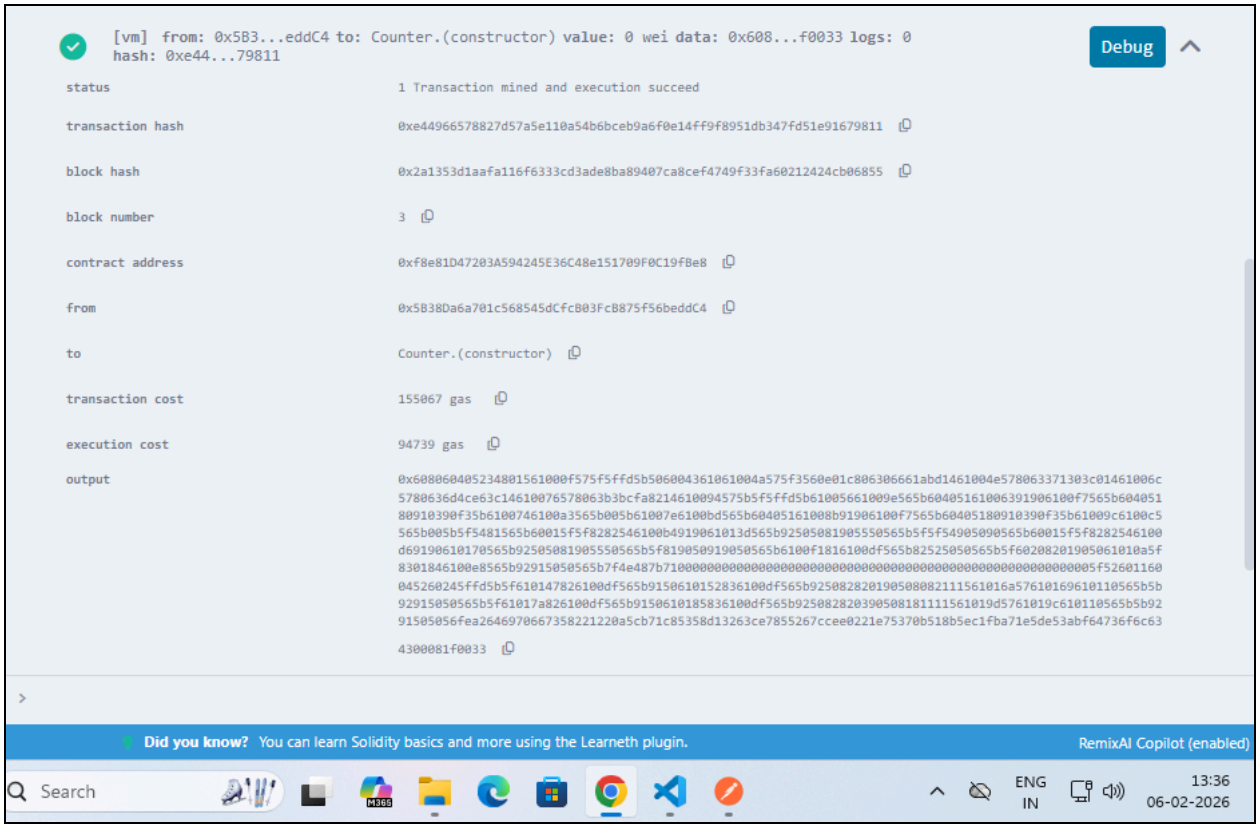
- **Ether and Wei**: Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price**: Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions**: Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Implementation:

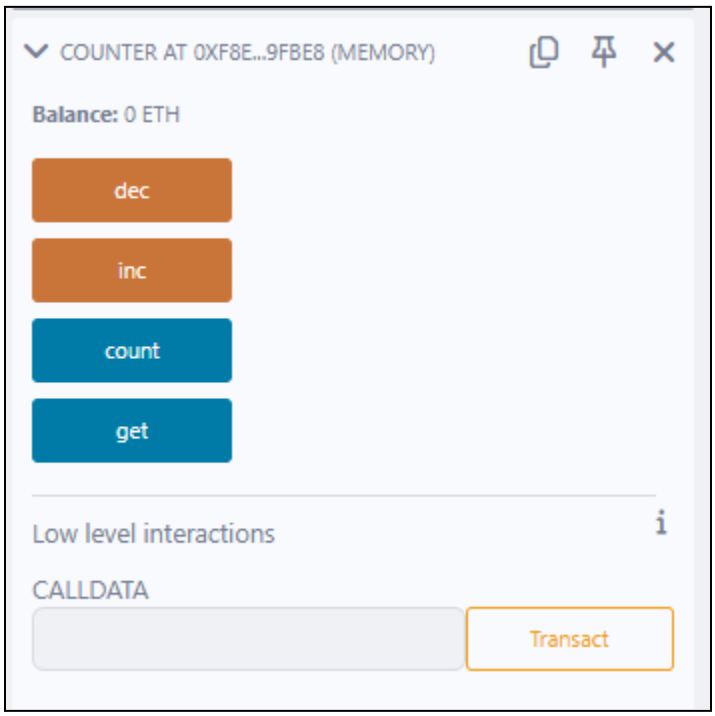
Tutorial - 1 (compile)



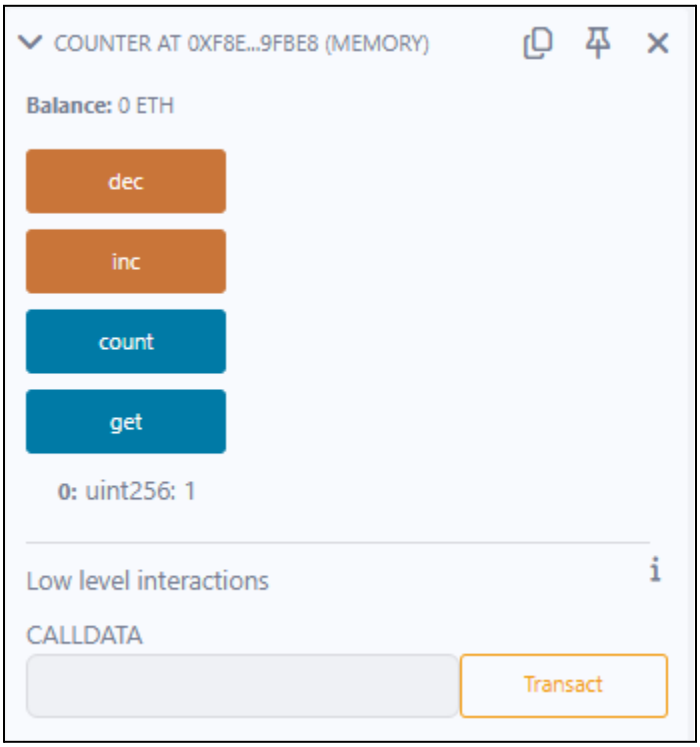




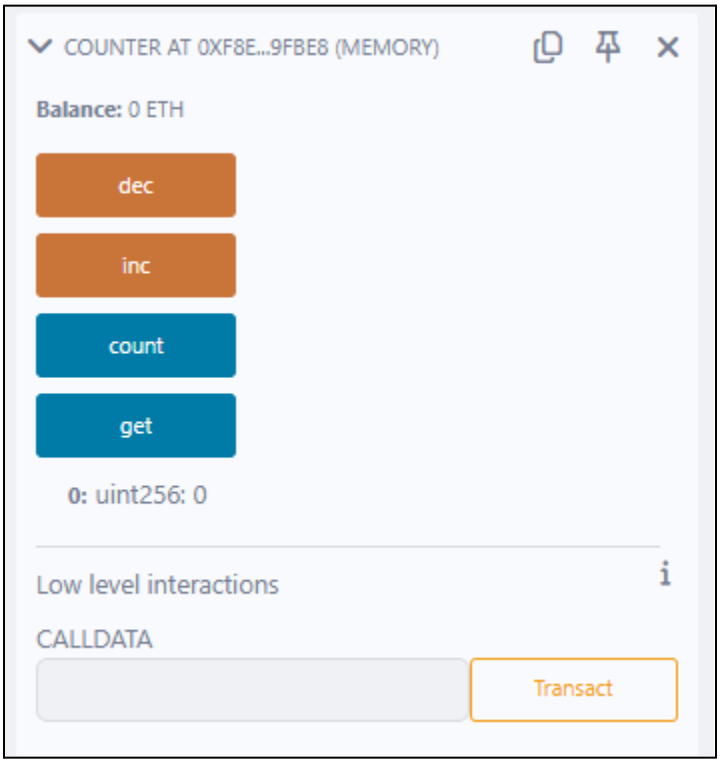
Tutorial - 1 (get)



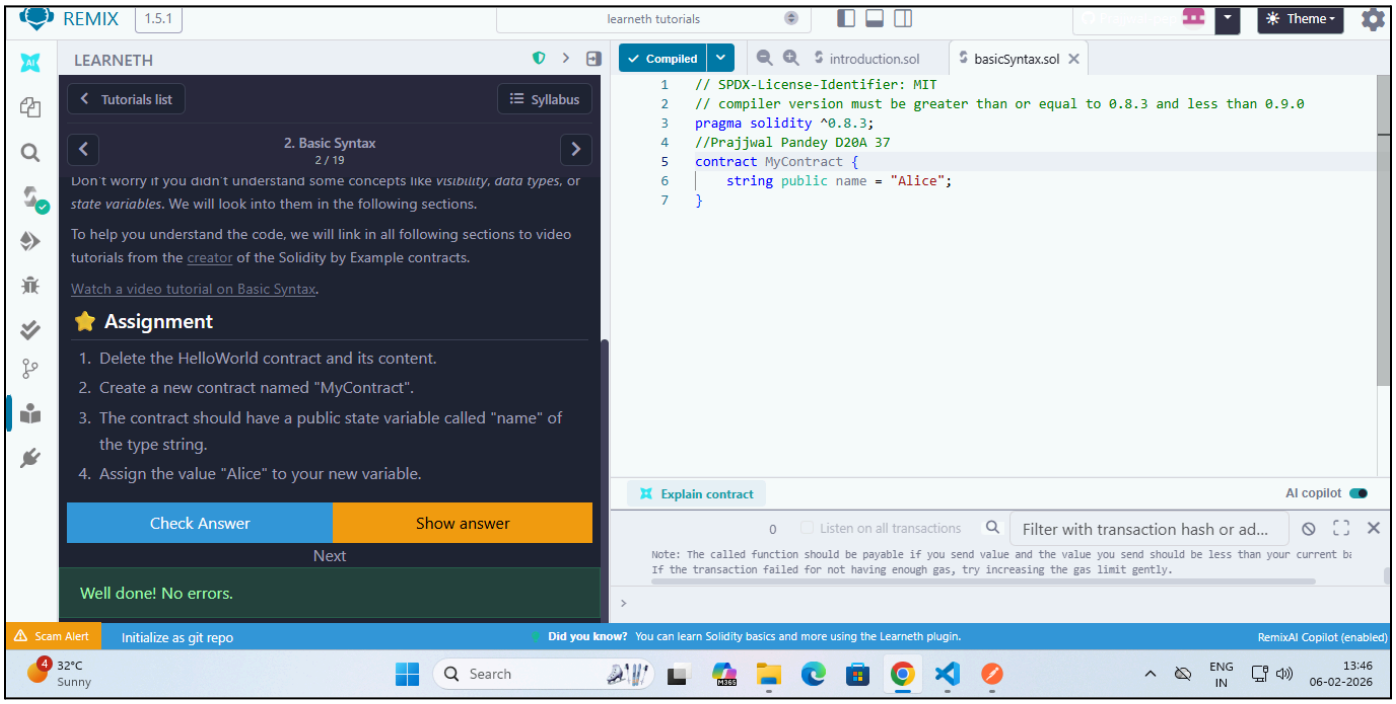
Tutorial - 1 (inc)



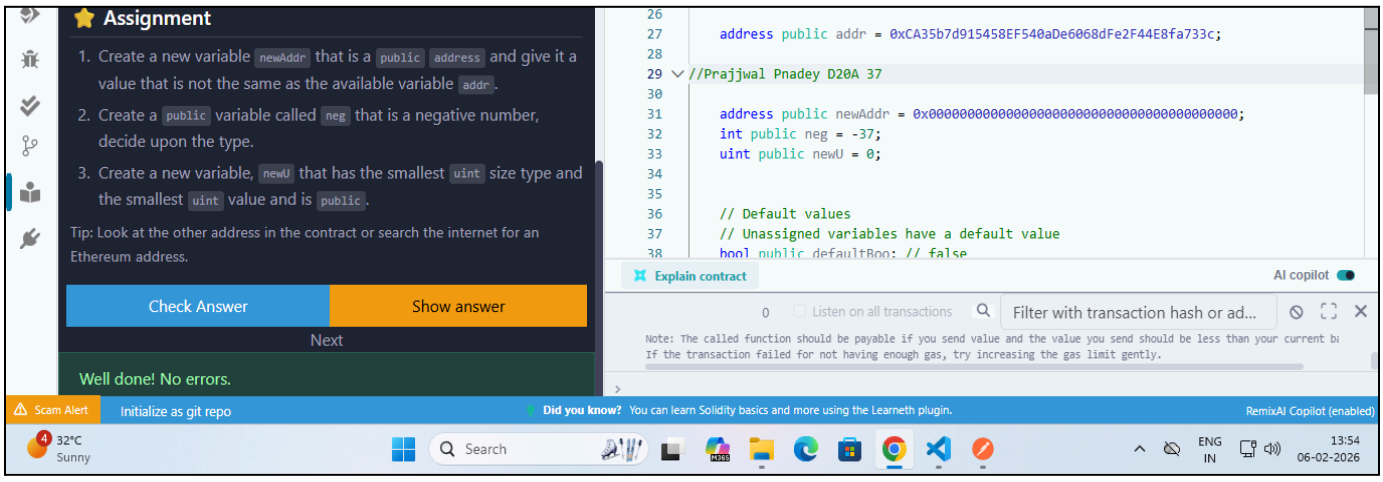
Tutorial - 1 (dec)



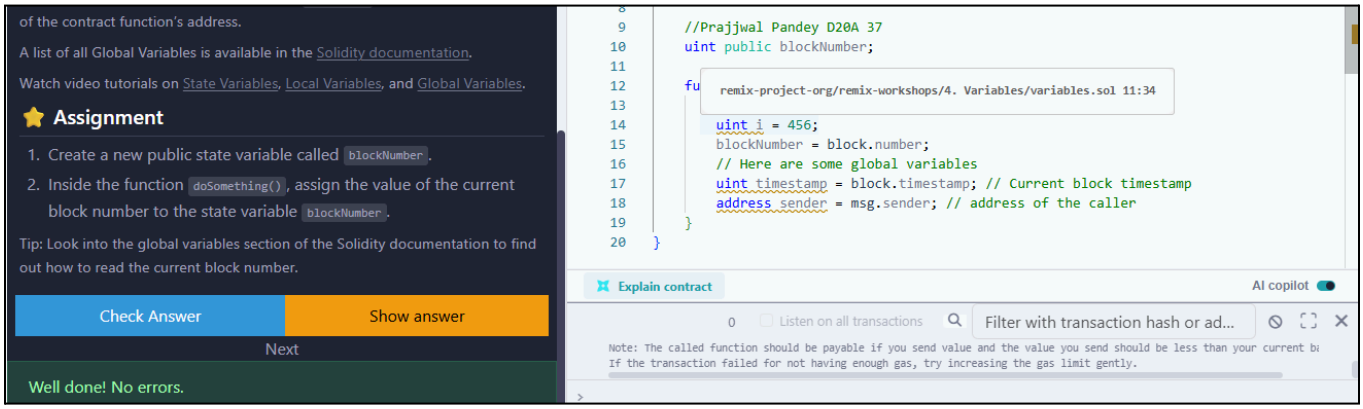
Tutorial - 2



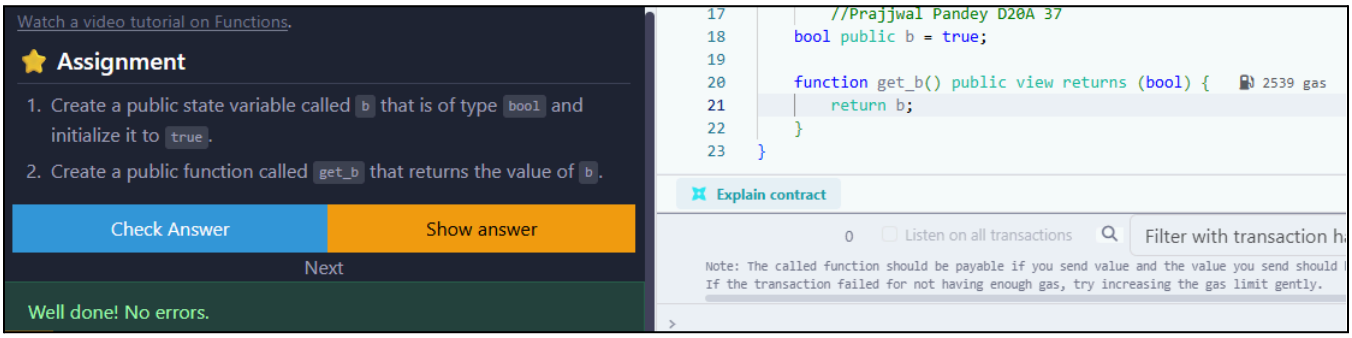
Tutorial - 3



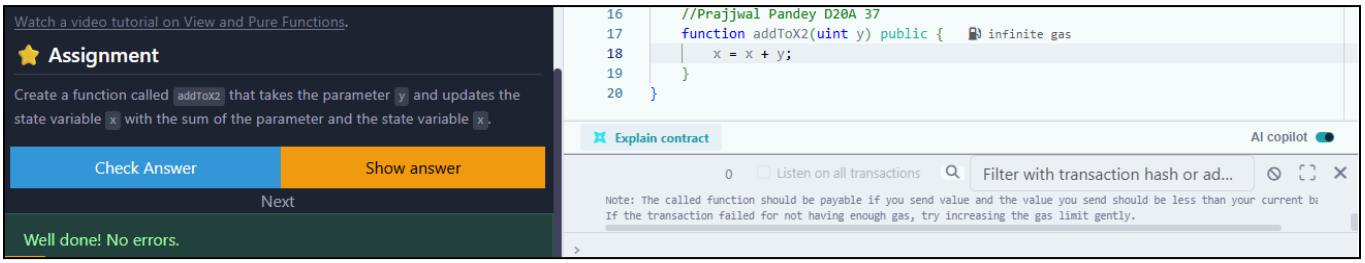
Tutorial - 4



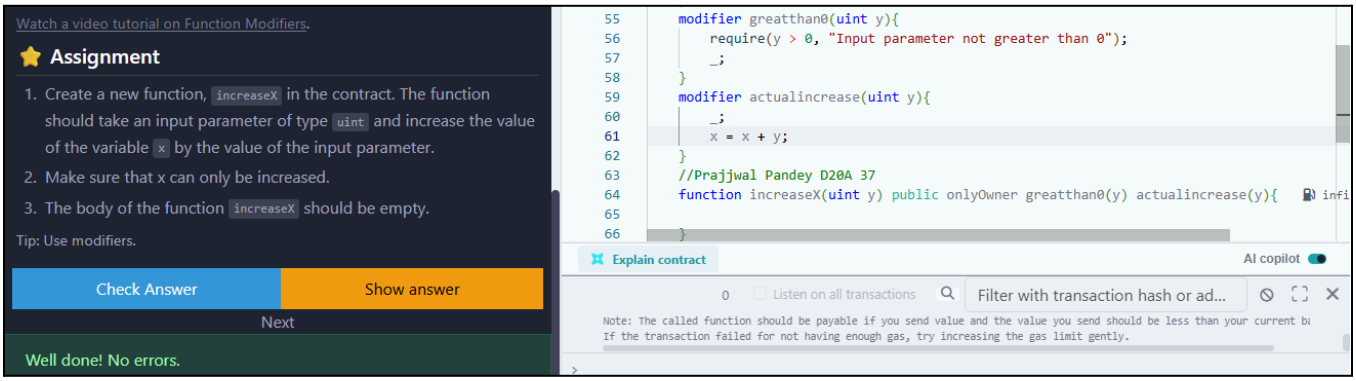
Tutorial - 5



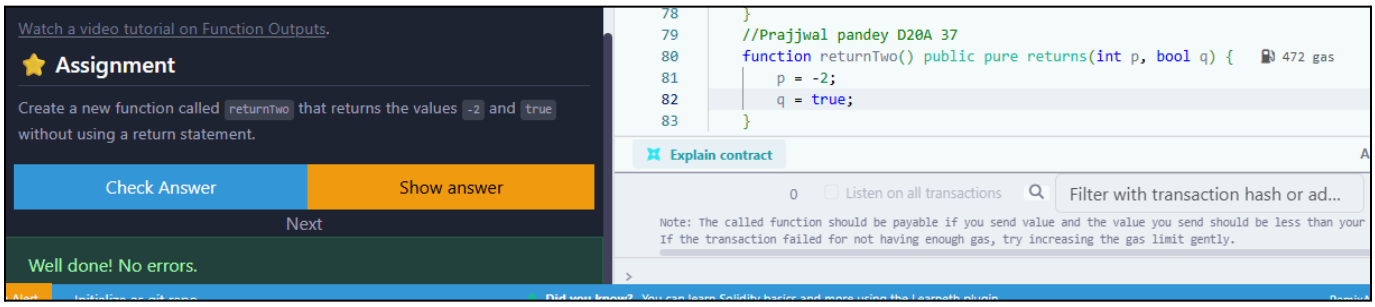
Tutorial - 6



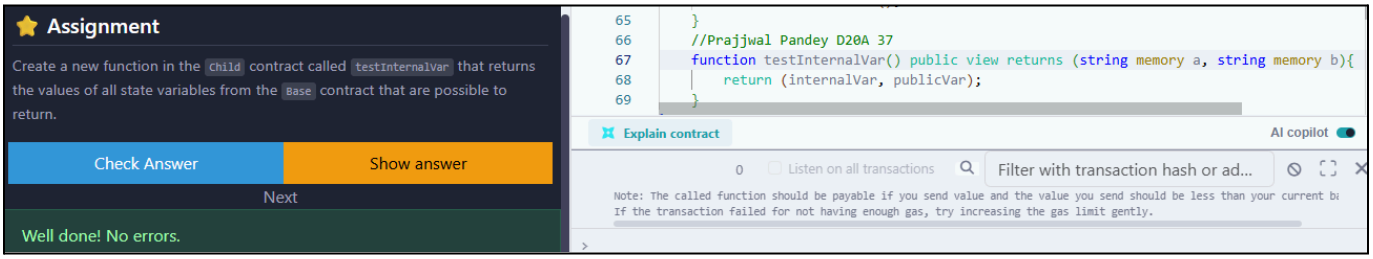
Tutorial - 7



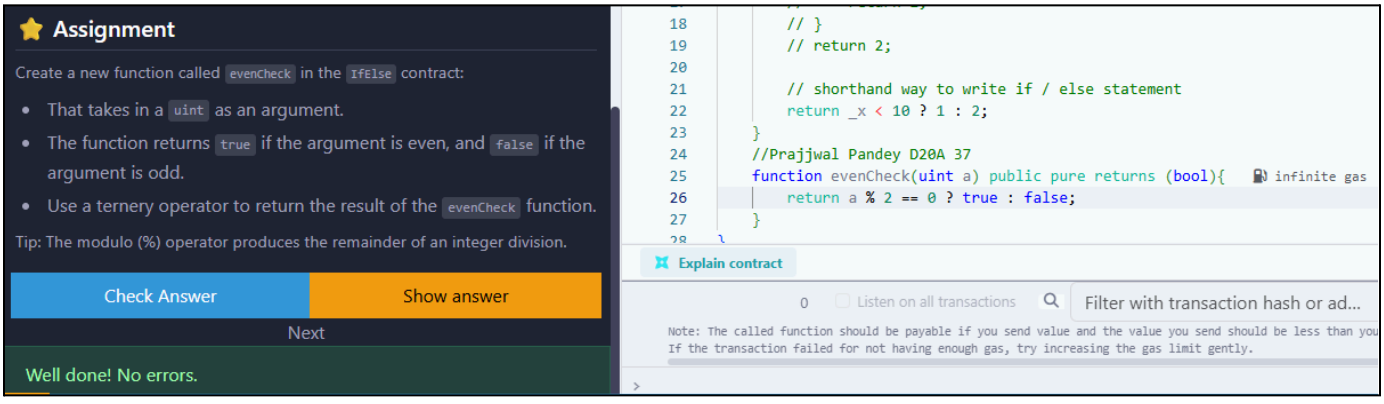
Tutorial - 8



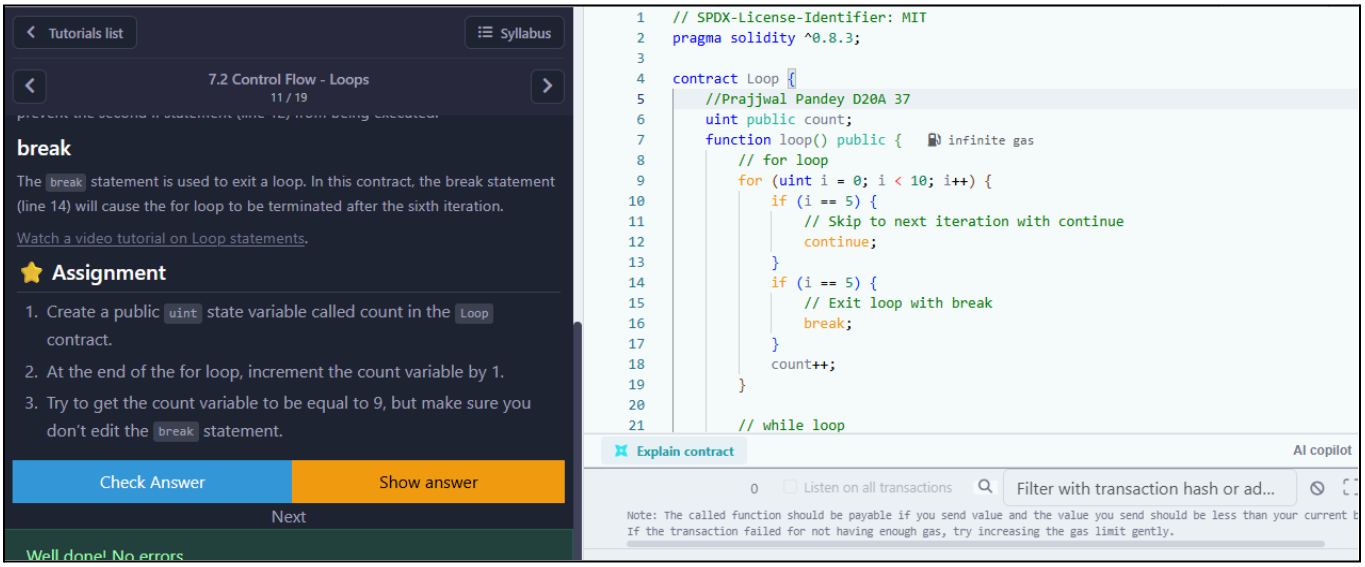
Tutorial - 9



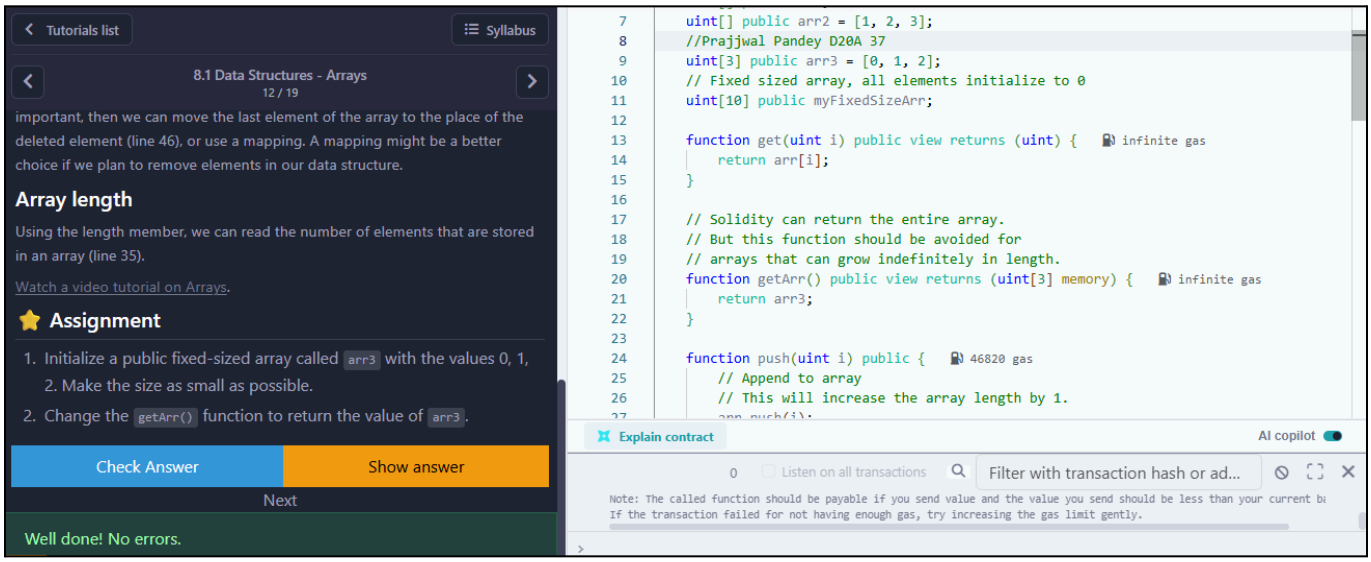
Tutorial - 10



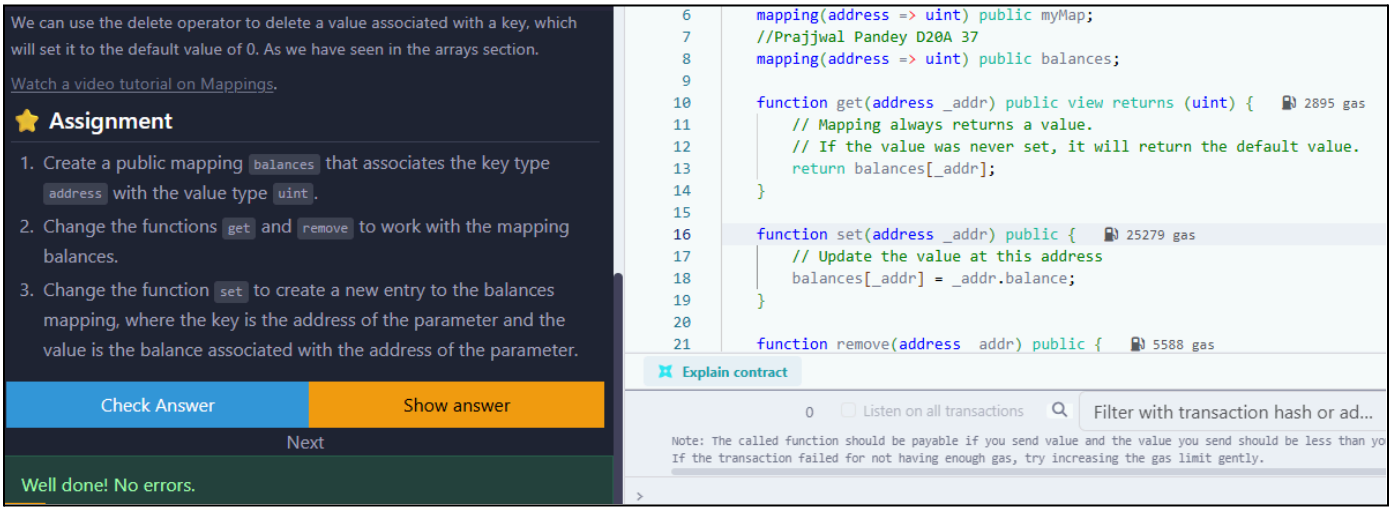
Tutorial - 11



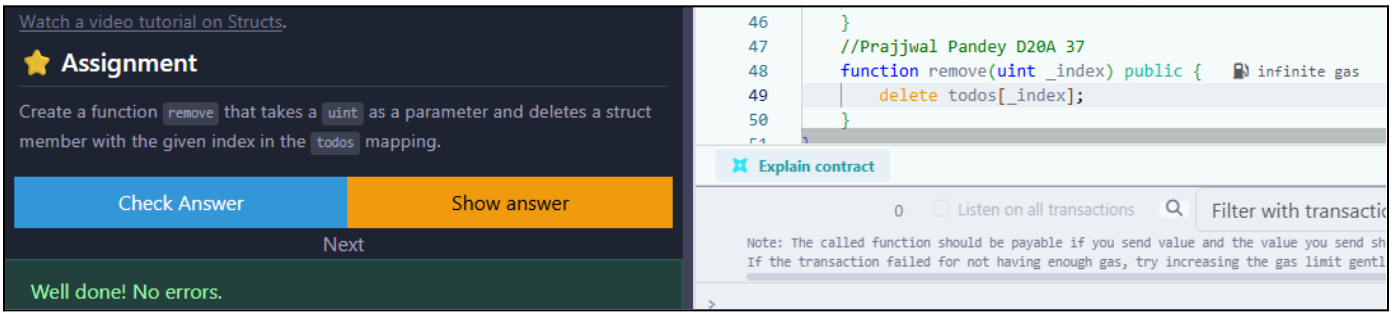
Tutorial - 12



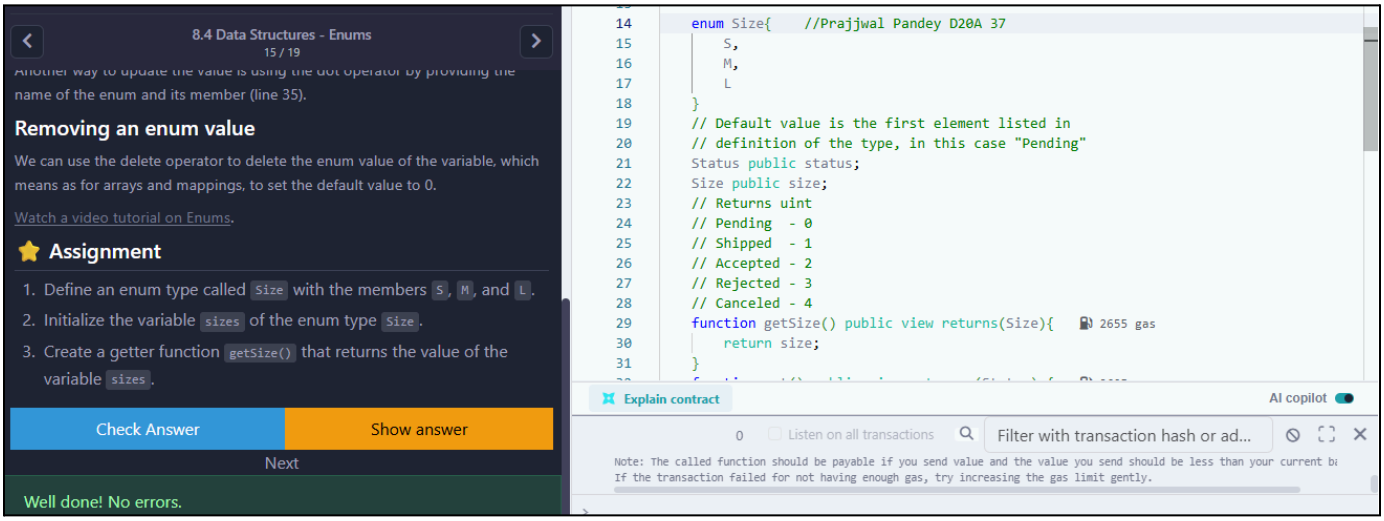
Tutorial - 13



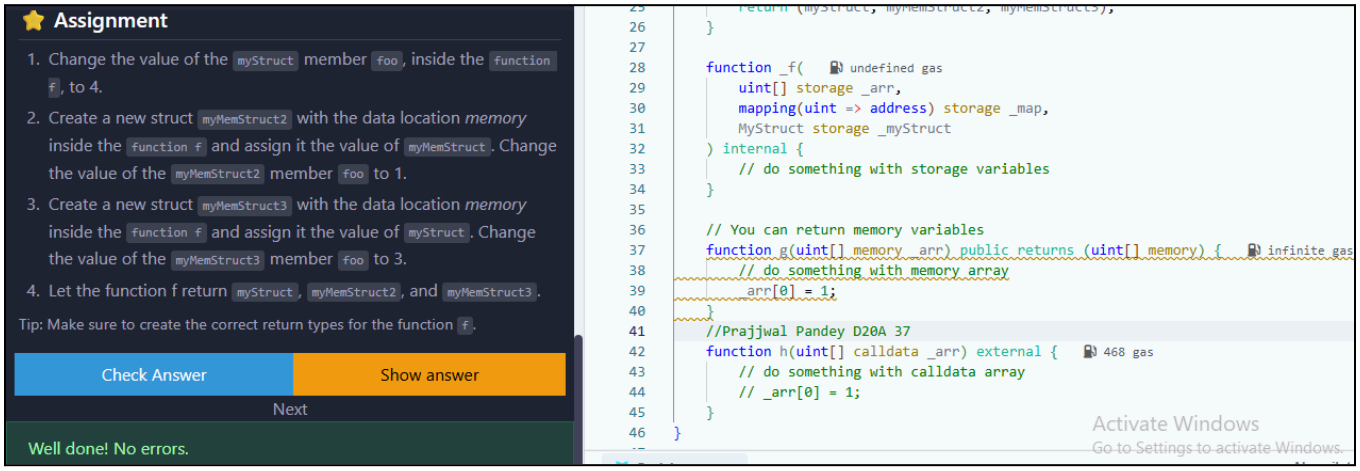
Tutorial - 14



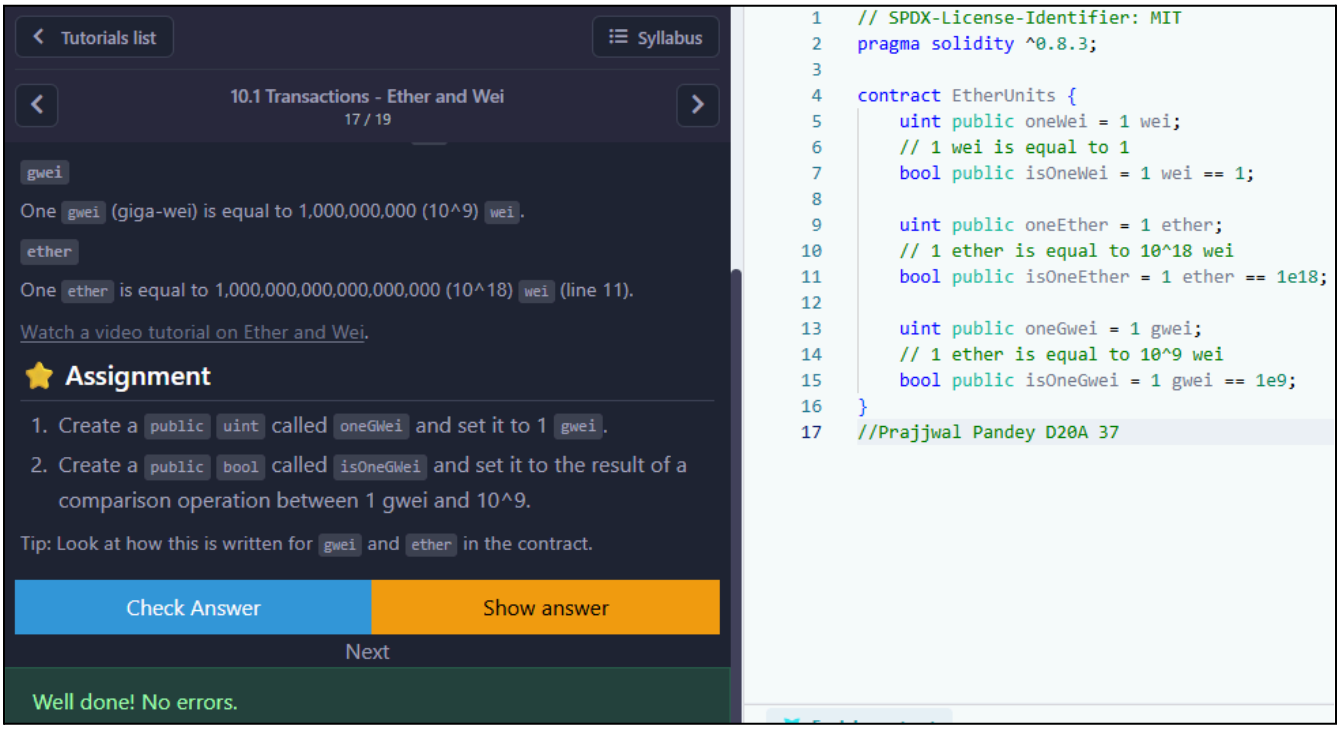
Tutorial - 15



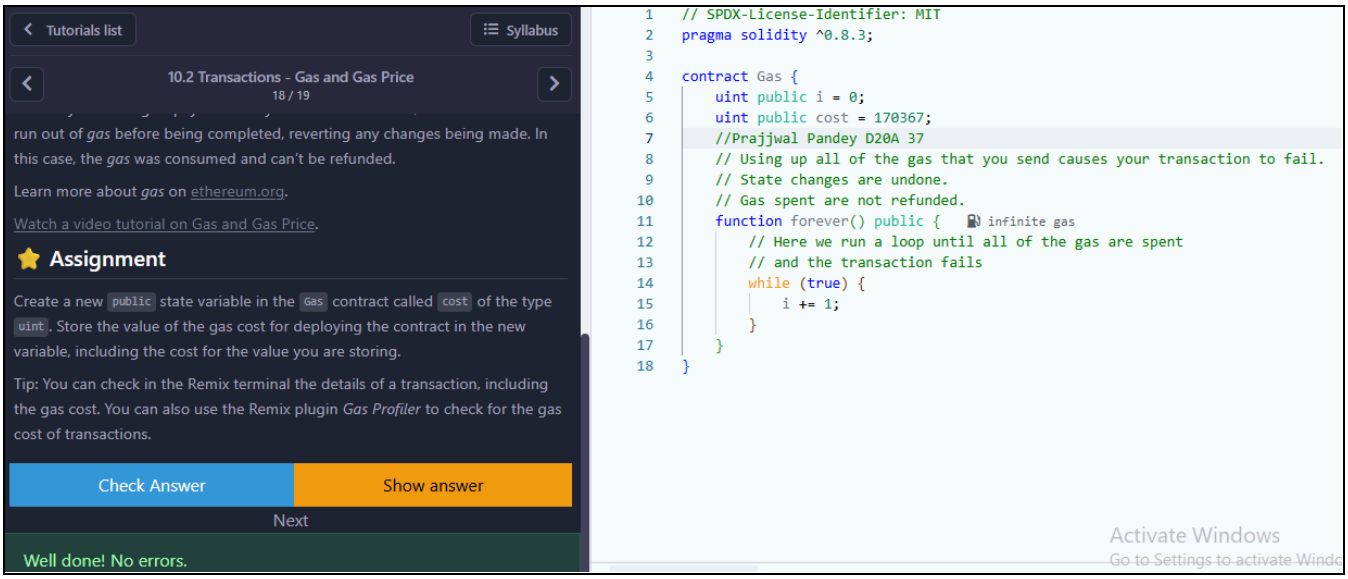
Tutorial - 16



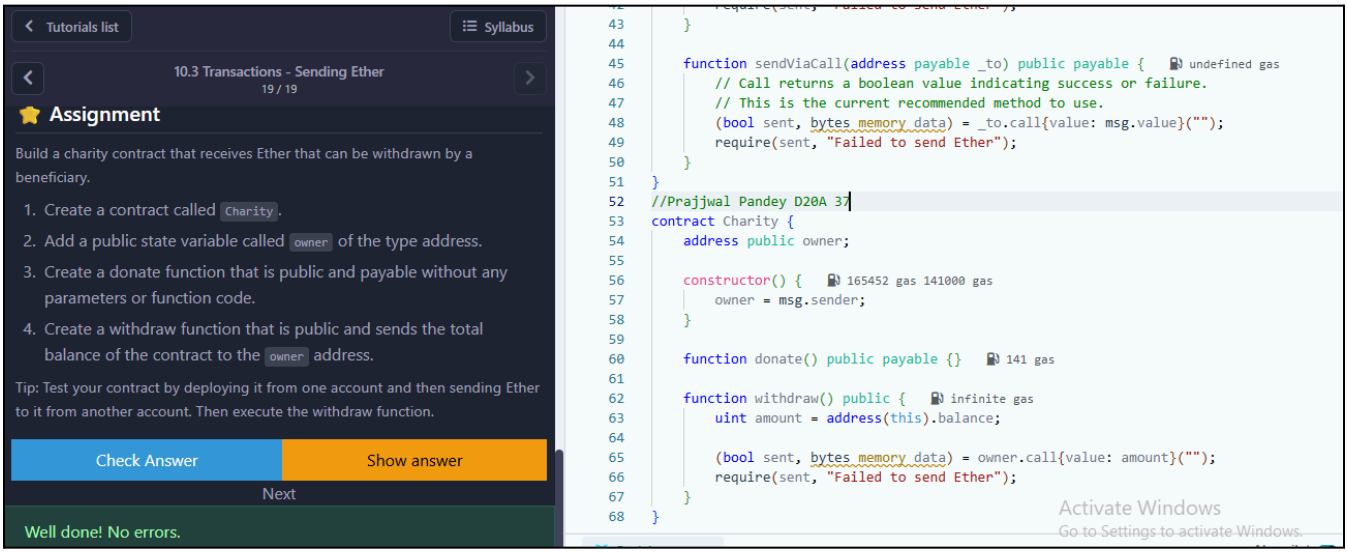
Tutorial - 17



Tutorial - 18



Tutorial - 19



Conclusion: Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

Experiment No. 5

Aim: Deploying a Voting/Ballot Smart Contract

Theory:

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a **guard condition** within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a **Voting Smart Contract**, require can be used to check:

- Whether the person calling the function has the right to vote (require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the **chairperson** before granting voting rights.

Thus, require statements enforce **security, correctness, and reliability** in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

2. Keywords: mapping, storage, and memory

- **mapping**:
  A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is mapping(keyType => valueType). For example:

  mapping(address => Voter) public voters;

  Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like **Ballot**, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them **gas efficient** for lookups but limited for enumeration.

- **storage**:
  In Solidity, storage refers to the **permanent memory** of the contract, stored on the Ethereum blockchain. Variables declared at the contract

level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

- **memory**:
  In contrast, memory is **temporary storage**, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must **balance between storage and memory** to ensure efficiency and cost-effectiveness.

3. Why bytes32 Instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies in **efficiency and gas optimization**.

- **bytes32** is a **fixed-size type**, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- **string** is a **dynamically sized type**, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the **Web3 Type Converter** help developers easily switch between these two types for deployment and testing.

In summary, bytes32 is used when performance and gas efficiency are priorities, while string is preferred for readability and ease of use.

Code:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.20;
```

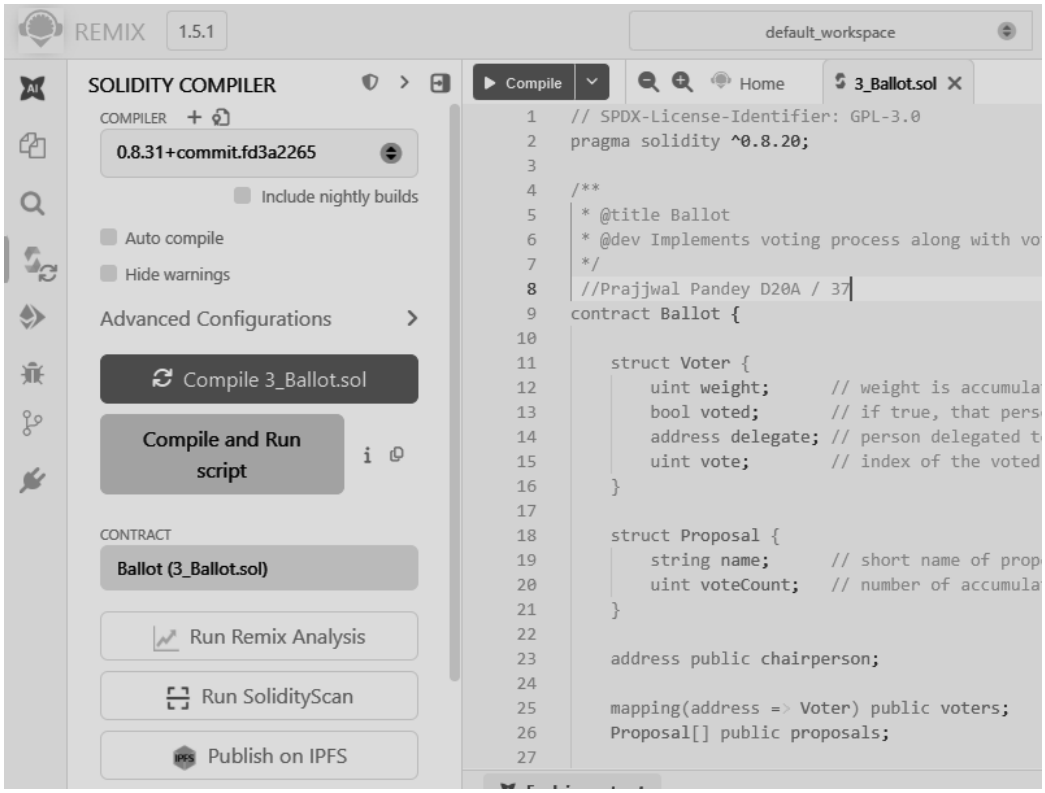
```
/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
 //Prajjwal Pandey D20A / 37
contract Ballot {

   struct Voter {
      uint weight;      // weight is accumulated by delegation
      bool voted;       // if true, that person already voted
      address delegate; // person delegated to
      uint vote;        // index of the voted proposal
   }

   struct Proposal {
      string name;      // short name of proposal
      uint voteCount;   // number of accumulated votes
   }

   address public chairperson;

   mapping(address => Voter) public voters;
   Proposal[] public proposals;

   /**
    * @dev Create a new ballot to choose one of `proposalNames`.
    */
   constructor(string[] memory proposalNames) {
      require(proposalNames.length > 0, "Must provide proposals");

      chairperson = msg.sender;
      voters[chairperson].weight = 1;

      for (uint i = 0; i < proposalNames.length; i++) {
         proposals.push(Proposal({
            name: proposalNames[i],
            voteCount: 0
         }));
      }
   }

   /**
    * @dev Give `voter` the right to vote on this ballot.
    * Can only be called by chairperson.
    */
```

```
function giveRightToVote(address voter) external {
    require(msg.sender == chairperson, "Only chairperson can give right to vote");
    require(!voters[voter].voted, "The voter already voted");
    require(voters[voter].weight == 0, "Voter already has right to vote");

    voters[voter].weight = 1;
}

/**
 * @dev Delegate your vote to the voter `to`.
 */
function delegate(address to) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted");
    require(to != msg.sender, "Self-delegation is disallowed");

    // Follow delegation chain
    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        require(to != msg.sender, "Found loop in delegation");
    }

    Voter storage delegate_ = voters[to];

    require(delegate_.weight >= 1, "Delegate has no right to vote");

    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        delegate_.weight += sender.weight;
    }
}

/**
 * @dev Give your vote (including delegated votes) to proposal `proposal`.
 */
function vote(uint proposal) external {
    require(proposal < proposals.length, "Invalid proposal");

    Voter storage sender = voters[msg.sender];
```

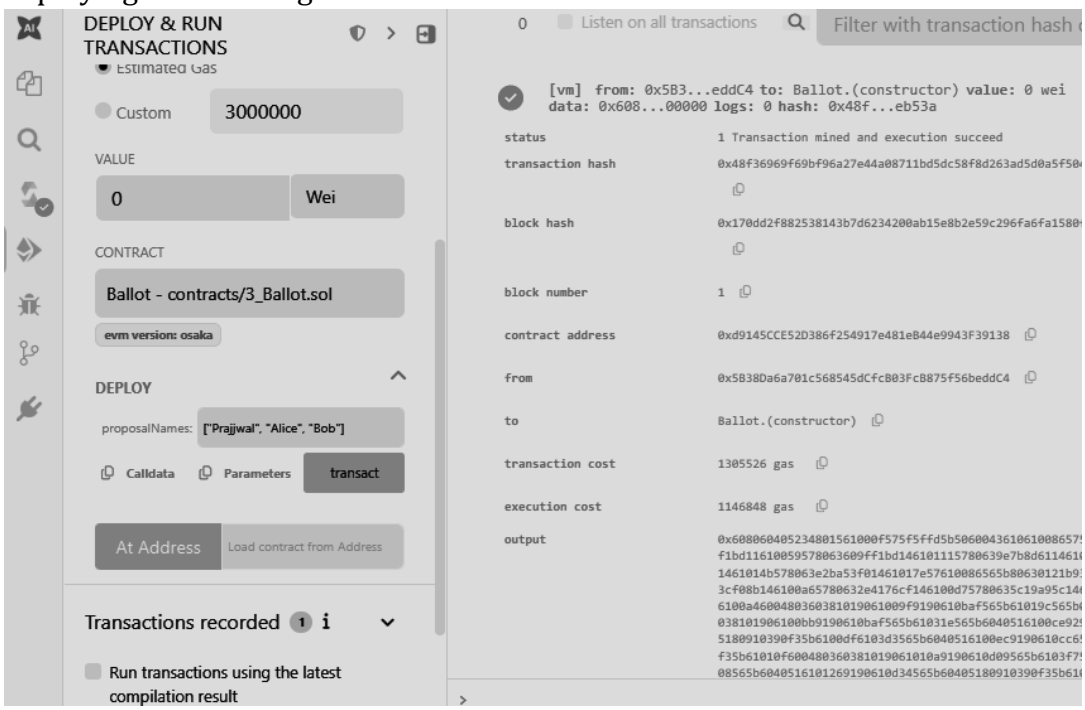
```
        require(sender.weight != 0, "Has no right to vote");
        require(!sender.voted, "Already voted");

        sender.voted = true;
        sender.vote = proposal;

        proposals[proposal].voteCount += sender.weight;
    }

    /**
     * @dev Computes the winning proposal.
     */
    function winningProposal() public view returns (uint winningProposal_) {
        uint winningVoteCount = 0;

        for (uint p = 0; p < proposals.length; p++) {
            if (proposals[p].voteCount > winningVoteCount) {
                winningVoteCount = proposals[p].voteCount;
                winningProposal_ = p;
            }
        }
    }

    /**
     * @dev Returns the name of the winning proposal.
     */
    function winnerName() external view returns (string memory winnerName_) {
        winnerName_ = proposals[winningProposal()].name;
    }
}
```

Output Screenshot:

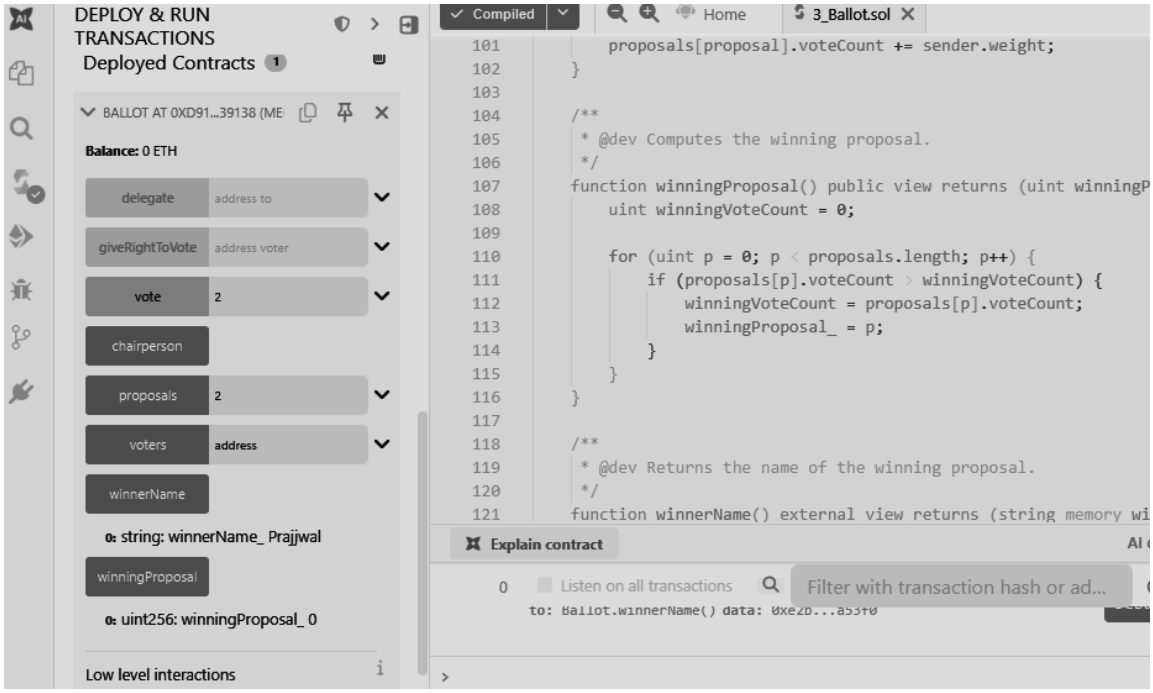
1. Compiled Ballot.sol contract



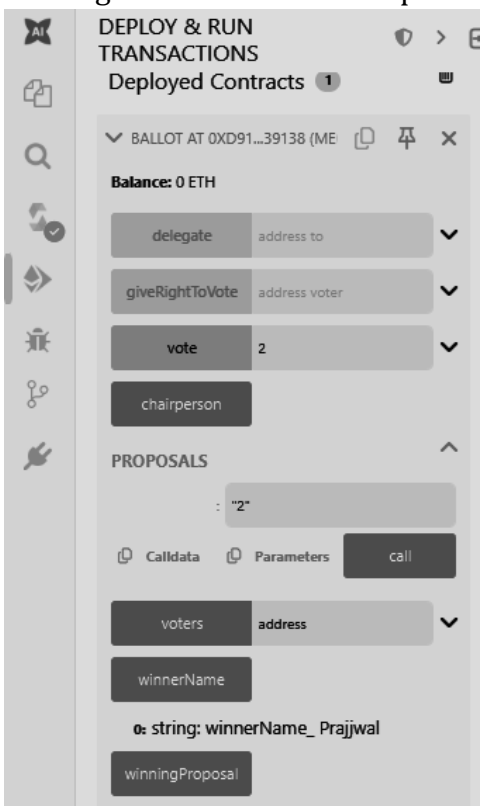
2. Deploying and running of the contract



3. Loading the Proposal Candidate's Names (string)

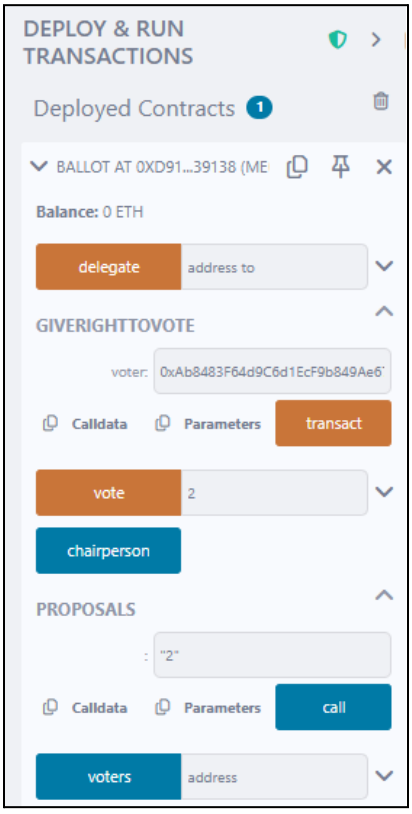


4. Viewing the details of the Proposal Candidate

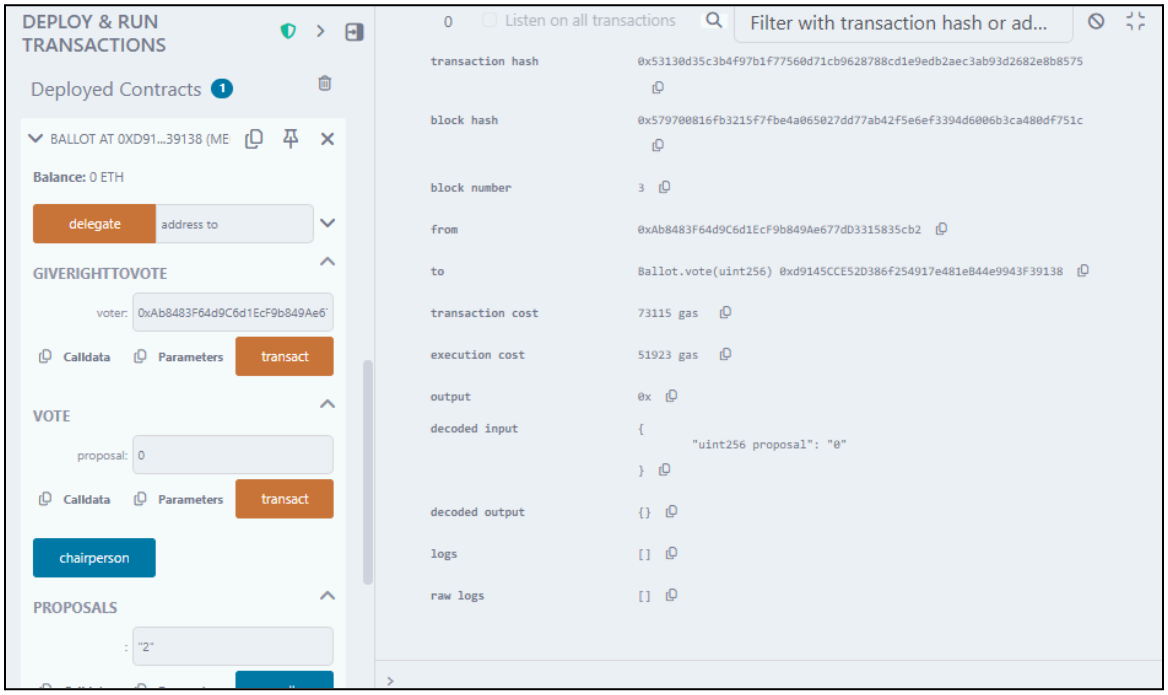


5. Giving the right to an account other than and by the chairman

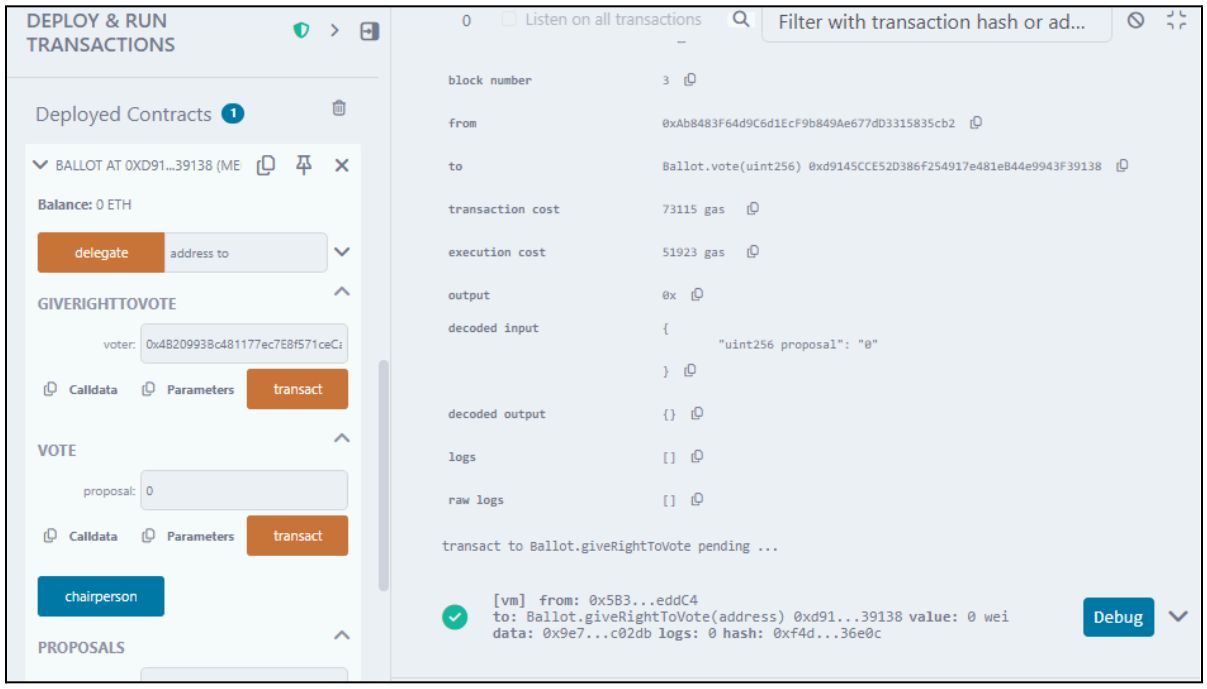
6. Selecting the account which was given the right to vote and then writing the proposal candidate's index to vote.



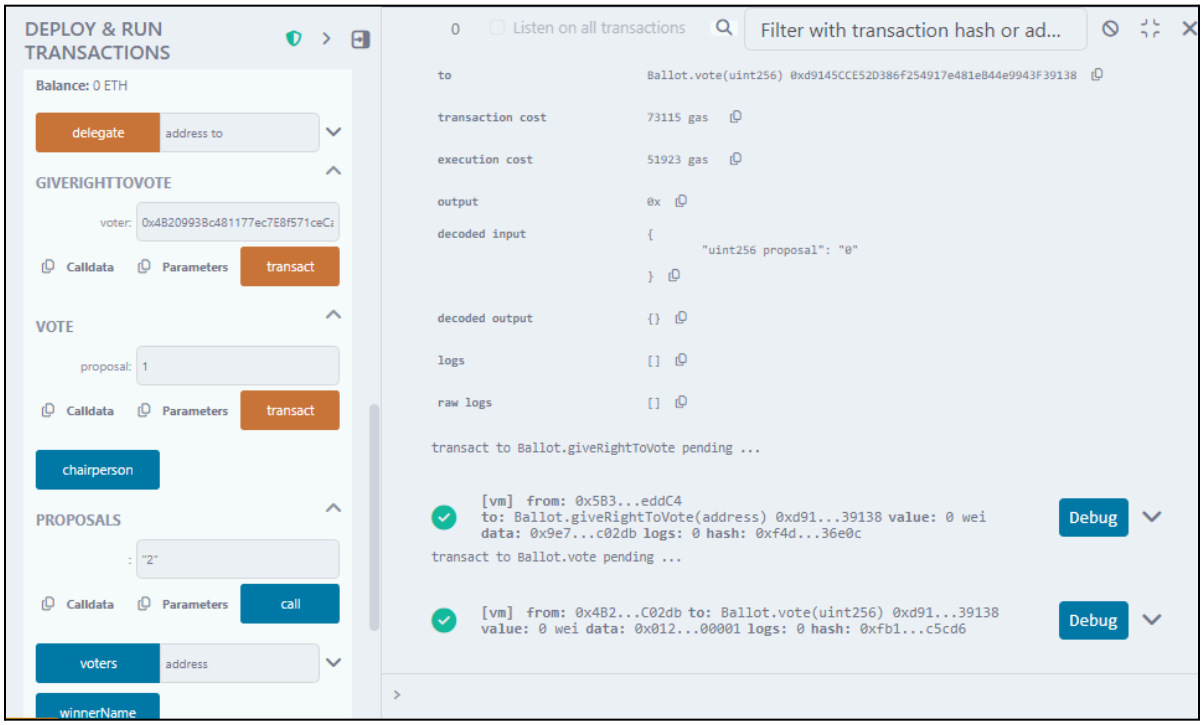
7. We can view the Voter's Information

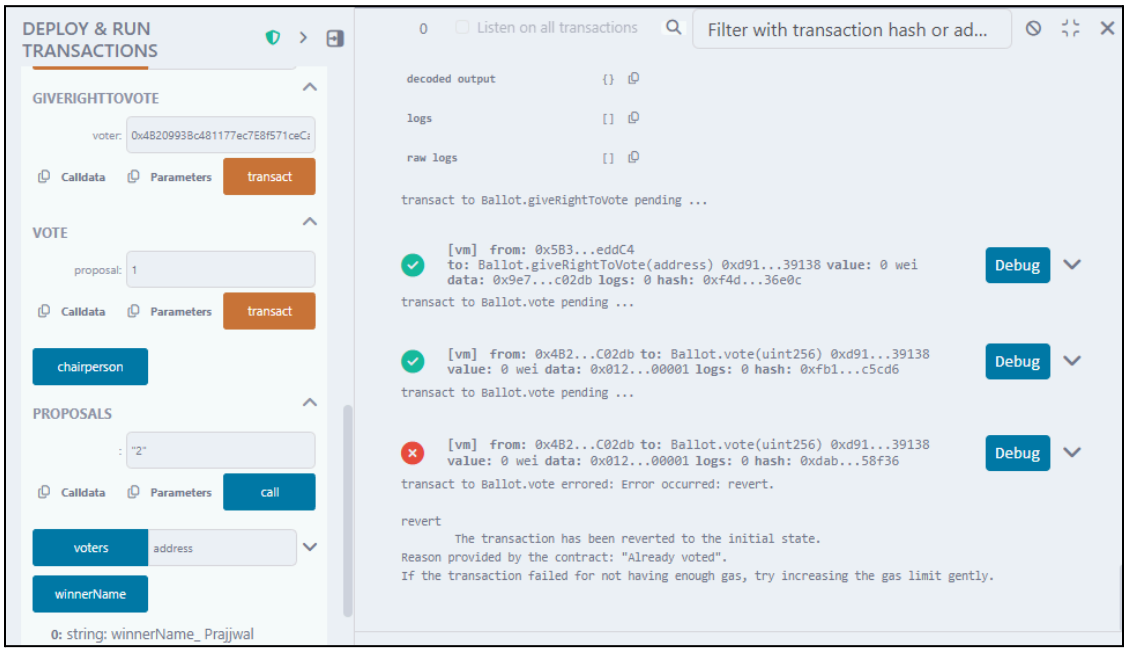


8. Selecting the account of the delegator and then writing the address of the voter to be delegated to.

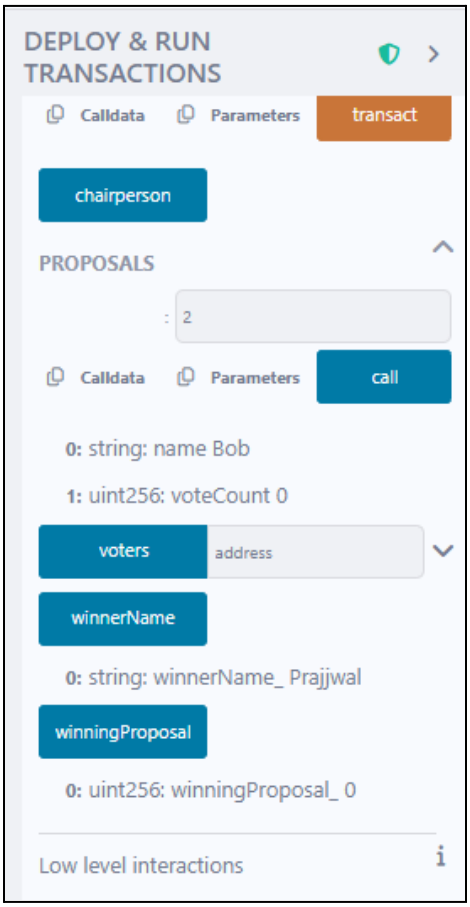


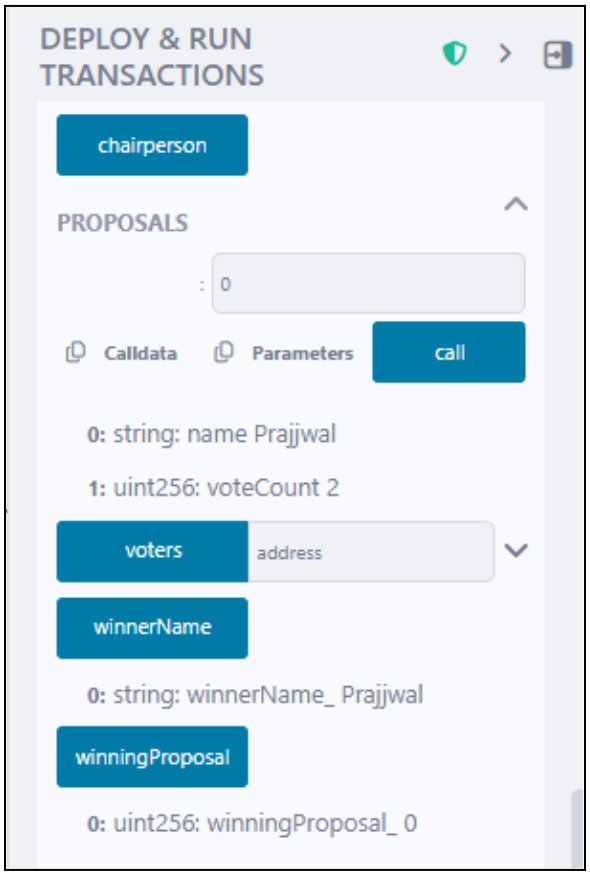
9. Proposal Candidate's Information before giving the Delegate's vote.





10. In this final screenshot we can see that after the delegation's vote the weight of one vote of the one voter who was delegated is the number of people who delegated the voter as their voter (Default weight of any voter is 1).





Conclusion: In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

BLOCKCHAIN EXPERIMENT - 6

AIM: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

THEORY:

What is a Smart Contract? A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- **Automation:** Executes actions automatically when conditions are satisfied.
- **Transparency:** Contract code and transactions are visible on the blockchain.
- **Security:** Once deployed, contracts are immutable and tamper-proof.
- **Cost-effective:** Removes intermediaries and reduces transaction costs.
- **Trustless System:** Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

About the Crowdfunding Platform: A blockchain-based crowdfunding platform replaces centralized authorities with smart contracts. Two contracts are designed — **CrowdfundingContract.sol**, which manages campaign creation and tracks donations, and **DonorContract.sol**, which handles donor interactions and communicates with the CrowdfundingContract through a Solidity interface. Together they ensure a transparent, trustless, and intermediary-free fundraising system.

Tools Used:

- **Solidity** — High-level language for writing smart contracts on Ethereum.
- **Remix IDE** — Browser-based IDE for writing, compiling, deploying, and testing smart contracts using a built-in Remix VM.

IMPLEMENTATION:

CrowdfundingContract.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

contract CrowdfundingContract {

    struct Campaign {
        uint256 campaignId;
        string title;
        uint256 goalAmount;
        uint256 raisedAmount;
        uint256 deadline;
        address payable creator;
        bool isActive;
    }

    uint256 public campaignCount = 0;
    mapping(uint256 => Campaign) public campaigns;

        event CampaignCreated(uint256 campaignId, string title, uint256 goalAmount,
address creator);
    event FundsReceived(uint256 campaignId, uint256 amount);

    function createCampaign(
        string memory _title,
        uint256 _goalAmount,
        uint256 _durationDays
    ) public {
        campaignCount++;
        uint256 deadline = block.timestamp + (_durationDays * 1 days);
        campaigns[campaignCount] = Campaign(
            campaignCount,
            _title,
            _goalAmount,
            0,
            deadline,
            payable(msg.sender),
            true
        );
        emit CampaignCreated(campaignCount, _title, _goalAmount, msg.sender);
    }

    function receiveDonation(uint256 _campaignId, uint256 _amount) external {
        Campaign storage c = campaigns[_campaignId];
```

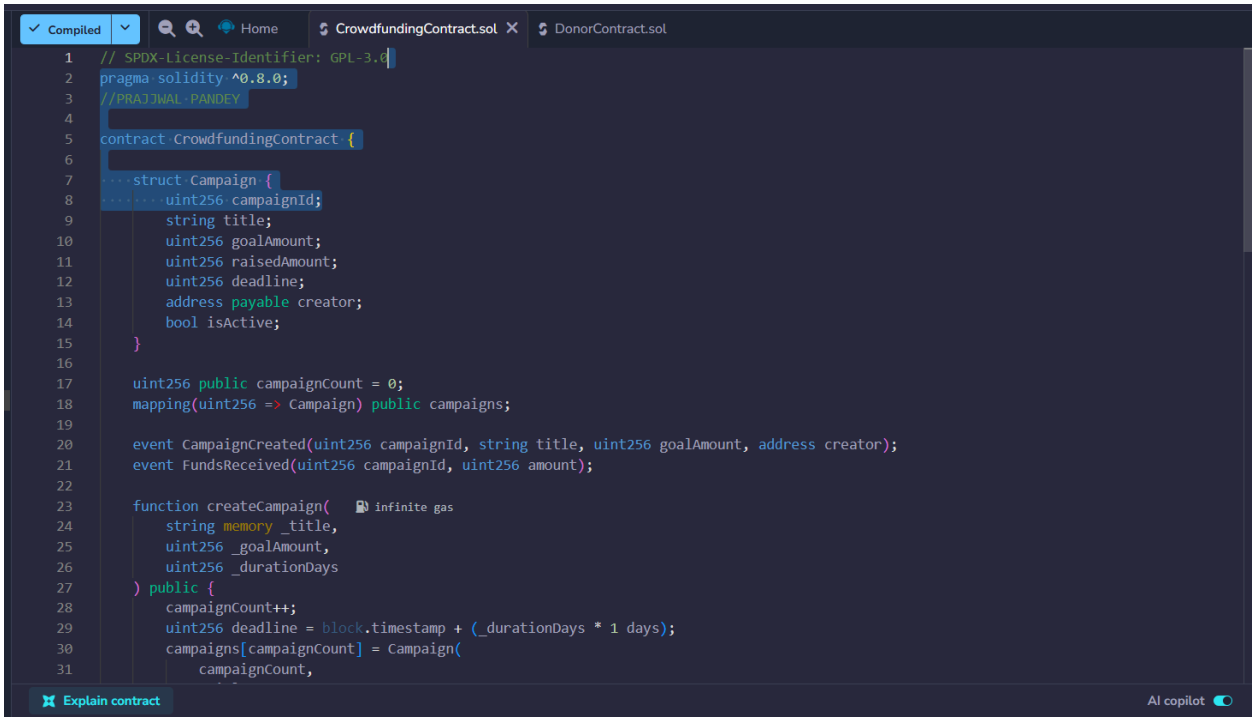
```
        require(c.isActive, "Campaign is not active");
        c.raisedAmount += _amount;
        emit FundsReceived(_campaignId, _amount);
    }

    function getCampaign(uint256 _campaignId) external view returns (
        uint256, string memory, uint256, uint256, uint256, address, bool
    ) {
        Campaign storage c = campaigns[_campaignId];
        return (
            c.campaignId,
            c.title,
            c.goalAmount,
            c.raisedAmount,
            c.deadline,
            c.creator,
            c.isActive
        );
    }

    function closeCampaign(uint256 _campaignId) external {
        campaigns[_campaignId].isActive = false;
    }
}
```

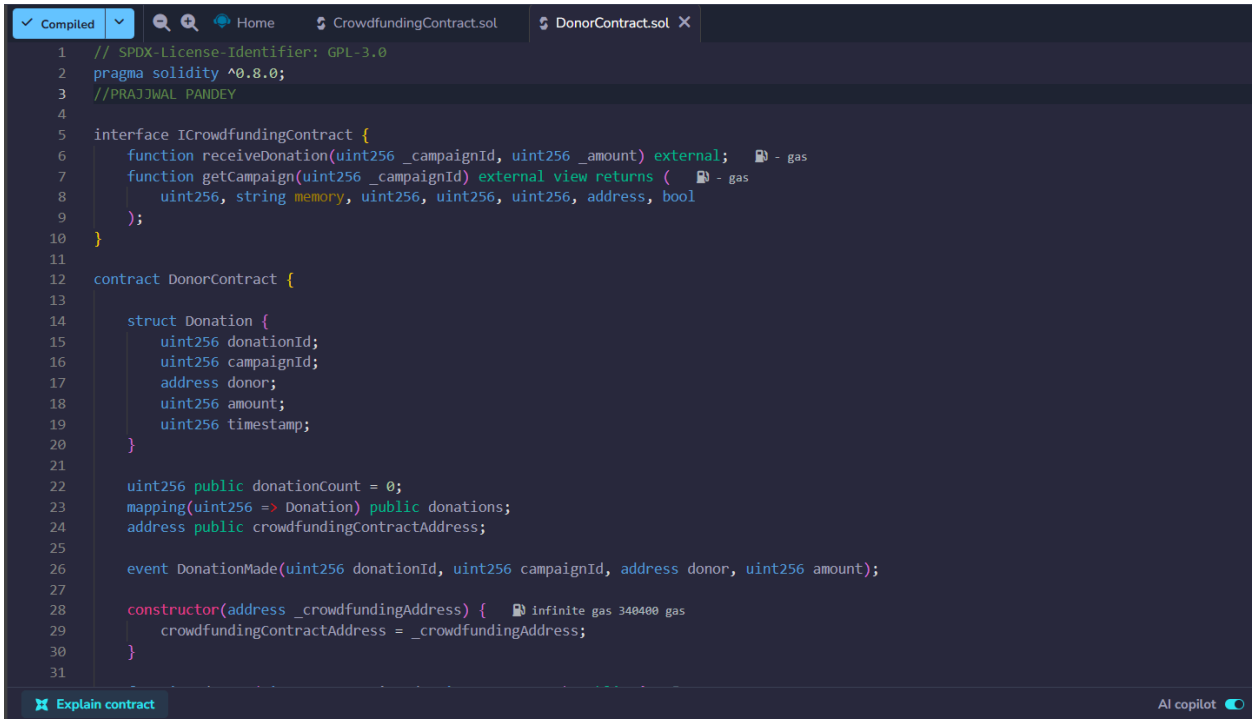
DonorContract.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

interface ICrowdfundingContract {
    function receiveDonation(uint256 _campaignId, uint256 _amount) external;
    function getCampaign(uint256 _campaignId) external view returns (
        uint256, string memory, uint256, uint256, uint256, address, bool
    );
}
contract DonorContract {

    struct Donation {
        uint256 donationId;
        uint256 campaignId;
        address donor;
        uint256 amount;
        uint256 timestamp;
    }
    uint256 public donationCount = 0;
    mapping(uint256 => Donation) public donations;
```

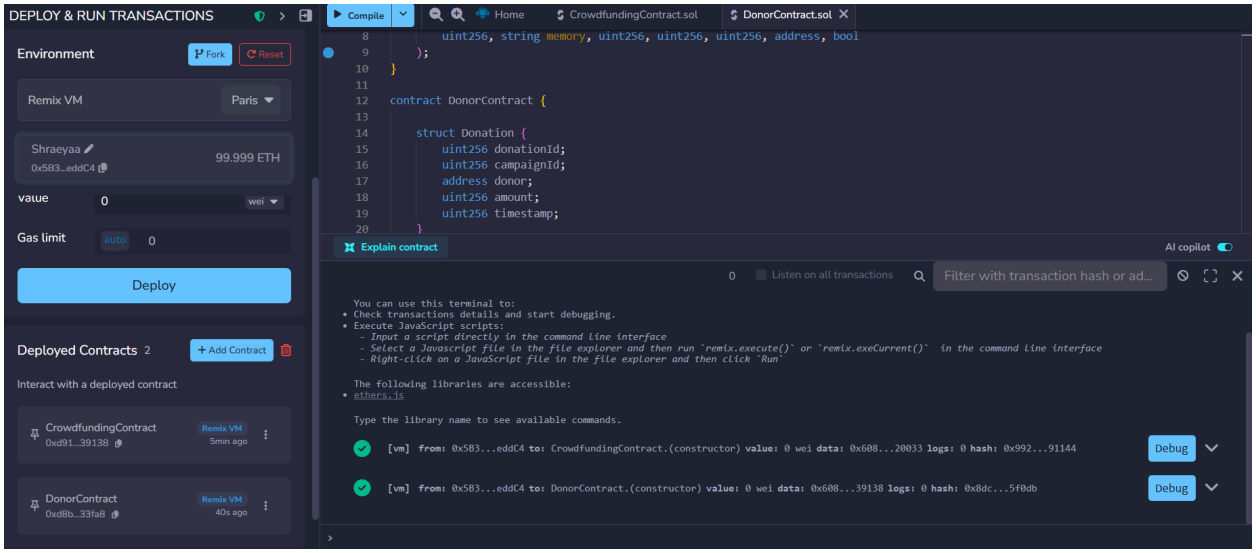
```
    address public crowdfundingContractAddress;

    event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256
amount);
    constructor(address _crowdfundingAddress) {
        crowdfundingContractAddress = _crowdfundingAddress;
    }
    function donate(uint256 _campaignId, uint256 _amount) public {
        donationCount++;
        donations[donationCount] = Donation(
            donationCount,
            _campaignId,
            msg.sender,
            _amount,
            block.timestamp
        );
        ICrowdfundingContract(crowdfundingContractAddress)
            .receiveDonation(_campaignId, _amount);

        emit DonationMade(donationCount, _campaignId, msg.sender, _amount);
    }
    function getDonation(uint256 _donationId) public view returns (
        uint256, uint256, address, uint256, uint256
    ) {
        Donation storage d = donations[_donationId];
        return (d.donationId, d.campaignId, d.donor, d.amount, d.timestamp);
    }
    function getCrowdfundingContract() public view returns (address) {
        return crowdfundingContractAddress;
    }
}
```

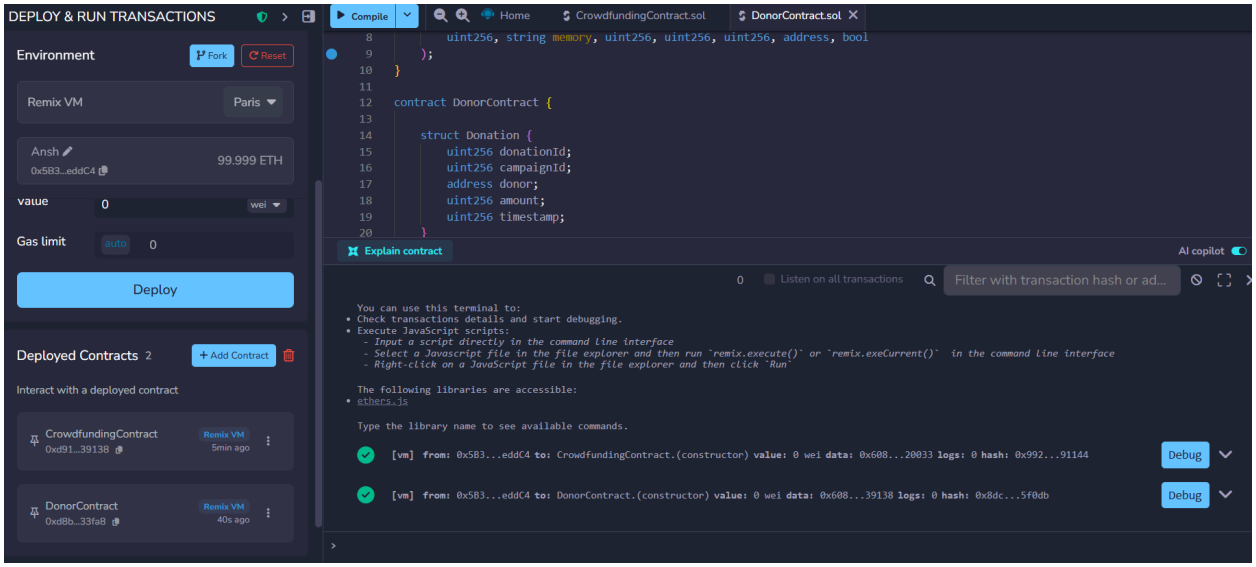
Compilation:

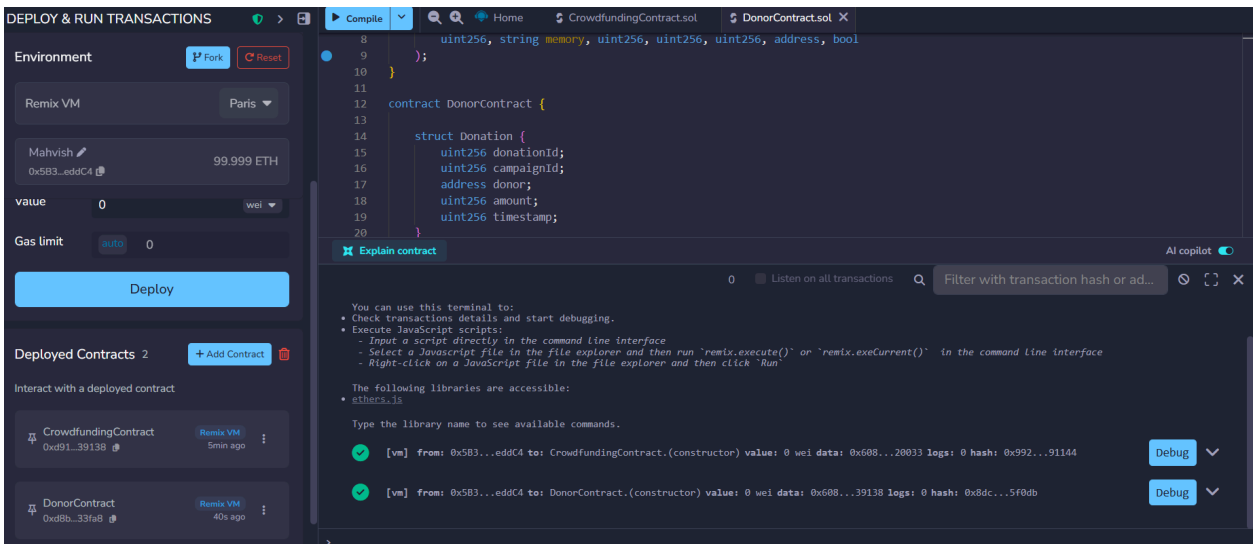


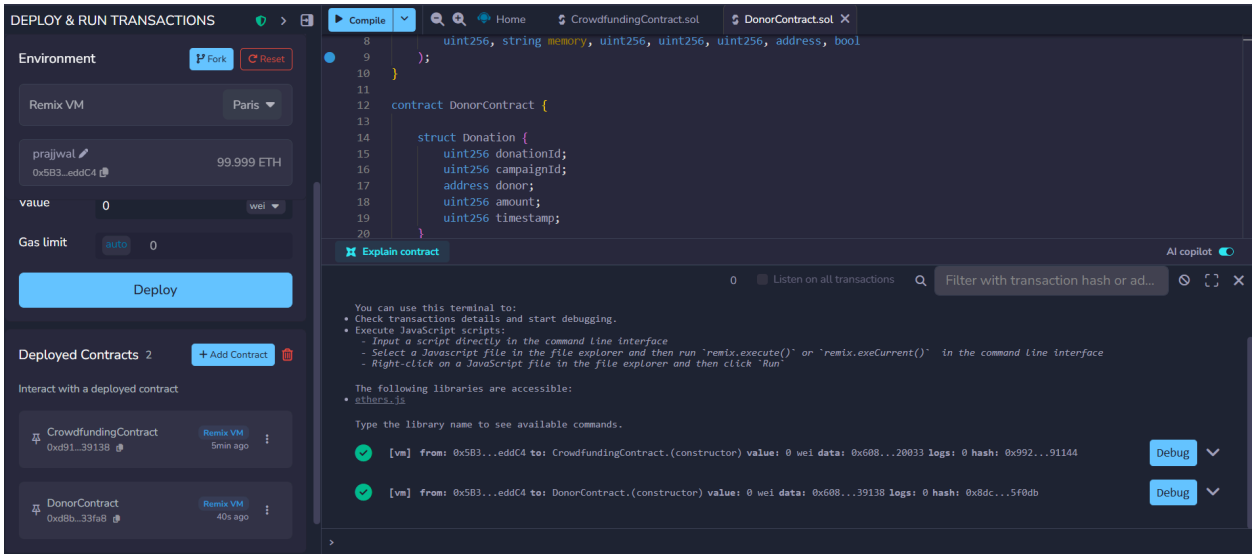


DEPLOYMENT:

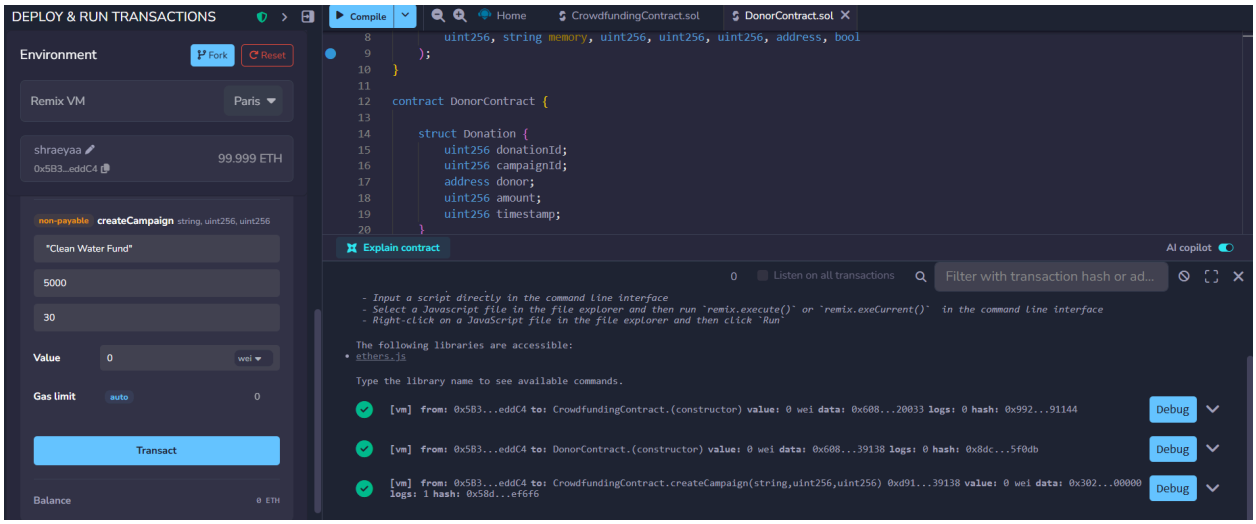

DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Paris
Shraeyaa
0x5B3...eddC4
99.999 ETH
value
0
wei
Gas limit
auto
0
Deploy
Deployed Contracts 2
+ Add Contract
Interact with a deployed contract
CrowdfundingContract
0xd91...39138
Remix VM
5min ago
DonorContract
0xd8b...33fa8
Remix VM
40s ago
Compile
Home
CrowdfundingContract.sol
DonorContract.sol
uint256, string memory, uint256, uint256, uint256, address, bool
);
}
contract DonorContract {
struct Donation {
uint256 donationId;
uint256 campaignId;
address donor;
uint256 amount;
uint256 timestamp;
}
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
You can use this terminal to:
Check transactions details and start debugging.
Execute JavaScript scripts:
Input a script directly in the command line interface
Select a Javascript file in the file explorer and then run `remix.execute()` or `remix.exeCurrent()` in the command line interface
Right-click on a JavaScript file in the file explorer and then click `Run`
The following libraries are accessible:
ethers.js
Type the library name to see available commands.
[vm] from: 0x5B3...eddC4 to: CrowdfundingContract.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x992...91144
Debug
[vm] from: 0x5B3...eddC4 to: DonorContract.(constructor) value: 0 wei data: 0x608...39138 logs: 0 hash: 0x8dc...5f0db
Debug

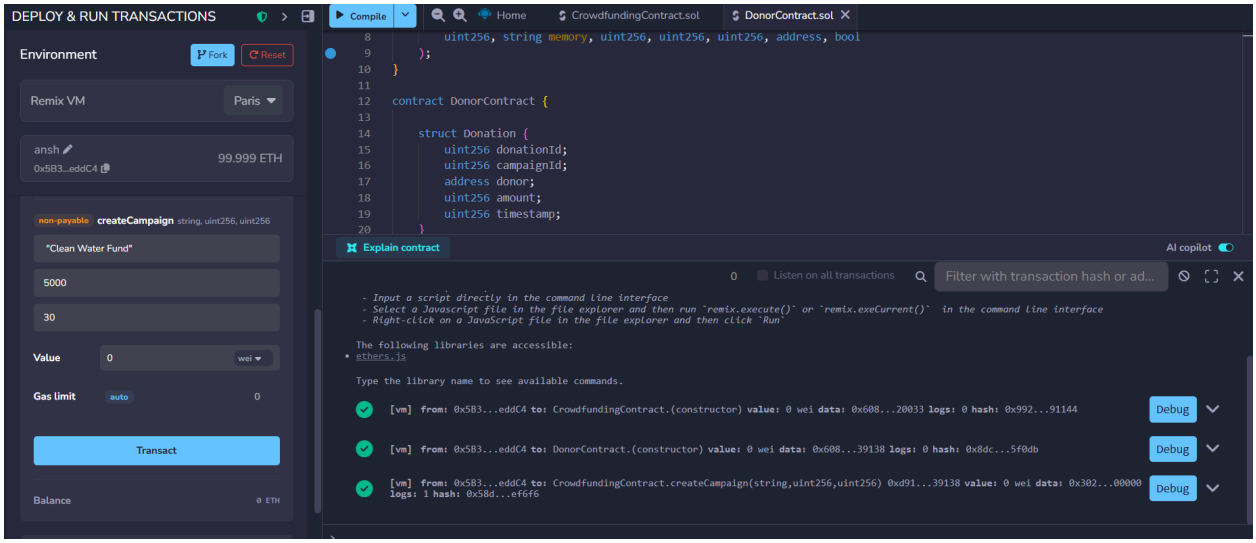

DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Paris
Ansh
0x5B3...eddC4
99.999 ETH
value
0
wei
Gas limit
auto
0
Deploy
Deployed Contracts 2
+ Add Contract
Interact with a deployed contract
CrowdfundingContract
0xd91...39138
Remix VM
5min ago
DonorContract
0xd8b...33fa8
Remix VM
40s ago
Compile
Home
CrowdfundingContract.sol
DonorContract.sol
uint256, string memory, uint256, uint256, uint256, address, bool
);
}
contract DonorContract {
struct Donation {
uint256 donationId;
uint256 campaignId;
address donor;
uint256 amount;
uint256 timestamp;
}
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
You can use this terminal to:
Check transactions details and start debugging.
Execute JavaScript scripts:
Input a script directly in the command line interface
Select a Javascript file in the file explorer and then run `remix.execute()` or `remix.exeCurrent()` in the command line interface
Right-click on a JavaScript file in the file explorer and then click `Run`
The following libraries are accessible:
ethers.js
Type the library name to see available commands.
[vm] from: 0x5B3...eddC4 to: CrowdfundingContract.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x992...91144
Debug
[vm] from: 0x5B3...eddC4 to: DonorContract.(constructor) value: 0 wei data: 0x608...39138 logs: 0 hash: 0x8dc...5f0db
Debug

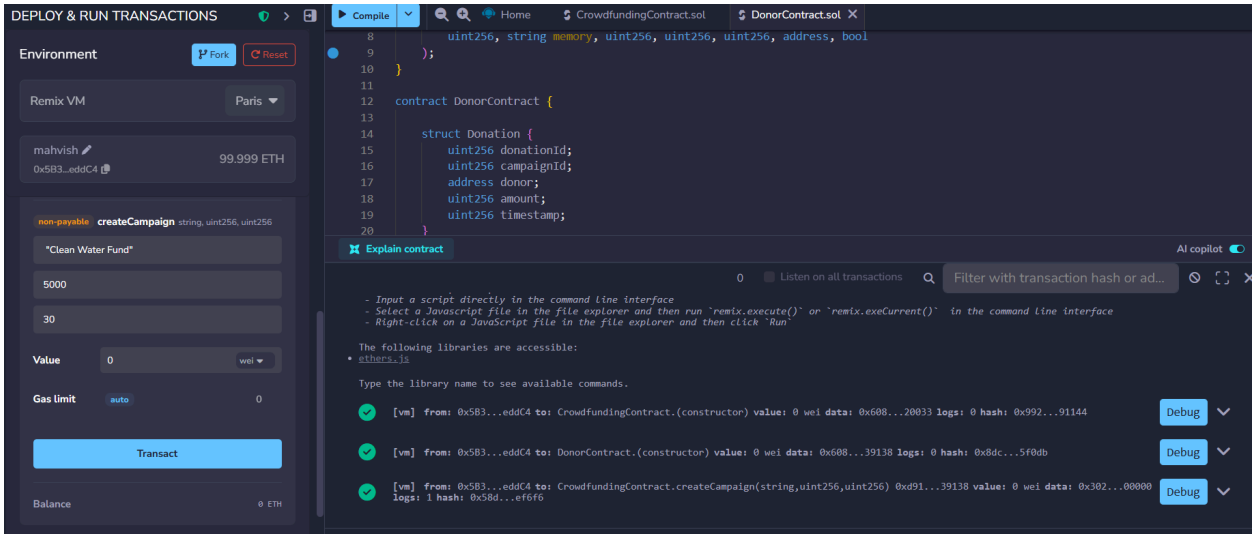

DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Paris
Mahvish
0x5B3...eddC4
99.999 ETH
value
0
wei
Gas limit
auto
0
Deploy
Deployed Contracts 2
+ Add Contract
Interact with a deployed contract
CrowdfundingContract
0xd91...39138
Remix VM
5min ago
DonorContract
0xd8b...33fa8
Remix VM
40s ago
Compile
Home
CrowdfundingContract.sol
DonorContract.sol
uint256, string memory, uint256, uint256, uint256, address, bool
);
}
contract DonorContract {
struct Donation {
uint256 donationId;
uint256 campaignId;
address donor;
uint256 amount;
uint256 timestamp;
}
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
You can use this terminal to:
Check transactions details and start debugging.
Execute JavaScript scripts:
Input a script directly in the command line interface
Select a Javascript file in the file explorer and then run `remix.execute()` or `remix.exeCurrent()` in the command line interface
Right-click on a JavaScript file in the file explorer and then click `Run`
The following libraries are accessible:
ethers.js
Type the library name to see available commands.
[vm] from: 0x5B3...eddC4 to: CrowdfundingContract.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x992...91144
Debug
[vm] from: 0x5B3...eddC4 to: DonorContract.(constructor) value: 0 wei data: 0x608...39138 logs: 0 hash: 0x8dc...5f0db
Debug

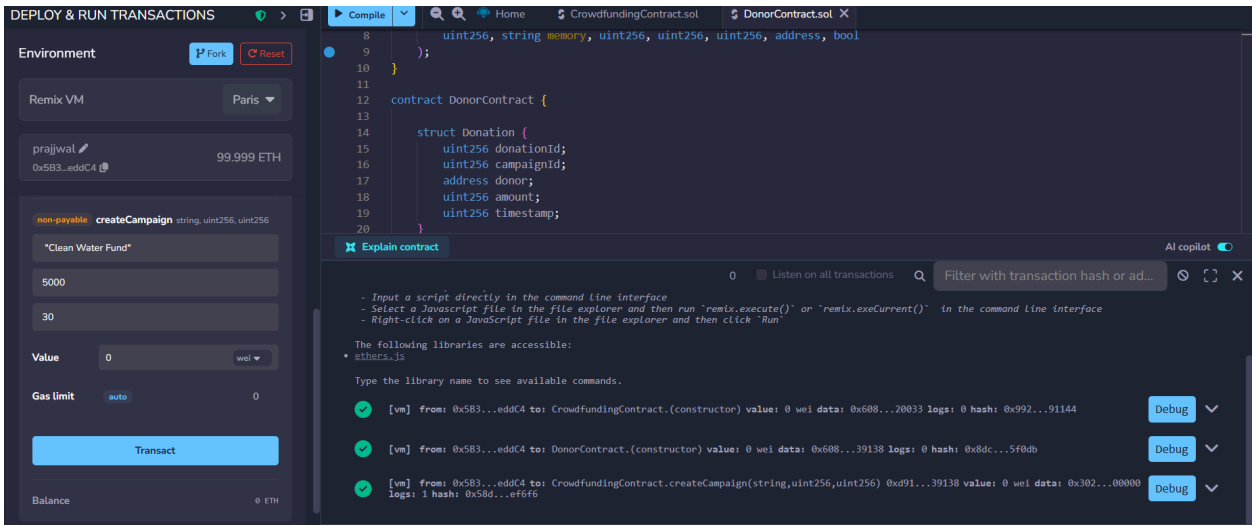


TRANSACTIONS:

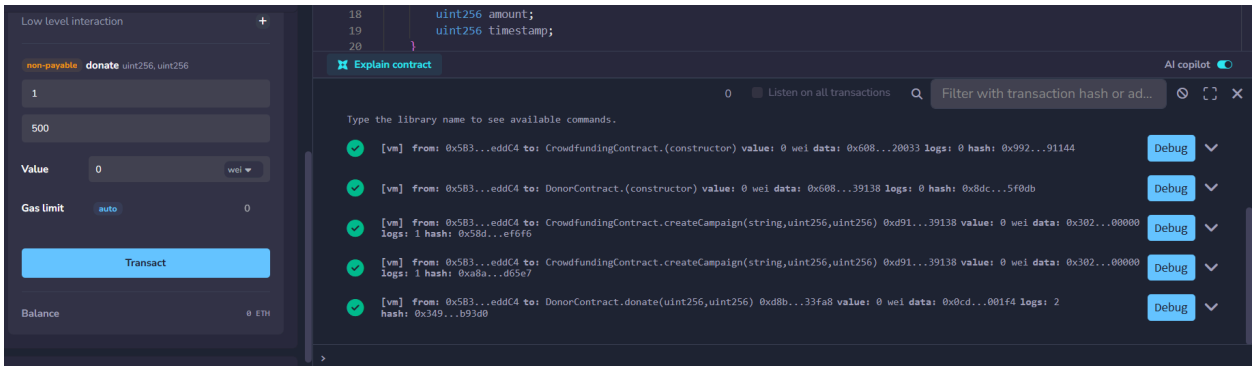


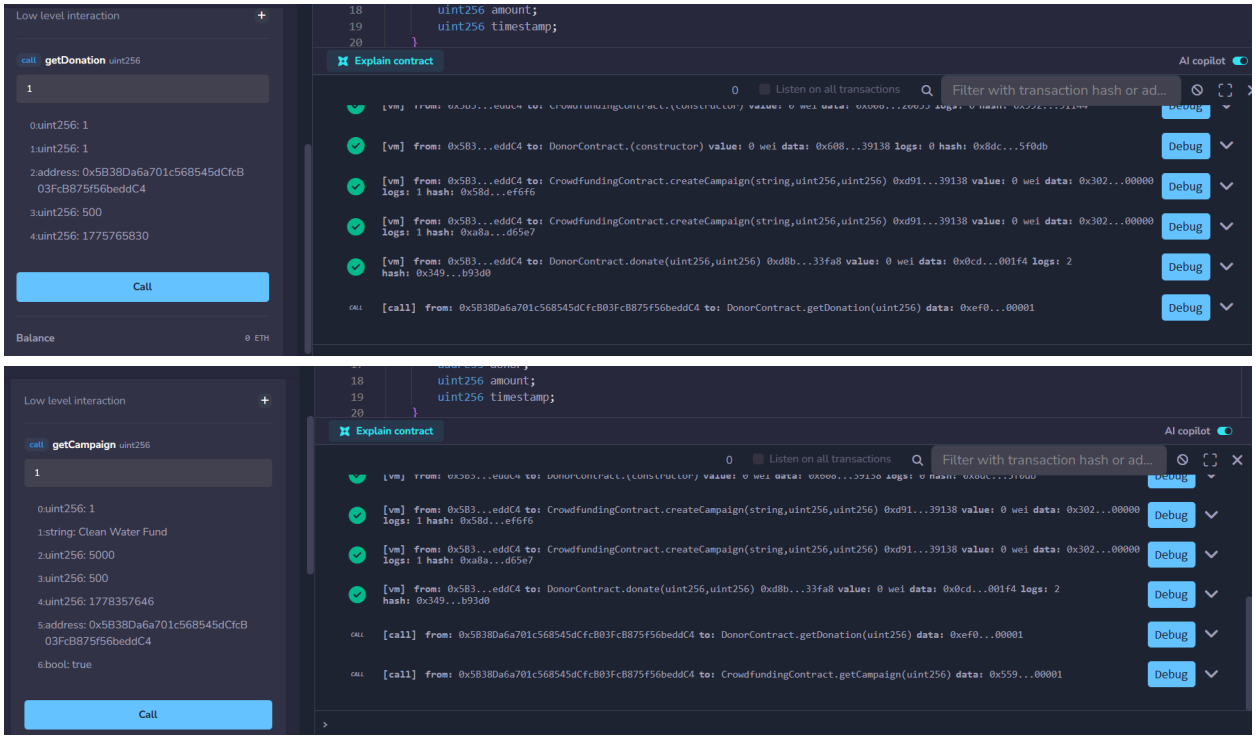



DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Paris
mahvish
0x5B3...eddC4
99.999 ETH
non-payable createCampaign string, uint256, uint256
"Clean Water Fund"
5000
30
Value
0
wei
Gas limit
auto
0
Transact
Balance
0 ETH
Compile
Home
CrowdfundingContract.sol
DonorContract.sol
uint256, string memory, uint256, uint256, uint256, address, bool
);
}
contract DonorContract {
struct Donation {
uint256 donationId;
uint256 campaignId;
address donor;
uint256 amount;
uint256 timestamp;
}
Explain contract
AI copilot
0
Listen on all transactions
Filter with transaction hash or ad...
- Input a script directly in the command line interface
- Select a Javascript file in the file explorer and then run `remix.execute()` or `remix.exeCurrent()` in the command line interface
- Right-click on a JavaScript file in the file explorer and then click `Run`
The following libraries are accessible:
ethers.js
Type the library name to see available commands.
[vm] from: 0x5B3...eddC4 to: CrowdfundingContract.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x992...91144
Debug
[vm] from: 0x5B3...eddC4 to: DonorContract.(constructor) value: 0 wei data: 0x608...39138 logs: 0 hash: 0x8dc...5f0db
Debug
[vm] from: 0x5B3...eddC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000
logs: 1 hash: 0x58d...ef6f6
Debug


DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Paris
prajjwal
0x5B3...eddC4
99.999 ETH
non-payable createCampaign string, uint256, uint256
"Clean Water Fund"
5000
30
Value
0
wei
Gas limit
auto
0
Transact
Balance
0 ETH
Compile
Home
CrowdfundingContract.sol
DonorContract.sol
uint256, string memory, uint256, uint256, uint256, address, bool
);
}
contract DonorContract {
struct Donation {
uint256 donationId;
uint256 campaignId;
address donor;
uint256 amount;
uint256 timestamp;
}
Explain contract
AI copilot
0
Listen on all transactions
Filter with transaction hash or ad...
- Input a script directly in the command line interface
- Select a Javascript file in the file explorer and then run `remix.execute()` or `remix.exeCurrent()` in the command line interface
- Right-click on a JavaScript file in the file explorer and then click `Run`
The following libraries are accessible:
ethers.js
Type the library name to see available commands.
[vm] from: 0x5B3...eddC4 to: CrowdfundingContract.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x992...91144
Debug
[vm] from: 0x5B3...eddC4 to: DonorContract.(constructor) value: 0 wei data: 0x608...39138 logs: 0 hash: 0x8dc...5f0db
Debug
[vm] from: 0x5B3...eddC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000
logs: 1 hash: 0x58d...ef6f6
Debug

DONATIONS


Low level interaction
non-payable donate uint256, uint256
1
500
Value
0
wei
Gas limit
auto
0
Transact
Balance
0 ETH
uint256 amount;
uint256 timestamp;
}
Explain contract
AI copilot
0
Listen on all transactions
Filter with transaction hash or ad...
Type the library name to see available commands.
[vm] from: 0x5B3...eddC4 to: CrowdfundingContract.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x992...91144
Debug
[vm] from: 0x5B3...eddC4 to: DonorContract.(constructor) value: 0 wei data: 0x608...39138 logs: 0 hash: 0x8dc...5f0db
Debug
[vm] from: 0x5B3...eddC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000
logs: 1 hash: 0x58d...ef6f6
Debug
[vm] from: 0x5B3...eddC4 to: CrowdfundingContract.createCampaign(string,uint256,uint256) 0xd91...39138 value: 0 wei data: 0x302...00000
logs: 1 hash: 0xa8a...d65e7
Debug
[vm] from: 0x5B3...eddC4 to: DonorContract.donate(uint256,uint256) 0xd8b...33fa8 value: 0 wei data: 0x0cd...001f4 logs: 2
hash: 0x349...b93d0
Debug



CONCLUSION:

This experiment helped in understanding how smart contracts can automate and secure a crowdfunding platform on the Ethereum blockchain. Using Solidity and Remix IDE, two contracts — CrowdfundingContract and DonorContract — were designed, deployed, and tested. The CrowdfundingContract manages campaign creation and tracks donations, while the DonorContract handles donor interactions. Together they demonstrate a trustless, transparent, and intermediary-free fundraising system.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

BLOCKCHAIN EXPERIMENT - 7

AIM: Implement the embedding wallet (Metamask) and transaction using Solidity

THEORY:

Q1: What is MetaMask? MetaMask is a widely used browser extension that functions as a digital wallet for Ethereum and other compatible blockchain networks. It allows users to manage their accounts, store cryptocurrencies, and send or receive tokens directly from the browser. Beyond basic wallet functionality, MetaMask serves as a gateway between the user and decentralized applications (DApps), enabling seamless signing and authorization of transactions without exposing private keys to third parties. It integrates smoothly with development tools like Remix IDE, making it an essential tool for smart contract deployment and blockchain interaction.

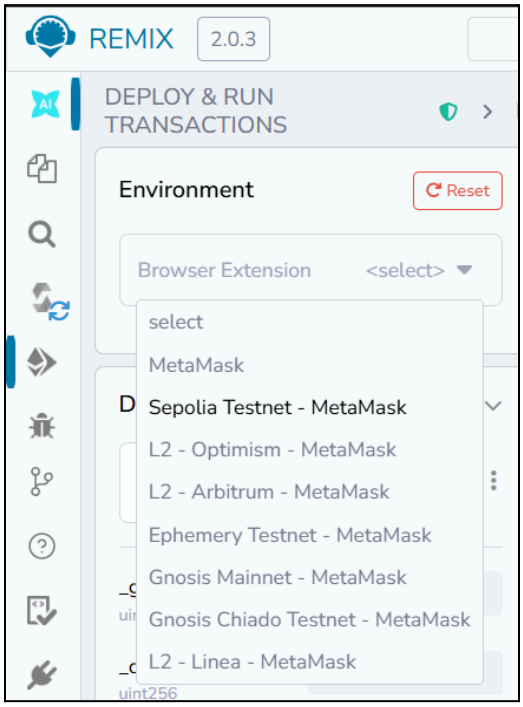
Q2: What is a Test Network (Testnet)? A testnet is a parallel blockchain environment designed specifically for development and experimentation. It replicates the behavior of the Ethereum mainnet but operates using valueless test currency, ensuring that developers can simulate real transactions without any financial consequences. Popular Ethereum testnets include Sepolia and Goerli. These networks allow developers to deploy smart contracts, test their logic, observe gas consumption, and debug errors in a realistic setting before releasing anything on the actual mainnet. Testnets are an essential step in the smart contract development lifecycle, significantly reducing the risk of costly mistakes in production.

Q3: Steps to Connect MetaMask with Remix IDE for Performing Transactions

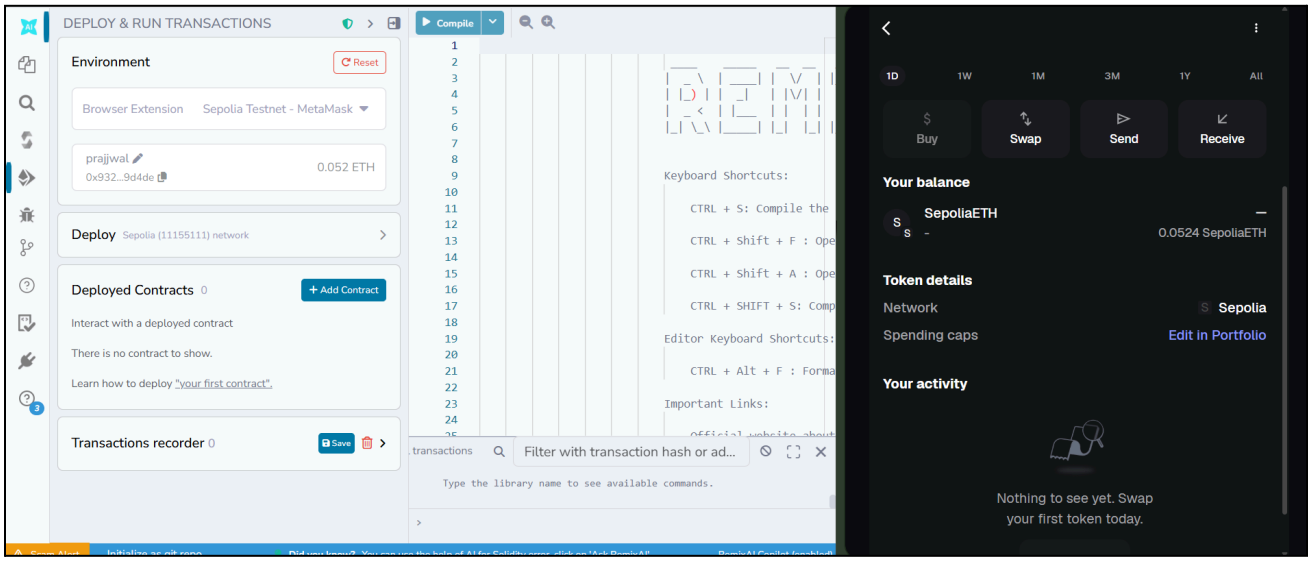
- Download and install the MetaMask extension from the Chrome Web Store and set up a new wallet by following the onboarding steps
- Securely note the Secret Recovery Phrase provided during wallet creation, as it is required to restore access if needed
- Navigate to MetaMask settings and enable the option to show test networks
- Switch the active network to a testnet such as Sepolia from the network dropdown
- Visit a Sepolia faucet website and request free test ETH by entering your wallet address, which will be used to pay gas fees during testing
- Open Remix IDE in the browser at remix.ethereum.org and create or open a Solidity smart contract file
- Use the Solidity Compiler tab in Remix to compile the contract and resolve any errors
- Navigate to the Deploy & Run Transactions tab and choose Injected Provider - MetaMask from the Environment dropdown
- Approve the connection request that MetaMask displays to link your wallet with Remix
- Select the appropriate contract from the dropdown, click Deploy, and confirm the transaction in the MetaMask popup
- Once deployed, use the contract's functions visible in the Deployed Contracts section and approve each transaction through MetaMask as required

IMPLEMENTATION:

Step 1: Select metamask sepolia testnet in environment



Initial setup:



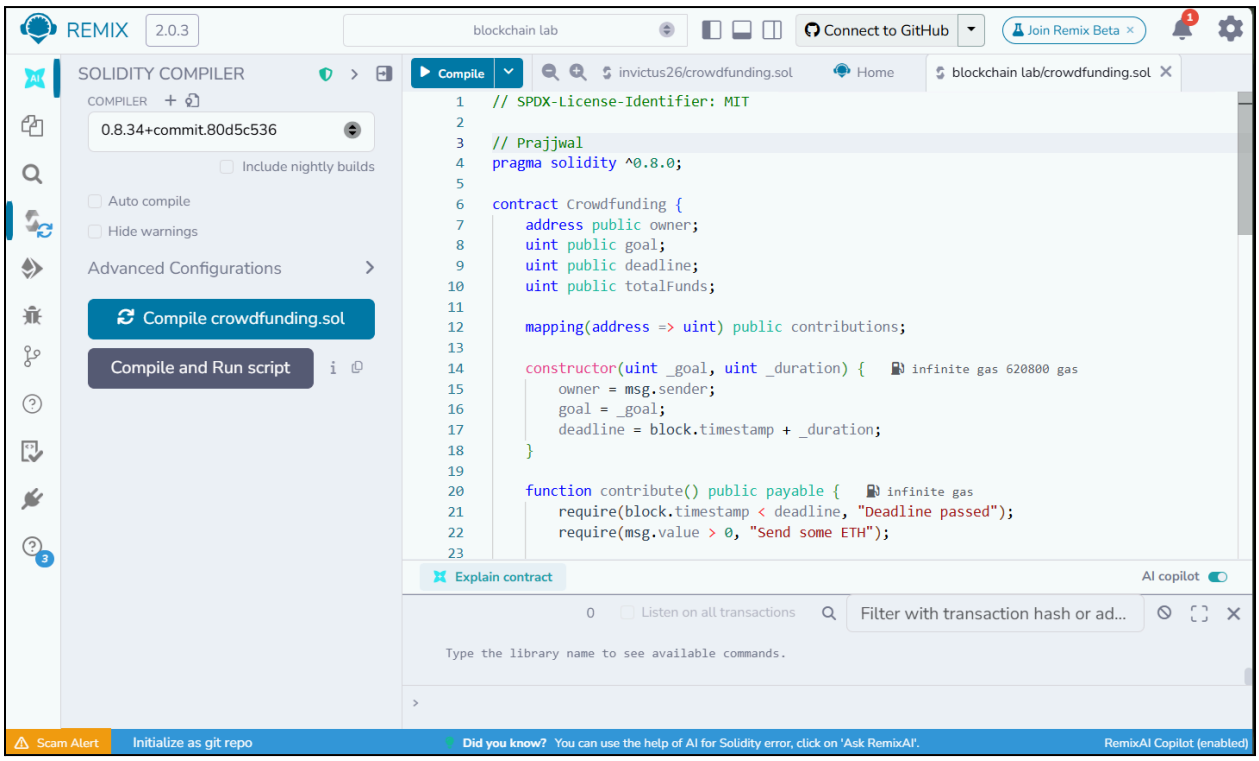
Step 2: Create a contract for a crowdfunding platform

CODE:

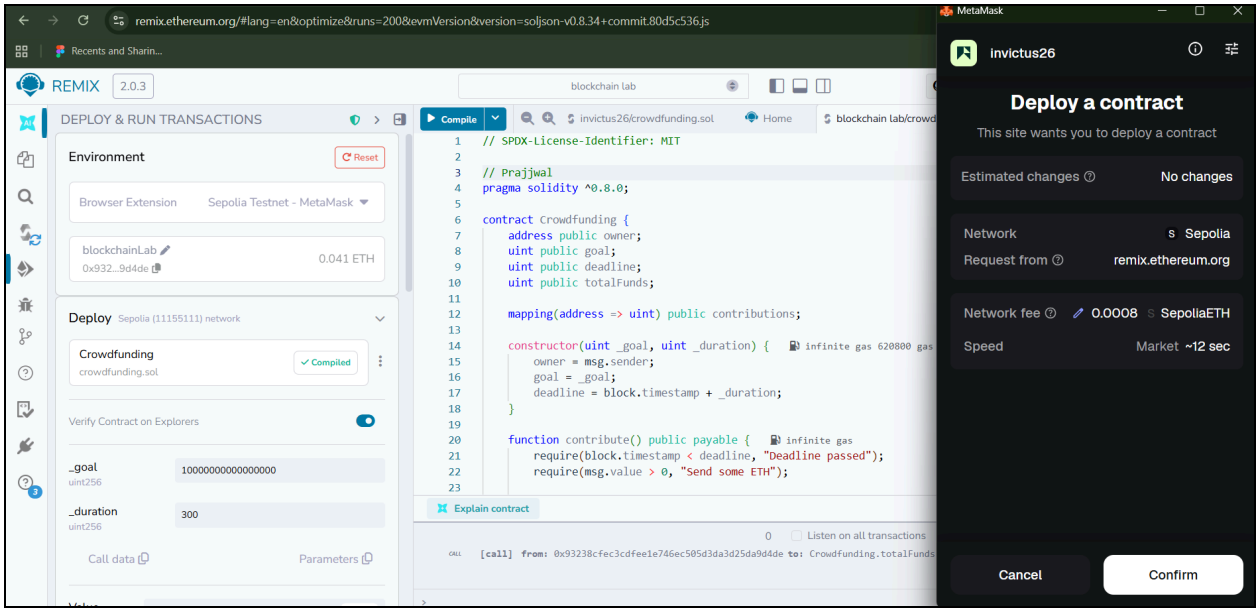
```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

contract Crowdfunding {
    address public owner;
    uint public goal;
    uint public deadline;
    uint public totalFunds;

    mapping(address => uint) public contributions;

    constructor(uint _goal, uint _duration) {
        owner = msg.sender;
        goal = _goal;
        deadline = block.timestamp + _duration;
    }

    function contribute() public payable {
        require(block.timestamp < deadline, "Deadline passed");
        require(msg.value > 0, "Send some ETH");

        contributions[msg.sender] += msg.value;
        totalFunds += msg.value;
    }
    function withdrawFunds() public {
        require(msg.sender == owner, "Only owner");
        require(totalFunds >= goal, "Goal not reached");

        (bool success, ) = payable(owner).call{value: totalFunds}("");
        require(success, "Transfer failed");
    }

    function refund() public {
        require(block.timestamp > deadline, "Not ended");
        require(totalFunds < goal, "Goal was reached");

        uint amount = contributions[msg.sender];
        require(amount > 0, "No contribution");
```

```
        contributions[msg.sender] = 0;

        (bool success, ) = payable(msg.sender).call{value: amount}("");
        require(success, "Refund failed");
    }

    function getBalance() public view returns (uint) {
        return address(this).balance;
    }
}
```

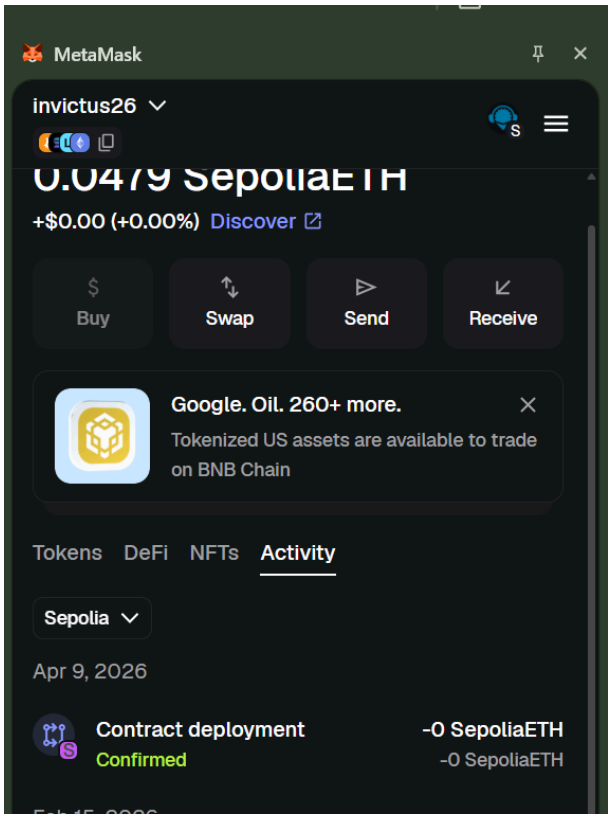
Step 3: save it as **crowdfunding.sol** and compile it



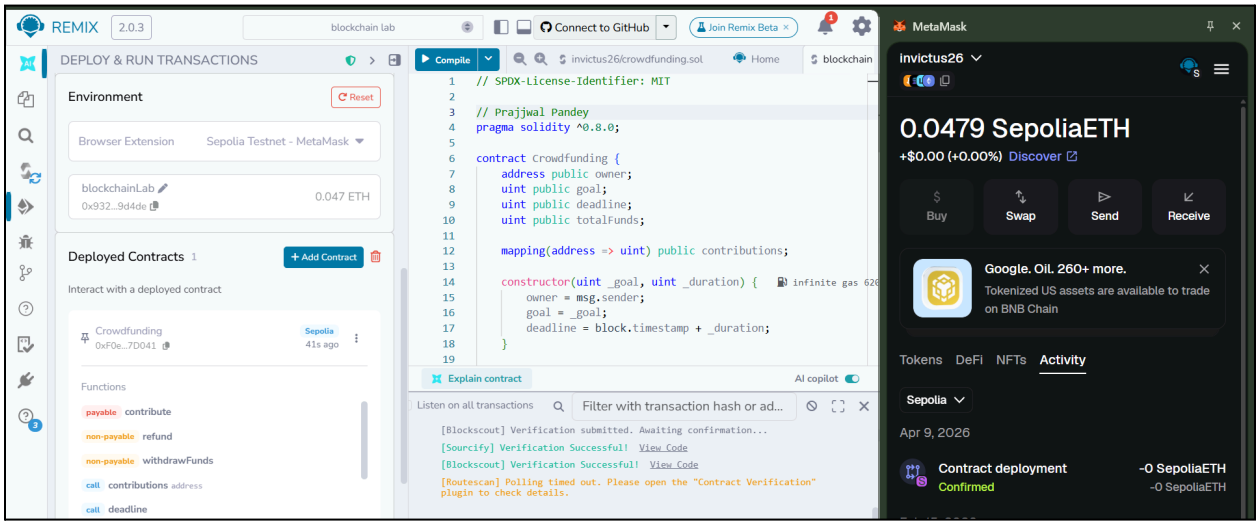
Step 4: deploy the contract with an initial goal and time limit. Confirm the transaction from metamask



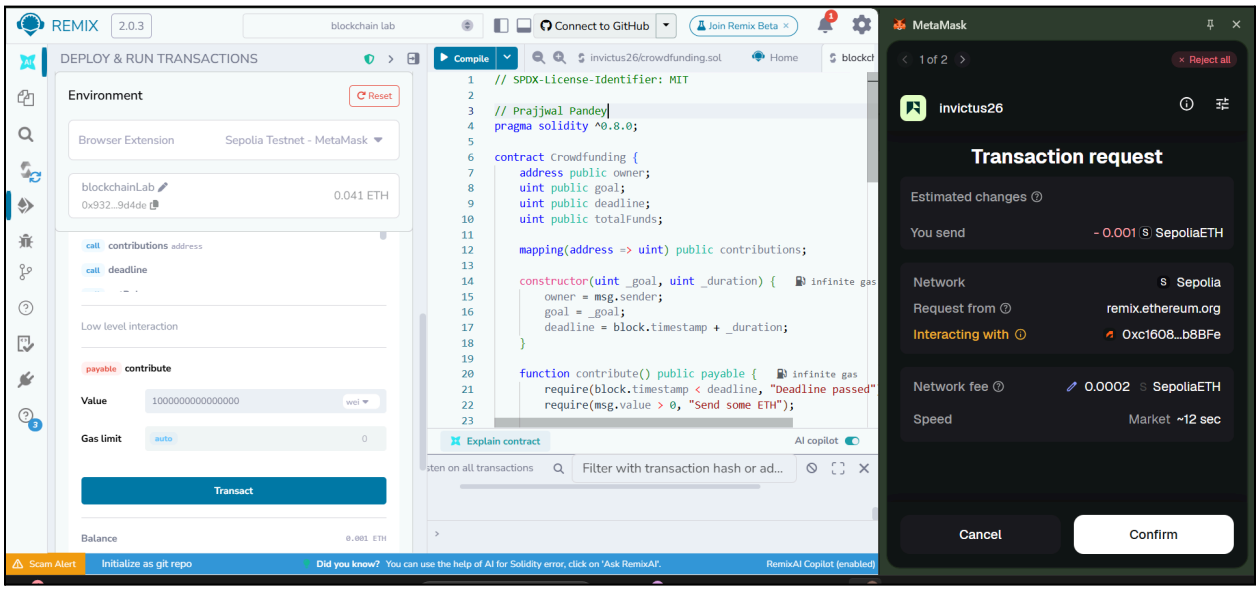
After deployment you can see: transaction recorded in metamask

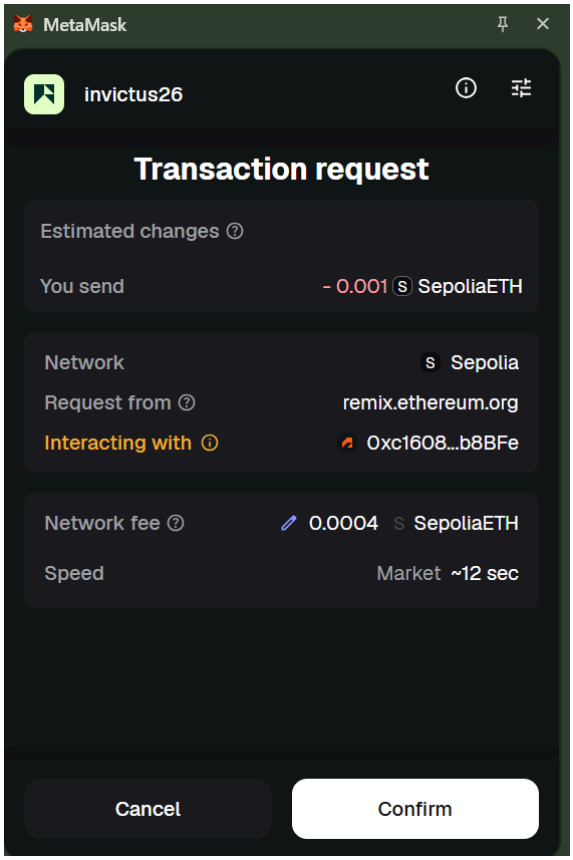


Step 5: After deployment in remix ide, we can see the deployed contract and its functions

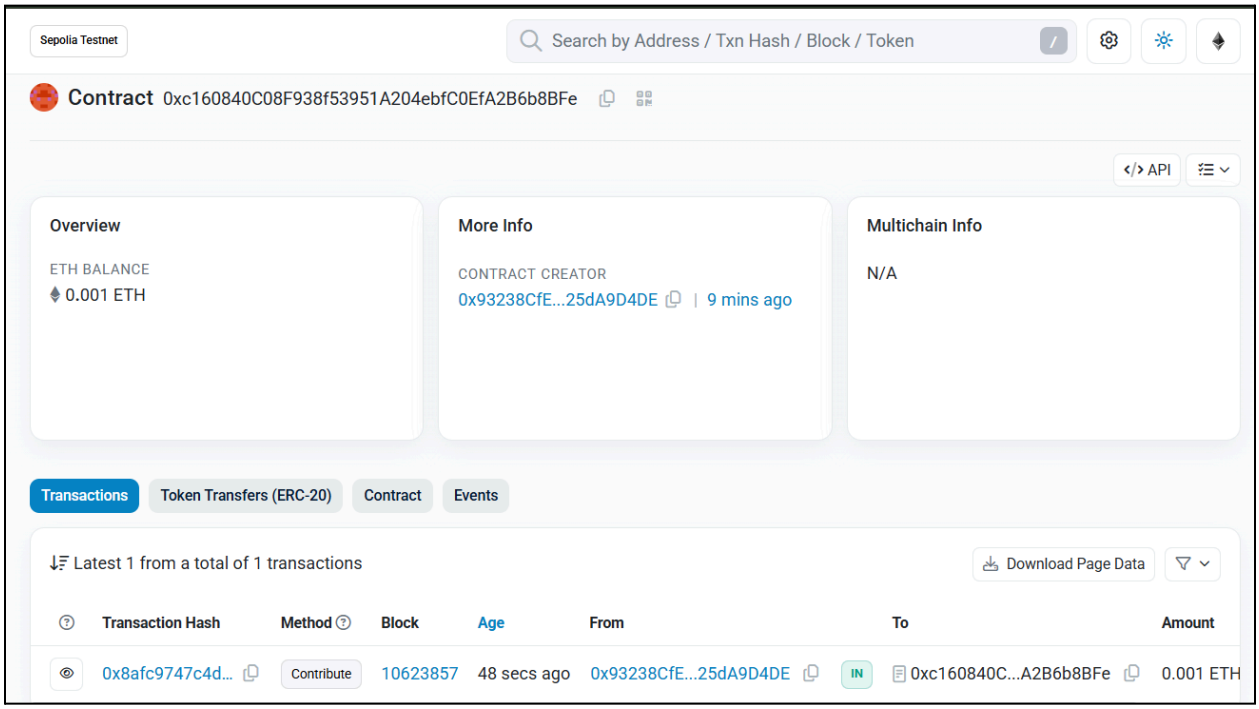


Step 6: Add a contribution of 0.01 ETH = 1000000000000000 wei. Transaction request appears in metamask. Click on confirm

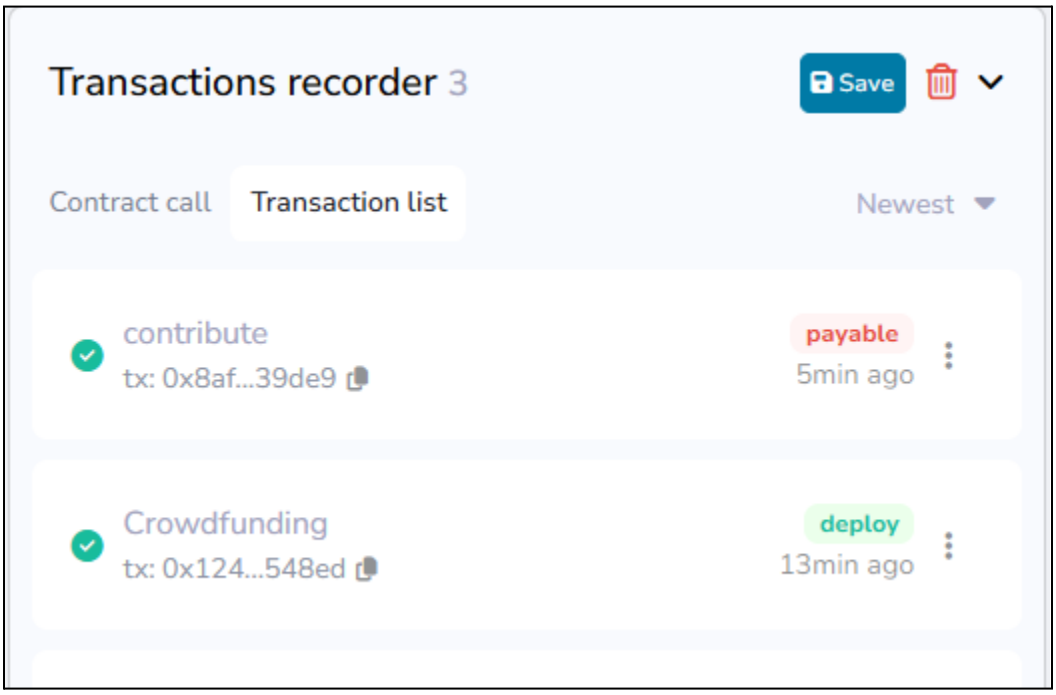




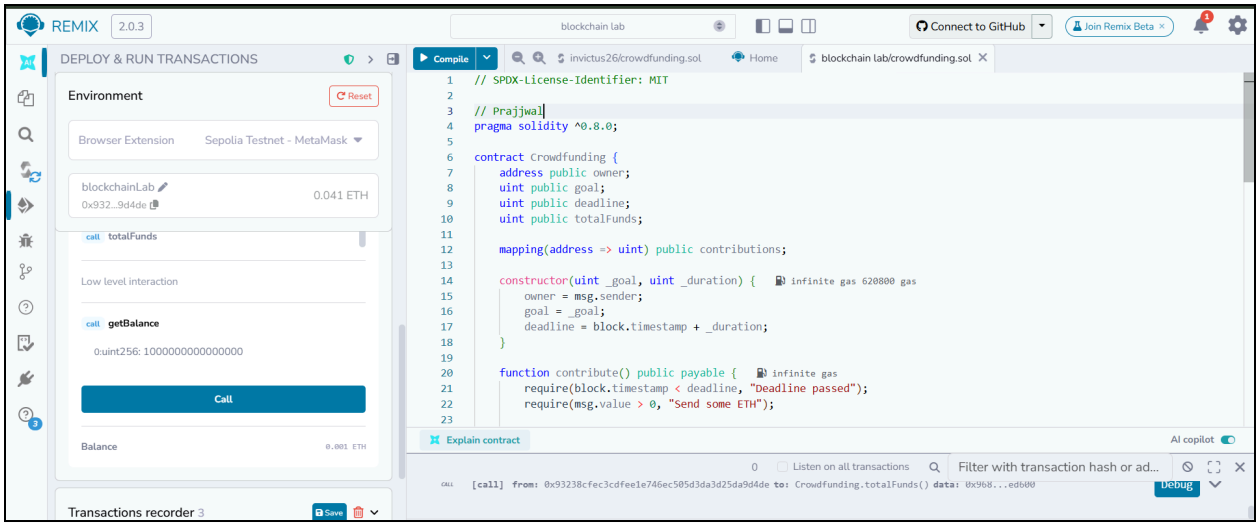
Step 7: copy paste the contracts address in https://sepolia.etherscan.io/ to see all transactions:



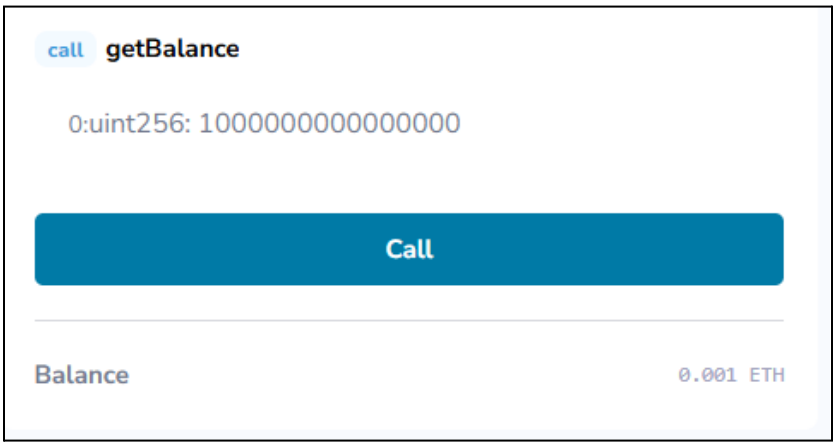
Also can see the transactions under all transactions in remix IDE



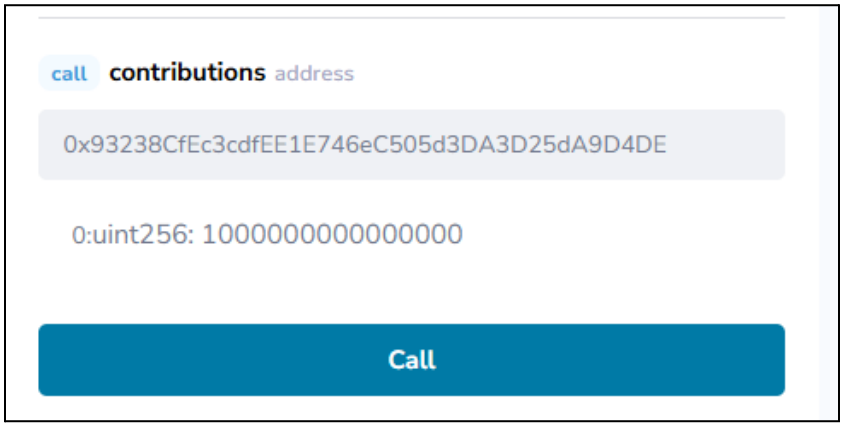
Step 8: use getBalance function to check the total funds received



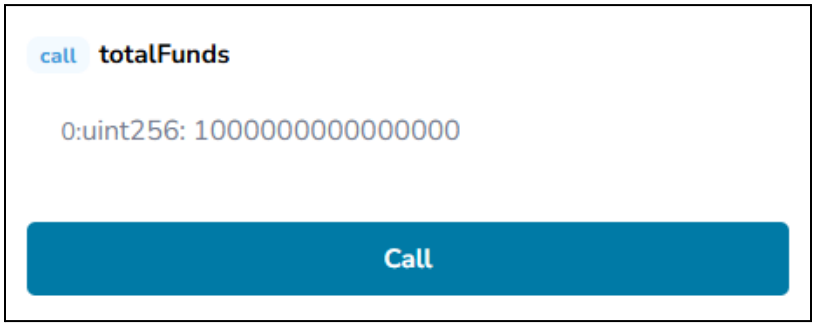
OUTPUT:



Step 9: Paste wallet address from metamask into contributions to see all transactions by the particular use



Step 10: use totalFunds function to see totalFunds recieved



CONCLUSION:

The integration of MetaMask with Remix IDE offers developers a hands-on approach to understanding how blockchain transactions work in practice. By deploying smart contracts on a testnet like Sepolia, developers can observe the complete transaction flow — from wallet authorization and gas estimation to block confirmation — without any financial risk. This setup bridges the gap between theoretical knowledge and real-world blockchain development, building familiarity with wallet management, network configuration, and contract interaction in a safe and controlled environment.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

BLOCKCHAIN EXPERIMENT - 8

AIM: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

THEORY:

1. Blockchain Technology Blockchain is a distributed ledger technology that maintains a continuously growing list of records (blocks), each cryptographically linked to the previous one. Since every node in the network holds an identical copy of the ledger, no single party can alter records without consensus from others, making it inherently tamper-proof. Core characteristics include decentralization (no central authority), immutability (records cannot be modified post-confirmation), transparency (all participants can view transactions), and security via cryptographic hashing algorithms like SHA-256.

2. Ethereum and Smart Contracts Ethereum extends basic blockchain functionality by enabling programmable logic through smart contracts — autonomous pieces of code deployed on-chain that execute automatically when specific conditions are triggered. These contracts are authored in Solidity, a statically-typed language, then compiled to EVM (Ethereum Virtual Machine) bytecode that runs identically across all network nodes. Every execution consumes *gas*, a fee mechanism that compensates validators and prevents resource abuse. Each deployed contract receives a unique address, and the `msg.sender` global variable identifies the calling account at runtime.

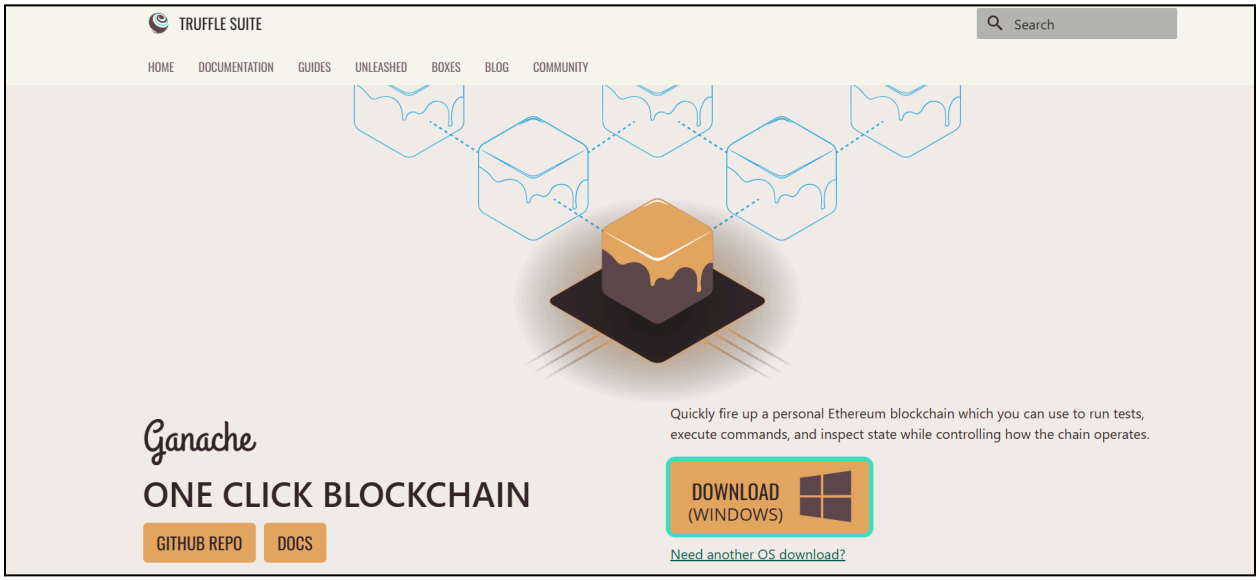
3. Ganache Ganache is a local Ethereum blockchain emulator primarily used during development and testing phases. It eliminates the need to interact with public networks by spinning up a private chain on the developer's machine, complete with pre-funded test accounts (10 accounts × 100 ETH each). Transactions are mined instantly, and its GUI provides real-time visibility into account balances, block data, and transaction logs. It exposes a local RPC endpoint (typically `http://127.0.0.1:7545`) that development tools can connect to seamlessly.

4. MetaMask MetaMask is a browser-based Ethereum wallet that bridges web applications and the blockchain. It manages account credentials, signs outgoing transactions, and routes them to whichever network it is configured for — in a development context, this is pointed to the local Ganache instance rather than the Ethereum mainnet, enabling cost-free testing with simulated ETH.

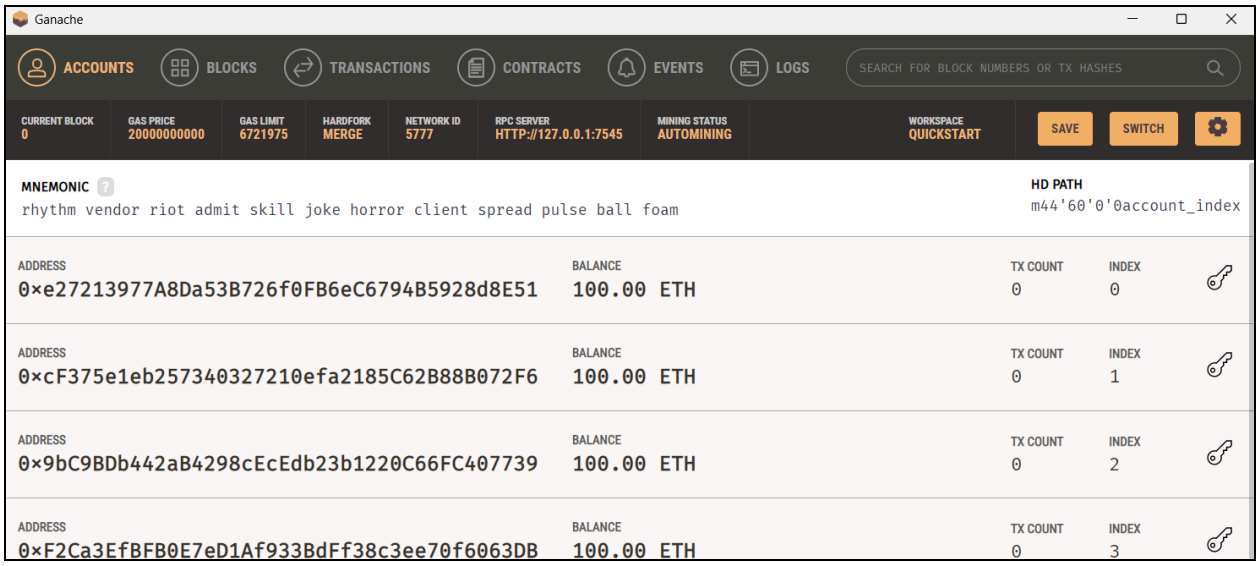
5. Remix IDE Remix is a browser-accessible development environment tailored for Solidity smart contract development. It bundles a code editor, compiler, deployment interface, and debugger into a single tool. By selecting the *Injected Provider* environment option, Remix delegates transaction signing to MetaMask, forwarding operations to whichever network MetaMask is currently connected to.

IMPLEMENTATION:

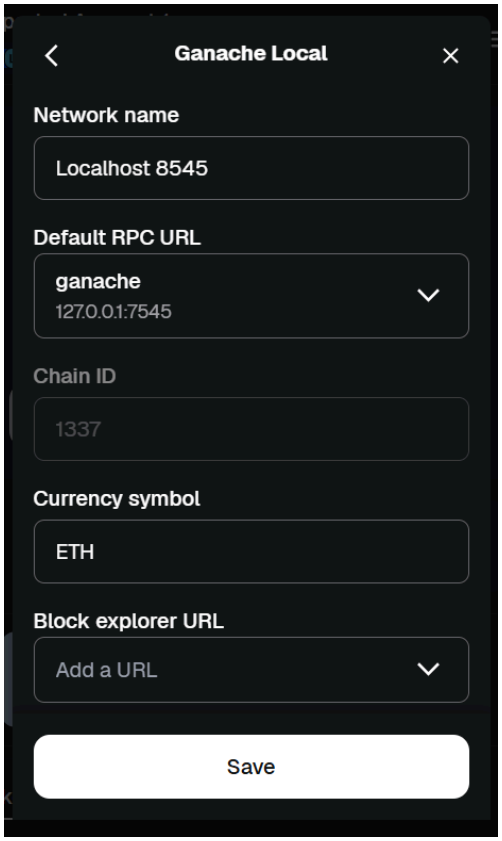
Step 1: Download and install ganache



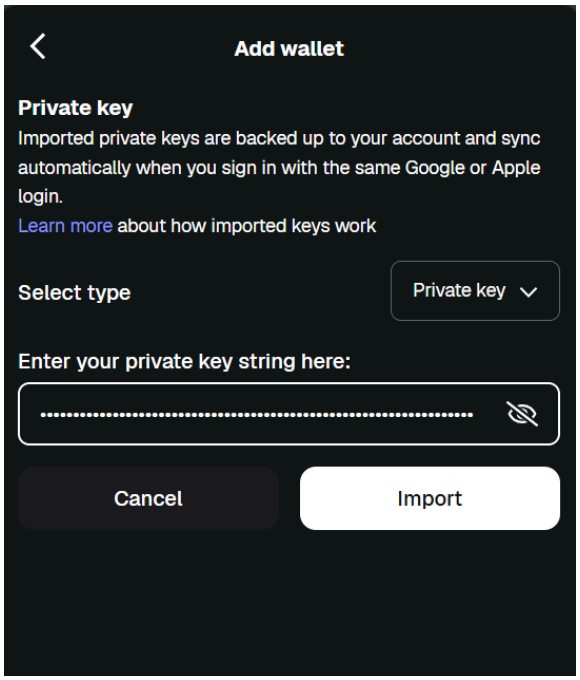
Once installed, open ganache and click on Quickstart Ethereum to start localhost.



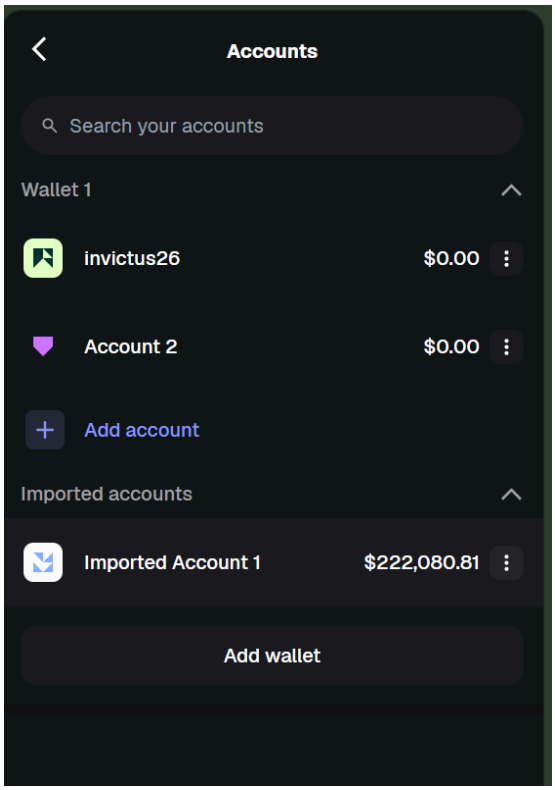
Step 2: In MetaMask, click on the networks tab and add a custom network.

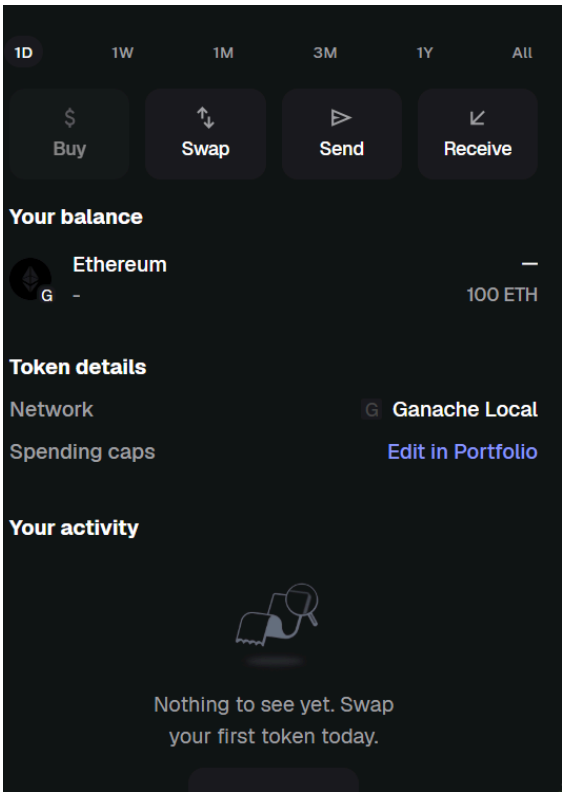


Step 3: Import ganache account into metamask



Wallet balance appears in metaMask account





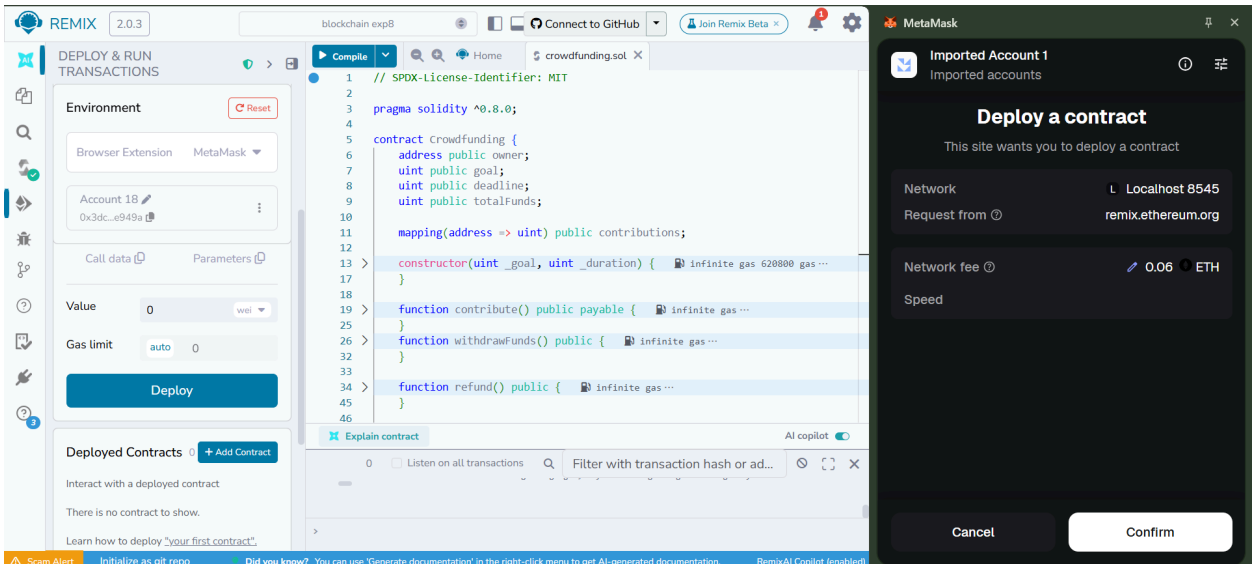
Step 4: In Remix IDE, create a new file **crowdfunding.sol**

Code:

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

contract Crowdfunding {
    address public owner;
    uint public goal;
    uint public deadline;
    uint public totalFunds;

    mapping(address => uint) public contributions;

    constructor(uint _goal, uint _duration) {
        owner = msg.sender;
        goal = _goal;
        deadline = block.timestamp + _duration;
```

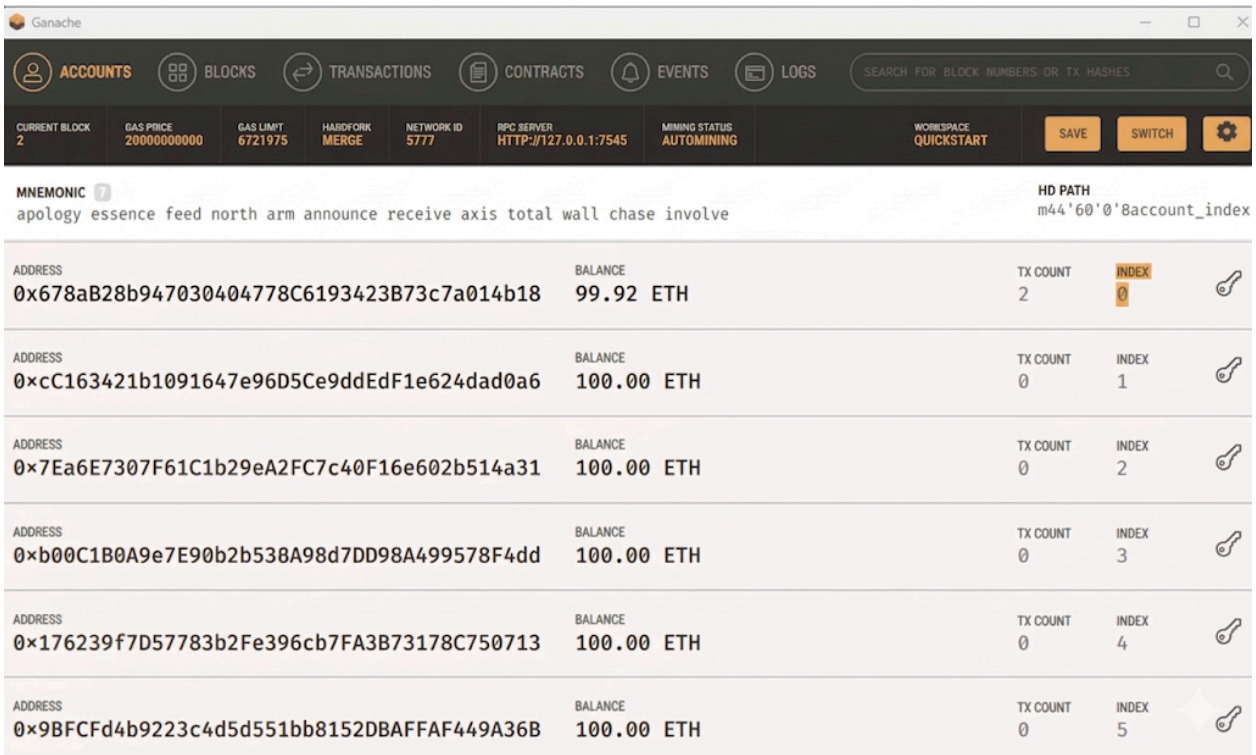
```
    }

    function contribute() public payable {
        require(block.timestamp < deadline, "Deadline passed");
        require(msg.value > 0, "Send some ETH");

        contributions[msg.sender] += msg.value;
        totalFunds += msg.value;
    }
    function withdrawFunds() public {
        require(msg.sender == owner, "Only owner");
        require(totalFunds >= goal, "Goal not reached");

        (bool success, ) = payable(owner).call{value: totalFunds}("");
        require(success, "Transfer failed");
    }

    function refund() public {
        require(block.timestamp > deadline, "Not ended");
        require(totalFunds < goal, "Goal was reached");

        uint amount = contributions[msg.sender];
        require(amount > 0, "No contribution");

        contributions[msg.sender] = 0;

        (bool success, ) = payable(msg.sender).call{value: amount}("");
        require(success, "Refund failed");
    }

    function getBalance() public view returns (uint) {
        return address(this).balance;
    }
}
```

Step 5: Compile and run the contact.
When prompted in metamask, confirm the transaction.



The contract is deployed!

The deducted amount can be seen in ganache app along with transaction count



CONCLUSION:

Ganache provides a simple and powerful environment to simulate a local Ethereum blockchain. By integrating it with **Metamask** and **Remix**, developers can deploy, test, and debug smart contracts efficiently before moving to a public network.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

BLOCKCHAIN EXPERIMENT - 9

AIM: Implement the Private Ethereum Blockchain using Geth.

THEORY:

Introduction to Ethereum

Ethereum is an open-source, decentralized blockchain platform that goes beyond simple cryptocurrency transactions by supporting programmable smart contracts and decentralized applications (DApps). It was proposed by Vitalik Buterin in 2013 and launched in 2015. Unlike Bitcoin which is primarily a digital currency, Ethereum functions as a programmable distributed computing platform where developers can deploy self-executing code called smart contracts. The native cryptocurrency of the Ethereum network is called Ether (ETH), which is used to pay for computational operations on the network, referred to as gas fees.

Ethereum maintains a global state that is updated with every transaction and smart contract execution. Every node in the network holds a copy of this state, ensuring decentralization and fault tolerance. The network uses an account-based model where each account has a balance and can interact with other accounts or smart contracts.

What is a Private Ethereum Network?

A private Ethereum network is a standalone blockchain that replicates the Ethereum protocol but operates in complete isolation from the public mainnet. Organizations, developers, and academic institutions use private networks to test and develop blockchain applications without incurring real transaction costs or exposing data to the public chain. All network parameters such as consensus mechanism, block time, gas limits, and initial account balances are fully customizable. This makes private networks ideal for controlled experimentation, proof-of-concept development, and educational purposes.

What is Geth (Go-Ethereum)?

Geth, short for Go-Ethereum, is the most widely used official implementation of the Ethereum protocol, written in the Go programming language. It allows a computer to function as a full Ethereum node capable of mining Ether, validating transactions, executing smart contracts, and interacting with the blockchain. Geth can be operated through a command-line interface, a built-in JavaScript console, or via JSON-RPC API calls. It supports both public Ethereum networks as well as custom private networks, making it the preferred tool for setting up and managing private Ethereum deployments.

Consensus Mechanism — Clique (Proof of Authority)

Private Ethereum networks typically use the Clique consensus algorithm instead of Proof of Work. Clique is a Proof of Authority (PoA) protocol where only a set of pre-approved accounts

called signers are authorized to produce and validate new blocks. Since block production does not involve solving computationally intensive puzzles, Clique is significantly faster and more energy efficient than traditional mining. The list of authorized signers is defined at the time of network creation and is embedded in the genesis block. This makes Clique well suited for private and consortium blockchain environments where participants are known and trusted.

Steps Involved in Implementing a Private Ethereum Network

1. Installing Geth: The first step involves downloading and installing the appropriate version of the Geth client on the system. Once installed, the binary is made accessible via the system PATH so it can be executed from any directory.

2. Creating Node Accounts: Each participating node requires an Ethereum account which consists of a public address and a corresponding private key stored in a keystore file. These accounts are created using the `geth account new` command and are protected by a password. The public addresses of these accounts are used in the genesis configuration.

3. Setting Up Password Files: To allow nodes to automatically unlock their accounts during startup without manual password entry each time, password files are created containing the account passwords in plain text.

4. Creating the Genesis Block: The genesis block is configured through a `genesis.json` file. This file defines the chain ID, consensus settings (Clique period and epoch), gas limit, initial Ether balances for each account, and the extradata field which encodes the authorized signer address. Every node in the network must be initialized with the same genesis file.

5. Initializing the Nodes: Each node's data directory is initialized using the `geth init` command along with the genesis file. This sets up the initial blockchain state and prepares the node to begin operating on the private network.

6. Running the Nodes: Node 1 is started as the signer node with mining enabled, responsible for sealing and producing new blocks. Node 2 is started as a member node that participates in the network but does not produce blocks. Both nodes are started with the Geth JavaScript console enabled for direct interaction.

7. Connecting the Nodes: Once both nodes are running, they are connected to each other using enode URLs. An enode is a unique identifier for a Geth node containing its public key and network address. Node 1's enode is retrieved using `admin.nodeInfo.enode` and then added to Node 2 using `admin.addPeer()`. Successful peering can be verified using `admin.peers`.

8. Sending Transactions: After the network is established and nodes are connected, Ether can be transferred between accounts using `eth.sendTransaction()` in the Geth console. The transaction is picked up by the signer node, included in a block, and confirmed on the private chain. Account balances can be verified using `eth.getBalance()`.

PROCEDURE:

Step 1: Install Geth

wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
mkdir -p ~/bin
cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113
export PATH="$HOME/bin:$PATH"
geth113 version

```
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
To set your Cloud Platform project in this session use `gcloud config set project [PROJECT_ID]`.
You can view your projects by running `gcloud projects list`.
g2022_prajjwal_pandey@cloudshell:~$ wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
mkdir -p ~/bin
cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113
export PATH="$HOME/bin:$PATH"
geth113 version
--2026-04-09 15:28:07--  https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
Resolving gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)... 20.60.40.164
Connecting to gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)|20.60.40.164|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 28248613 (27M) [application/octet-stream]
Saving to: 'geth-linux-amd64-1.13.14-2bd6bd01.tar.gz'

geth-linux-amd64-1.13.14-2bd6bd01.tar.g 100%[=========================================================================>]  26.94M   765KB/s    in 39s

2026-04-09 15:28:47 (715 KB/s) - 'geth-linux-amd64-1.13.14-2bd6bd01.tar.gz' saved [28248613/28248613]

geth-linux-amd64-1.13.14-2bd6bd01/
geth-linux-amd64-1.13.14-2bd6bd01/COPYING
geth-linux-amd64-1.13.14-2bd6bd01/geth
Geth
Version: 1.13.14-stable
Git Commit: 2bd6bd01d2e8561dd7fc21b631f4a34ac16627a1
Git Commit Date: 20240227
Architecture: amd64
Go Version: go1.21.6
Operating System: linux
GOPATH=/home/g2022_prajjwal_pandey/gopath:/google/gopath
GOROOT=
g2022_prajjwal_pandey@cloudshell:~$
```

Step 2: Create folders and accounts for both nodes

```
g2022_prajjwal_pandey@cloudshell:~$ mkdir -p ~/eth-private/node1 ~/eth-private/node2
g2022_prajjwal_pandey@cloudshell:~$ geth113 --datadir ~/eth-private/node1 account new
INFO [04-09|15:36:17.038] Maximum peer count                       ETH=50 total=50
INFO [04-09|15:36:17.041] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key:   0x93B0C547De2cF8228981Dd4a29aC3D0cA9a08859
Path of the secret key file: /home/g2022_prajjwal_pandey/eth-private/node1/keystore/UTC--2026-04-09T15-36-35.166702641Z--93b0c547de2cf8228981dd4a29ac3d0ca9a0885
9

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

g2022_prajjwal_pandey@cloudshell:~$
```

```
g2022_prajjwal_pandey@cloudshell:~$ geth113 --datadir ~/eth-private/node2 account new
INFO [04-09|15:38:49.547] Maximum peer count                       ETH=50 total=50
INFO [04-09|15:38:49.549] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key:   0x6406586b898212f380A55Bbf5e6596bD95825D3b
Path of the secret key file: /home/g2022_prajjwal_pandey/eth-private/node2/keystore/UTC--2026-04-09T15-38-54.774381074Z--6406586b898212f380a55bbf5e6596bd95825d3
b

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!
```

Step 3: Create password files

```
g2022_prajjwal_pandey@cloudshell:~$ echo "Prajjwal" > ~/eth-private/node1/password.txt
echo "Prajjwal" > ~/eth-private/node2/password.txt
g2022_prajjwal_pandey@cloudshell:~$
```

Step 4: Create genesis.json and verify it looks correct

```
NODE1="PASTE_NODE1_ADDRESS_WITHOUT_0x_HERE"
NODE2="PASTE_NODE2_ADDRESS_WITHOUT_0x_HERE"
cat > ~/eth-private/genesis.json << EOF
{
  "config": {
    "chainId": 1234,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "clique": {
      "period": 5,
      "epoch": 30000
    }
  },
  "difficulty": "1",
  "gasLimit": "8000000",
  "extradata":
"0x0000000000000000000000000000000000000000000000000000000000000000${NODE1}0
000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000",
  "alloc": {
    "0x${NODE1}": { "balance": "10000000000000000000" },
    "0x${NODE2}": { "balance": "10000000000000000000" }
  }
}
EOF
```

```
g2022_prajjwal_pandey@cloudshell:~$ NODE1="93B0C547De2cF8228981Dd4a29aC3D0cA9a08859"
NODE2="6406586b898212f380A55Bbf5e6596bD95825D3b"

cat > ~/eth-private/genesis.json << EOF
{
  "config": {
    "chainId": 1234,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "clique": {
      "period": 5,
      "epoch": 30000
    }
  },
  "difficulty": "1",
  "gasLimit": "8000000",
  "extradata": "0x0000000000000000000000000000000000000000000000000000000000000000${NODE1}0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000",
  "alloc": {
    "0x${NODE1}": { "balance": "10000000000000000000" },
    "0x${NODE2}": { "balance": "10000000000000000000" }
  }
}
EOF
g2022_prajjwal_pandey@cloudshell:~$
```

```
g2022_prajjwal_pandey@cloudshell:~$ cat ~/eth-private/genesis.json
{
  "config": {
    "chainId": 1234,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "clique": {
      "period": 5,
      "epoch": 30000
    }
  },
  "difficulty": "1",
  "gasLimit": "8000000",
  "extradata": "0x0000000000000000000000000000000000000000000000000000000000000000093B0C547De2cF8228981Dd4a29aC3D0cA9a08859000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000",
  "alloc": {
    "0x93B0C547De2cF8228981Dd4a29aC3D0cA9a08859": { "balance": "10000000000000000000" },
    "0x6406586b898212f380A55Bbf5e6596bD95825D3b": { "balance": "10000000000000000000" }
  }
}
g2022_prajjwal_pandey@cloudshell:~$
```

Step 5: Initialize both nodes

```
g2022_prajjwal_pandey@cloudshell:~$ geth113 --datadir ~/eth-private/node1 init ~/eth-private/genesis.json
INFO [04-09|15:49:20.956] Maximum peer count                       ETH=50 total=50
INFO [04-09|15:49:20.959] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-09|15:49:20.963] Set global gas cap                       cap=50,000,000
INFO [04-09|15:49:20.963] Initializing the KZG library             backend=gokzg
INFO [04-09|15:49:21.035] Defaulting to pebble as the backing database
INFO [04-09|15:49:21.035] Allocated cache and file handles         database=/home/g2022_prajjwal_pandey/eth-private/node1/geth/chaindata cache=16.00MiB handles=
16
INFO [04-09|15:49:21.104] Opened ancient database                  database=/home/g2022_prajjwal_pandey/eth-private/node1/geth/chaindata/ancient/chain readonly=
false
INFO [04-09|15:49:21.105] State schema set to default              scheme=hash
INFO [04-09|15:49:21.105] Writing custom genesis block
INFO [04-09|15:49:21.109] Persisted trie from memory database      nodes=3 size=411.00B time=3.93038ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.0
0B
INFO [04-09|15:49:21.152] Successfully wrote genesis state         database=chaindata hash=e03c67..887468
INFO [04-09|15:49:21.152] Defaulting to pebble as the backing database
INFO [04-09|15:49:21.152] Allocated cache and file handles         database=/home/g2022_prajjwal_pandey/eth-private/node1/geth/lightchaindata cache=16.00MiB han
dles=16
INFO [04-09|15:49:21.219] Opened ancient database                  database=/home/g2022_prajjwal_pandey/eth-private/node1/geth/lightchaindata/ancient/chain read
only=false
INFO [04-09|15:49:21.219] State schema set to default              scheme=hash
INFO [04-09|15:49:21.219] Writing custom genesis block
INFO [04-09|15:49:21.223] Persisted trie from memory database      nodes=3 size=411.00B time=4.12979ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.0
0B
INFO [04-09|15:49:21.266] Successfully wrote genesis state         database=lightchaindata hash=e03c67..887468
g2022_prajjwal_pandey@cloudshell:~$
```

```
g2022_prajjwal_pandey@cloudshell:~$ geth113 --datadir ~/eth-private/node2 init ~/eth-private/genesis.json
INFO [04-09|15:49:52.109] Maximum peer count                       ETH=50 total=50
INFO [04-09|15:49:52.111] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-09|15:49:52.115] Set global gas cap                       cap=50,000,000
INFO [04-09|15:49:52.116] Initializing the KZG library             backend=gokzg
INFO [04-09|15:49:52.189] Defaulting to pebble as the backing database
INFO [04-09|15:49:52.189] Allocated cache and file handles         database=/home/g2022_prajjwal_pandey/eth-private/node2/geth/chaindata cache=16.00MiB handles=
16
INFO [04-09|15:49:52.257] Opened ancient database                  database=/home/g2022_prajjwal_pandey/eth-private/node2/geth/chaindata/ancient/chain readonly=
false
INFO [04-09|15:49:52.257] State schema set to default              scheme=hash
INFO [04-09|15:49:52.257] Writing custom genesis block
INFO [04-09|15:49:52.262] Persisted trie from memory database      nodes=3 size=411.00B time=4.507603ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.
00B
INFO [04-09|15:49:52.306] Successfully wrote genesis state         database=chaindata hash=e03c67..887468
INFO [04-09|15:49:52.306] Defaulting to pebble as the backing database
INFO [04-09|15:49:52.306] Allocated cache and file handles         database=/home/g2022_prajjwal_pandey/eth-private/node2/geth/lightchaindata cache=16.00MiB han
dles=16
INFO [04-09|15:49:52.373] Opened ancient database                  database=/home/g2022_prajjwal_pandey/eth-private/node2/geth/lightchaindata/ancient/chain read
only=false
INFO [04-09|15:49:52.373] State schema set to default              scheme=hash
INFO [04-09|15:49:52.373] Writing custom genesis block
INFO [04-09|15:49:52.378] Persisted trie from memory database      nodes=3 size=411.00B time=4.105657ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.
00B
INFO [04-09|15:49:52.420] Successfully wrote genesis state         database=lightchaindata hash=e03c67..887468
g2022_prajjwal_pandey@cloudshell:~$
```

Step 6: Run Node 1 (signer)

```
cloudshell ×  cloudshell ×  +
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
To set your Cloud Platform project in this session use `gcloud config set project [PROJECT_ID]`.
You can view your projects by running `gcloud projects list`.
g2022_prajjwal_pandey@cloudshell:~$ export PATH="$HOME/bin:$PATH"
NODE1="93B0C547De2cF8228981Dd4a29aC3D0cA9a08859"
g2022_prajjwal_pandey@cloudshell:~$
```

```
g2022_prajjwal_pandey@cloudshell:~$ geth113 --datadir ~/eth-private/node1 \
  --networkid 1234 \
  --port 30303 \
  --http --http.port 8545 \
  --unlock 0x${NODE1} \
  --password ~/eth-private/node1/password.txt \
  --mine \
  --miner.etherbase 0x${NODE1} \
  --allow-insecure-unlock \
  console
INFO [04-09|15:52:04.688] Maximum peer count                       ETH=50 total=50
INFO [04-09|15:52:04.691] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-09|15:52:04.694] Set global gas cap                       cap=50,000,000
INFO [04-09|15:52:04.695] Initializing the KZG library             backend=gokzg
INFO [04-09|15:52:04.767] Allocated trie memory caches             clean=154.00MiB dirty=256.00MiB
INFO [04-09|15:52:04.767] Using pebble as the backing database
INFO [04-09|15:52:04.767] Allocated cache and file handles         database=/home/g2022_prajjwal_pandey/eth-private/node1/geth/chaindata cache=512.00MiB handles
=524,288
INFO [04-09|15:52:04.797] Opened ancient database                  database=/home/g2022_prajjwal_pandey/eth-private/node1/geth/chaindata/ancient/chain readonly=
false
INFO [04-09|15:52:04.798] State scheme set to already existing     scheme=hash
INFO [04-09|15:52:04.799] Initialising Ethereum protocol           network=1234 dbversion=<nil>
INFO [04-09|15:52:04.803]
INFO [04-09|15:52:04.803] ----------------------------------------------------------------------------------------------------------------------------------------
------------------
INFO [04-09|15:52:04.803] Chain ID:  1234 (unknown)
INFO [04-09|15:52:04.803] Consensus: Clique (proof-of-authority)
INFO [04-09|15:52:04.803]
INFO [04-09|15:52:04.803] Pre-Merge hard forks (block based):
INFO [04-09|15:52:04.803]  - Homestead:                   #0         (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/h
omestead.md)
INFO [04-09|15:52:04.803]  - Tangerine Whistle (EIP 150): #0         (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/t
angerine-whistle.md)
INFO [04-09|15:52:04.803]  - Spurious Dragon/1 (EIP 155): #0         (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/s
purious-dragon.md)
INFO [04-09|15:52:04.803]  - Spurious Dragon/2 (EIP 158): #0         (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/s
```

```
INFO [04-09|15:52:04.971] Starting peer-to-peer node               instance=Geth/v1.13.14-stable-2bd6bd01/linux-amd64/go1.21.6
INFO [04-09|15:52:04.991] New local node record                    seq=1,775,749,924,991 id=a9a288d6769625c8 ip=127.0.0.1 udp=30303 tcp=30303
INFO [04-09|15:52:04.994] IPC endpoint opened                      url=/home/g2022_prajjwal_pandey/eth-private/node1/geth.ipc
INFO [04-09|15:52:04.994] Started P2P networking                   self=enode://acfe33406144ee52760abb42a72a912f3123188c228f201424f105b39118698f0fc78a77e454680
d8898b9ca388cf647de953ae695bb21251e5f610baa5bb7d@127.0.0.1:30303
INFO [04-09|15:52:04.994] Generated JWT secret                     path=/home/g2022_prajjwal_pandey/eth-private/node1/geth/jwtsecret
INFO [04-09|15:52:04.995] HTTP server started                      endpoint=127.0.0.1:8545 auth=false prefix= cors= vhosts=localhost
INFO [04-09|15:52:04.996] WebSocket enabled                        url=ws://127.0.0.1:8551
INFO [04-09|15:52:04.996] HTTP server started                      endpoint=127.0.0.1:8551 auth=true  prefix= cors=localhost vhosts=localhost
INFO [04-09|15:52:06.133] Unlocked account                         address=0x93B0C547De2cF8228981Dd4a29aC3D0cA9a08859
INFO [04-09|15:52:06.133] Legacy pool tip threshold updated        tip=0
INFO [04-09|15:52:06.133] Legacy pool tip threshold updated        tip=1,000,000,000
INFO [04-09|15:52:06.133] Snap sync complete, auto disabling
INFO [04-09|15:52:06.134] Commit new sealing work                  number=1 sealhash=16e9bf..5ee3c4 txs=0 gas=0 fees=0 elapsed="200.259µs"
INFO [04-09|15:52:06.144] Successfully sealed new block            number=1 sealhash=16e9bf..5ee3c4 hash=6baf2d..432713 elapsed=10.718ms
INFO [04-09|15:52:06.145] Commit new sealing work                  number=2 sealhash=95fbef..db0a46 txs=0 gas=0 fees=0 elapsed="583.298µs"
INFO [04-09|15:52:06.149] Indexed transactions                     blocks=2 txs=0 tail=0 elapsed=3.670ms
Welcome to the Geth JavaScript console!

instance: Geth/v1.13.14-stable-2bd6bd01/linux-amd64/go1.21.6
coinbase: 0x93b0c547de2cf8228981dd4a29ac3d0ca9a08859
at block: 1 (Thu Apr 09 2026 15:52:06 GMT+0000 (UTC))
 datadir: /home/g2022_prajjwal_pandey/eth-private/node1
 modules: admin:1.0 clique:1.0 debug:1.0 engine:1.0 eth:1.0 miner:1.0 net:1.0 rpc:1.0 txpool:1.0 web3:1.0

To exit, press ctrl-d or type exit
> INFO [04-09|15:52:07.394] New local node record                    seq=1,775,749,924,992 id=a9a288d6769625c8 ip=34.126.99.110 udp=30303 tcp=30303
INFO [04-09|15:52:11.010] Successfully sealed new block            number=2 sealhash=95fbef..db0a46 hash=4ad89a..cdb782 elapsed=4.864s
INFO [04-09|15:52:11.010] Commit new sealing work                  number=3 sealhash=6684cc..c8ae91 txs=0 gas=0 fees=0 elapsed="585.63µs"
INFO [04-09|15:52:15.529] Looking for peers                        peercount=0 tried=83 static=0
INFO [04-09|15:52:16.010] Successfully sealed new block            number=3 sealhash=6684cc..c8ae91 hash=d3d30d..b6a37c elapsed=4.999s
INFO [04-09|15:52:16.011] Commit new sealing work                  number=4 sealhash=2e769c..084439 txs=0 gas=0 fees=0 elapsed="672.511µs"
INFO [04-09|15:52:21.010] Successfully sealed new block            number=4 sealhash=2e769c..084439 hash=f585e4..392e05 elapsed=4.998s
INFO [04-09|15:52:21.010] Commit new sealing work                  number=5 sealhash=998266..47aa05 txs=0 gas=0 fees=0 elapsed="619.955µs"
INFO [04-09|15:52:25.681] Looking for peers                        peercount=0 tried=68 static=0
INFO [04-09|15:52:26.010] Successfully sealed new block            number=5 sealhash=998266..47aa05 hash=5865a2..bceb3e elapsed=4.999s
```

Step 7: Open new terminal tab and run Node 2 (member)

```
g2022_prajjwal_pandey@cloudshell:~$ export PATH="$HOME/bin:$PATH"
NODE2="6406586b898212f380A55Bbf5e6596bD95825D3b"
```

```
g2022_prajjwal_pandey@cloudshell:~$ geth113 --datadir ~/eth-private/node2   --networkid 1234   --port 30304   --http --http.port 8546   --unlock 0x${NODE2}   --
password ~/eth-private/node2/password.txt   --allow-insecure-unlock   console
INFO [04-09|16:10:33.186] Maximum peer count                       ETH=50 total=50
INFO [04-09|16:10:33.189] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-09|16:10:33.193] Set global gas cap                       cap=50,000,000
INFO [04-09|16:10:33.193] Initializing the KZG library             backend=gokzg
INFO [04-09|16:10:33.268] Allocated trie memory caches             clean=154.00MiB dirty=256.00MiB
INFO [04-09|16:10:33.268] Using pebble as the backing database
INFO [04-09|16:10:33.268] Allocated cache and file handles         database=/home/g2022_prajjwal_pandey/eth-private/node2/geth/chaindata cache=512.00MiB handles
=524,288
INFO [04-09|16:10:33.300] Opened ancient database                  database=/home/g2022_prajjwal_pandey/eth-private/node2/geth/chaindata/ancient/chain readonly=
false
INFO [04-09|16:10:33.300] State scheme set to already existing     scheme=hash
INFO [04-09|16:10:33.301] Initialising Ethereum protocol           network=1234 dbversion=8
INFO [04-09|16:10:33.302]
INFO [04-09|16:10:33.302] ---------------------------------------------------------------------------------------------------------------------------------------
------------------
INFO [04-09|16:10:33.302] Chain ID:  1234 (unknown)
INFO [04-09|16:10:33.302] Consensus: Clique (proof-of-authority)
INFO [04-09|16:10:33.302]
INFO [04-09|16:10:33.302] Pre-Merge hard forks (block based):
INFO [04-09|16:10:33.302]  - Homestead:                    #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/h
omestead.md)
INFO [04-09|16:10:33.302]  - Tangerine Whistle (EIP 150): #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/t
angerine-whistle.md)
INFO [04-09|16:10:33.302]  - Spurious Dragon/1 (EIP 155): #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/s
purious-dragon.md)
INFO [04-09|16:10:33.302]  - Spurious Dragon/2 (EIP 158): #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/s
purious-dragon.md)
INFO [04-09|16:10:33.302]  - Byzantium:                    #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/b
yzantium.md)
INFO [04-09|16:10:33.302]  - Constantinople:               #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/c
onstantinople.md)
INFO [04-09|16:10:33.302]  - Petersburg:                   #0        (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/p
etersburg.md)
```

```
INFO [04-09|16:10:33.302] Post-Merge hard forks (timestamp based):
INFO [04-09|16:10:33.302]
INFO [04-09|16:10:33.302] ------------------------------------------------------------------------------------------------------------------------------------------------------
------------------
INFO [04-09|16:10:33.302]
INFO [04-09|16:10:33.302] Loaded most recent local block           number=0 hash=e03c67..887468 td=1 age=57y1mo2d
WARN [04-09|16:10:33.303] Loaded snapshot journal                  diffs=missing
INFO [04-09|16:10:33.303] Initialized transaction indexer          range="last 2350000 blocks"
INFO [04-09|16:10:33.303] Loaded local transaction journal         transactions=0 dropped=0
INFO [04-09|16:10:33.325] Enabled snap sync                        head=0 hash=e03c67..887468
INFO [04-09|16:10:33.326] Gasprice oracle is ignoring threshold set threshold=2
WARN [04-09|16:10:33.330] Unclean shutdown detected                booted=2026-04-09T16:01:48+0000 age=8m45s
WARN [04-09|16:10:33.331] Engine API enabled                       protocol=eth
WARN [04-09|16:10:33.331] Engine API started but chain not configured for merge yet
INFO [04-09|16:10:33.331] Starting peer-to-peer node               instance=Geth/v1.13.14-stable-2bd6bd01/linux-amd64/go1.21.6
INFO [04-09|16:10:33.363] New local node record                    seq=1,775,750,508,266 id=40d680c8eeed08a8 ip=127.0.0.1 udp=30304 tcp=30304
INFO [04-09|16:10:33.366] Started P2P networking                   self=enode://39855208b63767867bd73a27f0fee7a0a210588ad82c875d858df2c94a59e2d547e132ef6deda610
lcfba712be53477af7154b6ab8e6a03b814bf90fcdd60526@127.0.0.1:30304
INFO [04-09|16:10:33.366] IPC endpoint opened                      url=/home/g2022_prajjwal_pandey/eth-private/node2/geth.ipc
INFO [04-09|16:10:33.366] Loaded JWT secret file                   path=/home/g2022_prajjwal_pandey/eth-private/node2/geth/jwtsecret crc32=0x10258536
INFO [04-09|16:10:33.368] HTTP server started                      endpoint=127.0.0.1:8546 auth=false prefix= cors= vhosts=localhost
INFO [04-09|16:10:33.369] WebSocket enabled                        url=ws://127.0.0.1:8551
INFO [04-09|16:10:33.369] HTTP server started                      endpoint=127.0.0.1:8551 auth=true  prefix= cors=localhost vhosts=localhost
INFO [04-09|16:10:34.473] Unlocked account                         address=0x6406586b898212f380A55Bbf5e6596bD95825D3b
WARN [04-09|16:10:34.526] Served eth_coinbase                      reqid=3 duration="30.392µs" err="etherbase must be explicitly specified"
Welcome to the Geth JavaScript console!

instance: Geth/v1.13.14-stable-2bd6bd01/linux-amd64/go1.21.6
at block: 0 (Thu Jan 01 1970 00:00:00 GMT+0000 (UTC))
 datadir: /home/g2022_prajjwal_pandey/eth-private/node2
 modules: admin:1.0 clique:1.0 debug:1.0 engine:1.0 eth:1.0 miner:1.0 net:1.0 rpc:1.0 txpool:1.0 web3:1.0

To exit, press ctrl-d or type exit
> INFO [04-09|16:10:35.821] New local node record                    seq=1,775,750,508,267 id=40d680c8eeed08a8 ip=34.126.99.110 udp=30304 tcp=30304
INFO [04-09|16:10:43.413] Looking for peers                        peercount=0 tried=52 static=0
INFO [04-09|16:10:53.510] Looking for peers                        peercount=0 tried=29 static=0
```

Step 8: Connect the two nodes

In Node 1 console, get its enode

```
> admin.nodeInfo.enode
"enode://acfe33406144ee52760abb42a72a912f3123188c228f201424f105b391186998f0fc78a77e454680d8898b9ca388cf647de953ae695bb21251e5f610baa5bb7d@34.126.99.110:30303"
> INFO [04-09|16:22:47.013] Successfully sealed new block            number=231 sealhash=bb4346..610566 hash=d1fc70..84fdc7 elapsed=5.001s
INFO [04-09|16:22:47.014] Commit new sealing work                  number=232 sealhash=8aca46..08c3ca txs=0 gas=0 fees=0 elapsed="538.827µs"
INFO [04-09|16:22:52.010] Successfully sealed new block            number=232 sealhash=8aca46..08c3ca hash=d65637..b7846e elapsed=4.996s
INFO [04-09|16:22:52.010] Commit new sealing work                  number=233 sealhash=e1df14..1d6155 txs=0 gas=0 fees=0 elapsed="485.675µs"
INFO [04-09|16:22:52.270] Looking for peers                        peercount=0 tried=60 static=0
INFO [04-09|16:22:57.011] Successfully sealed new block            number=233 sealhash=e1df14..1d6155 hash=0be857..1fcf8d elapsed=5.001s
INFO [04-09|16:22:57.012] Commit new sealing work                  number=234 sealhash=51ca87..5b8c95 txs=0 gas=0 fees=0 elapsed="501.891µs"
```

Now, switch to Node 2 console and paste the Node 1 enode

```
> admin.addPeer("enode://acfe33406144ee52760abb42a72a912f3123188c228f201424f105b391186998f0fc78a77e454680d8898b9ca388cf647de953ae695bb21251e5f610baa5bb7d@34.126
.99.110:30303")
true
> INFO [04-09|16:25:38.021] Looking for peers                        peercount=0 tried=45 static=1
```

In Node 2 console, check they are connected

```
> admin.peers
[{
    caps: ["eth/68", "eth/69", "snap/1"],
    enode: "enode://00162513d2f3e10b7ccd0dc4a3d139842e3f9681147bf7367aa7fe40421ce3904f8a4ea0db1254cc2d0d8a0dfec08457c3938017bf424a952c92c70931183abb@47.187.149.99:31404",
    id: "26184e724beb8194376e59f1fd0493aee8d3cb40469857695c966147c8b8aeeb",
    name: "Nethermind/v1.36.0+31cb81b7/linux-x64/dotnet10.0.1",
    network: {
      inbound: false,
      localAddress: "10.88.0.4:55586",
      remoteAddress: "47.187.149.99:31404",
      static: false,
      trusted: false
    },
    protocols: {
      eth: "handshake",
      snap: "handshake"
    }
}]
```

Step 9: Send a transaction

In Node 1 console, send transaction to Node 2

```
> eth.sendTransaction({
......   from: eth.accounts[0],
......   to: "0x5fd63344c1f40712A9A85d9d7e2B304a6C558716",
......   value: web3.toWei(1, "ether")
...... })
INFO [04-05|13:43:48.276] Setting new local account                address=0x1D8b6d4F6C677D2fB66CA97FbD81bED2cD23c4D3
INFO [04-05|13:43:48.276] Submitted transaction                    hash=0x566ac1b0f280ebd7b6d4a68991416a3624ed3ec43bd24f8a08af6badab79089 from=0x1D8b6d4F6C677D2fB66CA97FbD81b
ED2cD23c4D3 nonce=0 recipient=0x5fd63344c1f40712A9A85d9d7e2B304a6C558716 value=1,000,000,000,000,000,000
"0x566ac1b0f280ebd7b6d4a68991416a3624ed3ec43bd24f8a08af6badab79089"
```

and check the balance of Node 2

```
> eth.getBalance("0x5fd63344c1f40712A9A85d9d7e2B304a6C558716")
10100000000000000000
```

CONCLUSION

This experiment successfully demonstrated the implementation of a private Ethereum blockchain network using the Geth (Go-Ethereum) client. A two-node private network was set up from scratch, beginning with the installation of Geth, creation of individual node accounts, and configuration of the genesis block using the Clique Proof of Authority consensus mechanism. Both nodes were initialized with the same genesis file, started with appropriate network parameters, and successfully connected to each other using enode URLs.

Once the network was operational, the signer node (Node 1) began producing and sealing blocks at regular intervals, confirming that the consensus mechanism was functioning correctly. The two nodes were peered together and their connection was verified using the `admin.peers` command. A transaction was then executed from Node 1 to Node 2 using `eth.sendTransaction()`, and the updated balance of Node 2 was confirmed using `eth.getBalance()`, validating that transactions are being processed and recorded on the private chain.

Through this experiment, a practical understanding of how Ethereum nodes operate, how private blockchain networks are configured, and how peer-to-peer communication works between nodes was gained. It also highlighted the advantages of using Proof of Authority consensus in private networks, particularly in terms of faster block production and reduced computational overhead compared to Proof of Work. Overall, this experiment provided hands-on exposure to core blockchain concepts including genesis block configuration, node initialization, peer discovery, and on-chain transaction verification.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

BLOCKCHAIN EXPERIMENT - 10

AIM: Explore the Cryptocurrency Landscape

THEORY:

This experiment explores the **cryptocurrency landscape** by combining real-world market data analysis with blockchain technology research. The focus asset for this analysis is **Chainlink (LINK)**, and data was collected from **three reputable sources** as per the lab requirement:

1. **CoinGecko API** – Provided historical price data, market capitalization, and trading volumes in a structured JSON format, enabling direct processing in Python.
2. **CoinMarketCap** – Offered real-time and historical price charts, circulating supply, and market rank for visual cross-verification.
3. **Crypto.com** – Supplied quick-access market statistics, simplified price trends, and cryptocurrency news updates.

Cryptocurrency Landscape

1. **Definition** – Cryptocurrencies are decentralized digital assets built on blockchain networks, enabling secure, transparent, and immutable transactions without central authority.
2. **Market Diversity** – Thousands of cryptocurrencies exist, each serving unique roles:
    - **Store of Value** – Bitcoin, Litecoin.
    - **Utility Tokens** – Chainlink, Ethereum.
    - **Stablecoins** – USDT, USDC.
    - **Governance Tokens** – UNI, AAVE.
3. **Volatility** – Prices are highly sensitive to market sentiment, technological developments, and regulatory updates.
4. **Data Analysis Importance** – Tracking historical trends, market capitalization, and trading volume is essential for understanding asset performance.
5. **Advancements** – Growing use of **DeFi platforms**, **NFT ecosystems**, **layer-2 scaling**, and **cross-chain protocols**.

Data Collection Process

The program used the **CoinGecko API** to automate historical data fetching for Chainlink. The retrieved data was processed with **Pandas** to compute:

- **Daily Returns** – Percentage change between consecutive days to measure volatility.
- **Moving Averages (MA7, MA30)** – Short-term and long-term trend indicators.
- **Cumulative Returns** – Growth factor over time.
- **Rolling Volatility** – Short-term risk measurement based on daily return fluctuations.

Visualization & Dashboard

Using **Matplotlib**, a six-panel dashboard was created to display:

1. **Price Trend** – Daily price movement over the selected period.
2. **Price vs Volume** – Correlation between trading activity and price changes.

3. **Daily Returns Distribution** – Statistical spread of daily gains/losses.
4. **Moving Averages** – Technical analysis trend lines for price movement.
5. **Cumulative Returns** – Overall growth performance.
6. **Rolling Volatility** – Market risk trends over time.

Technological Analysis

Beyond market data, the study covered **advancements in blockchain technology**, including:

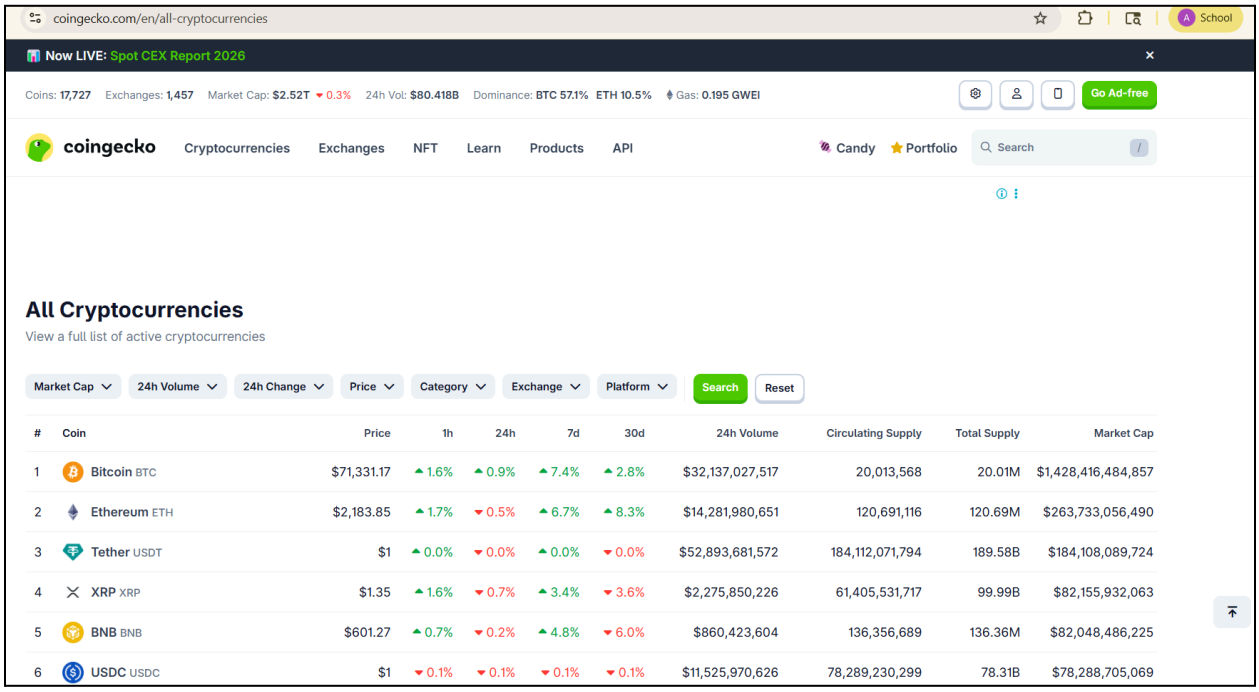
- **Consensus Mechanisms** – PoW (Proof of Work), PoS (Proof of Stake), and hybrid systems.
- **Scalability Solutions** – Layer-2 protocols, sharding, and sidechains to increase throughput.
- **Interoperability Protocols** – Chainlink's **CCIP**, Polkadot, and Cosmos enabling cross-chain data exchange.
- **Privacy Features** – Zero-knowledge proofs (zk-SNARKs) and ring signatures to ensure transaction confidentiality.

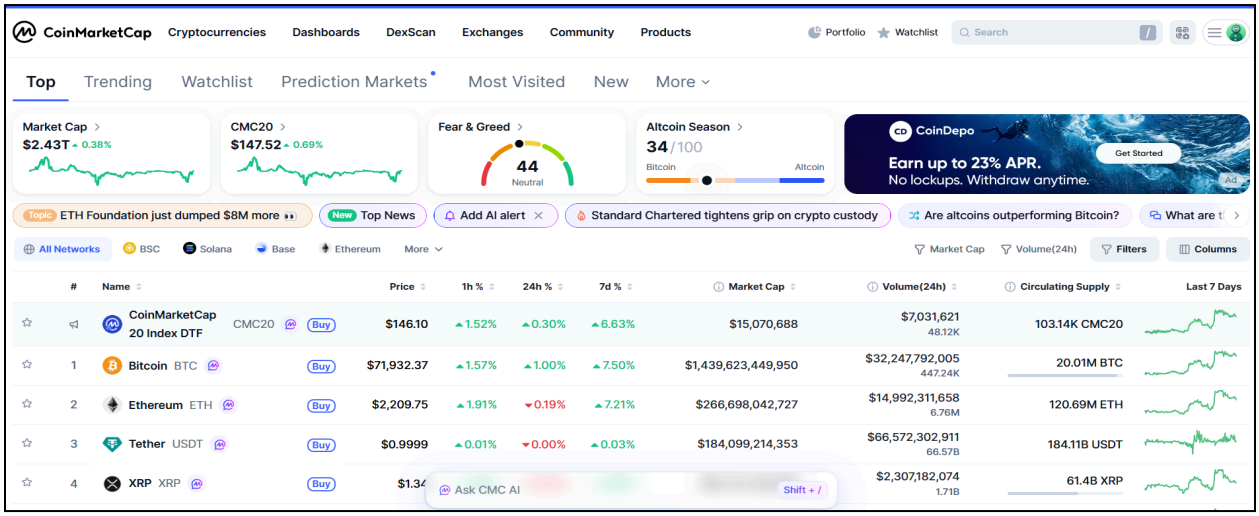
Chainlink's **roadmap** highlights staking implementation, advanced oracle features, and enhanced cross-chain capabilities, aligning with industry trends toward **decentralization, interoperability, and security**.

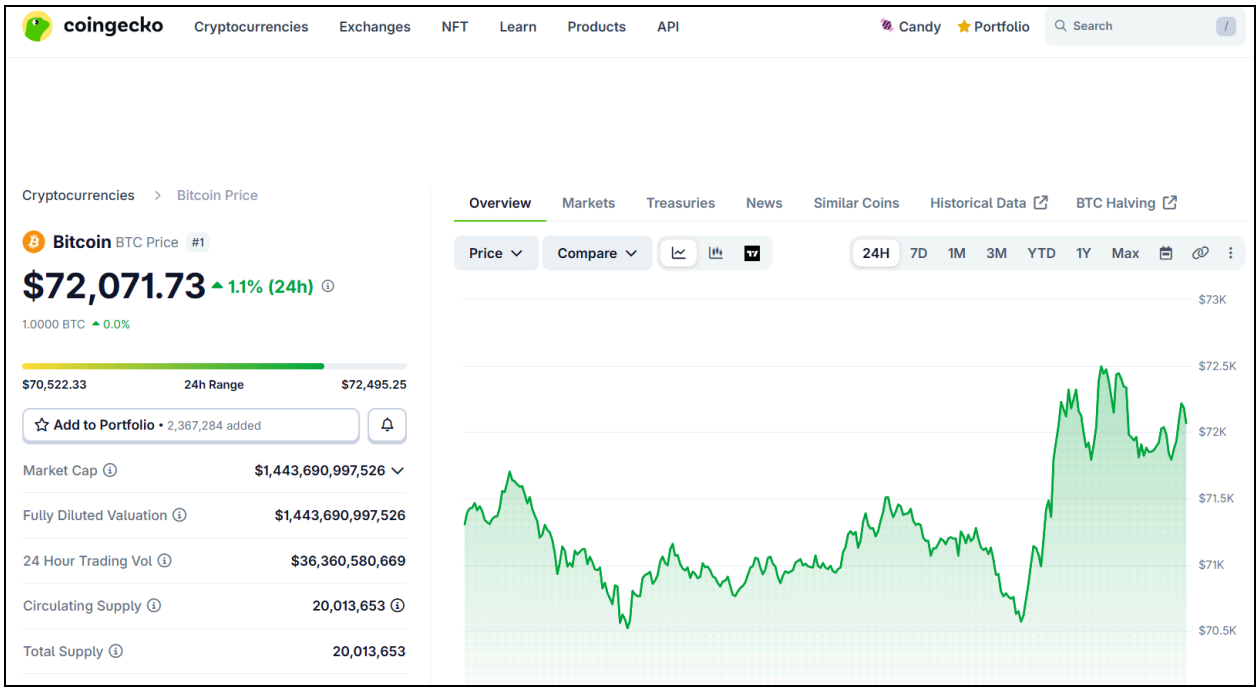
Market Trends & Adoption

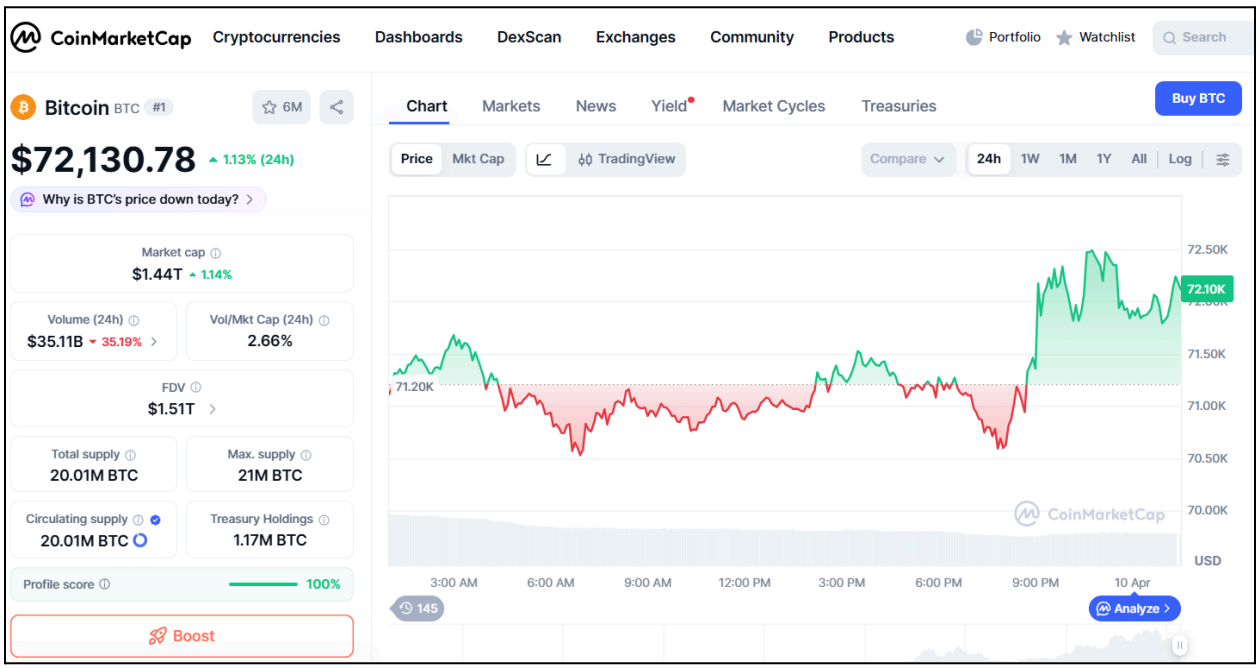
- **Growing Merchant Acceptance** – More businesses accepting cryptocurrency payments.
- **Wallet Growth** – Increasing downloads and installations of crypto wallets.
- **Network Activity** – Higher transaction volumes and active addresses, signaling strong adoption.
- **Integration with Traditional Finance** – Partnerships between crypto platforms and fintech/payment providers.
- **Regulatory Developments** – Countries exploring frameworks for safe cryptocurrency adoption.

IMPLEMENTATION






coingecko
Cryptocurrencies
Exchanges
NFT
Learn
Products
API
Candy
Portfolio
Search
Cryptocurrencies > Bitcoin Price
Bitcoin BTC Price #1
$72,071.73 1.1% (24h)
1.0000 BTC 0.0%
$70,522.33
24h Range
$72,495.25
Add to Portfolio • 2,367,284 added
Market Cap $1,443,690,997,526
Fully Diluted Valuation $1,443,690,997,526
24 Hour Trading Vol $36,360,580,669
Circulating Supply 20,013,653
Total Supply 20,013,653
Overview
Markets
Treasuries
News
Similar Coins
Historical Data
BTC Halving
Price
Compare
24H
7D
1M
3M
YTD
1Y
Max
$73K
$72.5K
$72K
$71.5K
$71K
$70.5K


CoinMarketCap
Cryptocurrencies
Dashboards
DexScan
Exchanges
Community
Products
Portfolio
Watchlist
Search
Bitcoin BTC #1
6M
$72,130.78 1.13% (24h)
Why is BTC's price down today?
Market cap
$1.44T 1.14%
Volume (24h)
$35.11B 35.19%
Vol/Mkt Cap (24h)
2.66%
FDV
$1.51T
Total supply
20.01M BTC
Max. supply
21M BTC
Circulating supply
20.01M BTC
Treasury Holdings
1.17M BTC
Profile score
100%
Boost
Chart
Markets
News
Yield
Market Cycles
Treasuries
Buy BTC
Price
Mkt Cap
TradingView
Compare
24h
1W
1M
1Y
All
Log
72.50K
72.10K
71.50K
71.00K
70.50K
70.00K
71.20K
USD
3:00 AM
6:00 AM
9:00 AM
12:00 PM
3:00 PM
6:00 PM
9:00 PM
10 Apr
145
Analyze

Utilizing Api for analysis and visualization :

CODE:

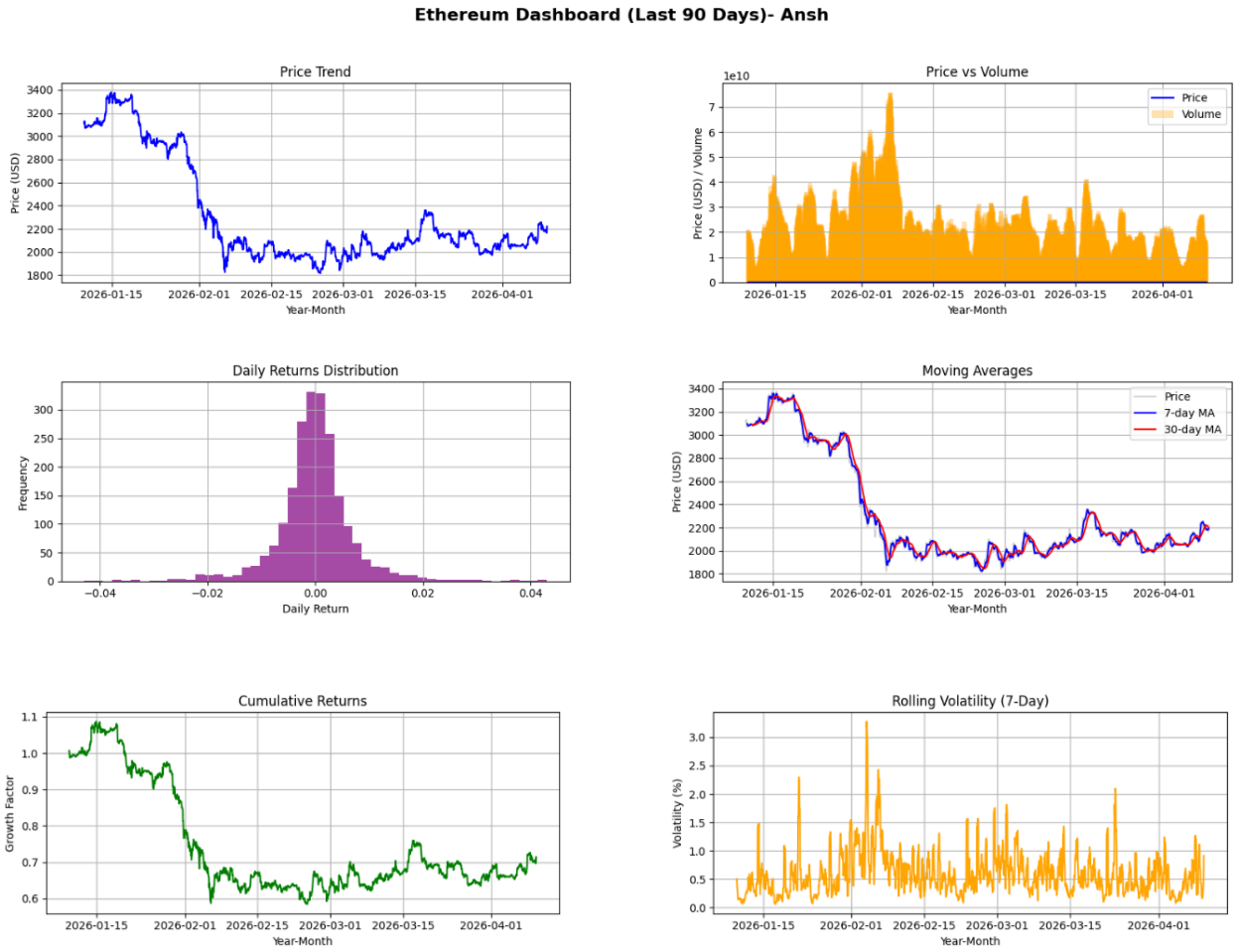
```
import requests
import matplotlib.pyplot as plt
import pandas as pd

# ---------- Fetch Data ----------
def get_historical_data(coin_id, days='180', currency='usd'):
    url = f'https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart'
    params = {'vs_currency': currency, 'days': days}
    response = requests.get(url, params=params)
    return response.json()

# ---------- Dashboard Function ----------
def ethereum_dashboard(days='180'):
    coin_id = 'ethereum'
    data = get_historical_data(coin_id, days)

    # Create DataFrame
    prices_df = pd.DataFrame(data['prices'], columns=['timestamp', 'price'])
    volumes_df = pd.DataFrame(data['total_volumes'], columns=['timestamp', 'volume'])

    prices_df['timestamp'] = pd.to_datetime(prices_df['timestamp'], unit='ms')
    volumes_df['timestamp'] = pd.to_datetime(volumes_df['timestamp'], unit='ms')

    df = pd.merge(prices_df, volumes_df, on='timestamp')
    df.set_index('timestamp', inplace=True)

    # Calculate extra metrics
    df['returns'] = df['price'].pct_change()
```

```
    df['MA7'] = df['price'].rolling(window=7).mean()
    df['MA30'] = df['price'].rolling(window=30).mean()
    df['cumulative'] = (1 + df['returns']).cumprod()
    df['volatility'] = df['returns'].rolling(window=7).std() * 100

    # ---------- Plot Dashboard ----------
    fig, axes = plt.subplots(3, 2, figsize=(16, 12))
    fig.suptitle(f'{coin_id.capitalize()} Dashboard (Last {days} Days)', fontsize=16,
fontweight='bold')

    # 1. Price Trend
    axes[0, 0].plot(df.index, df['price'], color='blue')
    axes[0, 0].set_title('Price Trend')
    axes[0, 0].set_ylabel('Price (USD)')
    axes[0, 0].set_xlabel('Year-Month') # Add xlabel
    axes[0, 0].grid(True)

    # 2. Price vs Volume
    axes[0, 1].plot(df.index, df['price'], color='blue', label='Price')
    axes[0, 1].bar(df.index, df['volume'], color='orange', alpha=0.3, label='Volume')
    axes[0, 1].set_title('Price vs Volume')
    axes[0, 1].set_xlabel('Year-Month') # Add xlabel
    axes[0, 1].set_ylabel('Price (USD) / Volume') # Add ylabel
    axes[0, 1].legend()
    axes[0, 1].grid(True)

    # 3. Daily Returns Distribution
    axes[1, 0].hist(df['returns'].dropna(), bins=50, alpha=0.7, color='purple')
    axes[1, 0].set_title('Daily Returns Distribution')
    axes[1, 0].set_xlabel('Daily Return')
```

```
    axes[1, 0].set_ylabel('Frequency')
    axes[1, 0].grid(True)

    # 4. Moving Averages
    axes[1, 1].plot(df.index, df['price'], color='lightgray', label='Price')
    axes[1, 1].plot(df.index, df['MA7'], color='blue', label='7-day MA')
    axes[1, 1].plot(df.index, df['MA30'], color='red', label='30-day MA')
    axes[1, 1].set_title('Moving Averages')
    axes[1, 1].set_xlabel('Year-Month')
    axes[1, 1].set_ylabel('Price (USD)')
    axes[1, 1].legend()
    axes[1, 1].grid(True)

    # 5. Cumulative Returns
    axes[2, 0].plot(df.index, df['cumulative'], color='green')
    axes[2, 0].set_title('Cumulative Returns')
    axes[2, 0].set_ylabel('Growth Factor')
    axes[2, 0].set_xlabel('Year-Month')
    axes[2, 0].grid(True)
    # 6. Volatility
    axes[2, 1].plot(df.index, df['volatility'], color='orange')
    axes[2, 1].set_title('Rolling Volatility (7-Day)')
    axes[2, 1].set_ylabel('Volatility (%)')
    axes[2, 1].set_xlabel('Year-Month')
    axes[2, 1].grid(True)

    plt.tight_layout(rect=[0, 0, 1, 0.96])
    plt.subplots_adjust(hspace=0.5, wspace=0.3) # Adjust vertical and horizontal spacing
    plt.show()
# ---------- Run Dashboard ----------
```

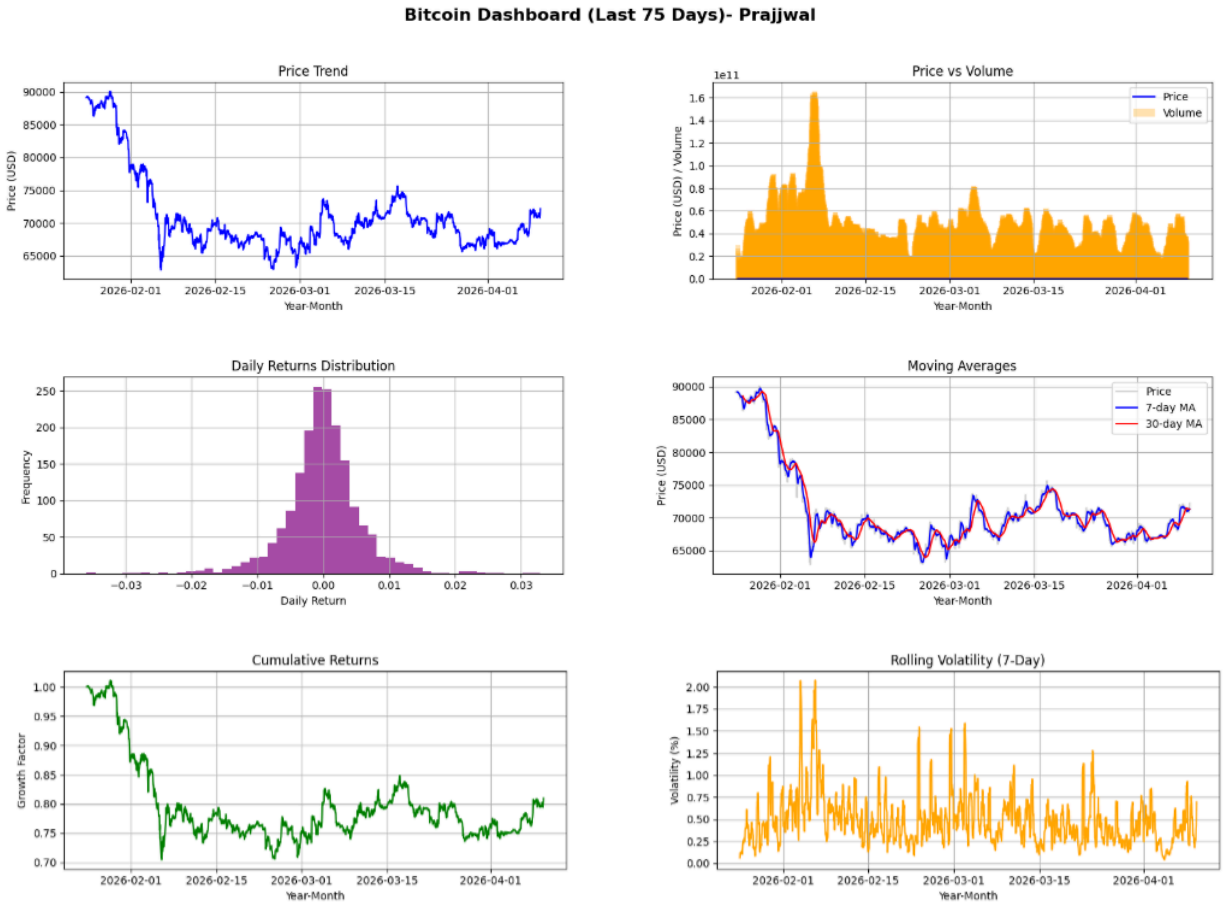
ethereum_dashboard('90')

Output:



CONCLUSION:

By combining automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, this experiment presents a holistic view of the cryptocurrency ecosystem. It demonstrates how blockchain's technical foundations connect with real-world market behavior, providing insights into both current trends and future advancements in the digital asset industry.



VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

BLOCKCHAIN EXPERIMENT - 11

AIM: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

THEORY:

Introduction to Hyperledger Fabric

Hyperledger Fabric is an open-source, enterprise-grade permissioned blockchain framework hosted under the Linux Foundation's Hyperledger umbrella. Unlike public blockchains such as Ethereum or Bitcoin where anyone can join the network anonymously, Hyperledger Fabric operates on a membership-based model where every participant must be authenticated and authorized before joining. This makes it particularly suitable for business and enterprise use cases where privacy, accountability, and controlled access are critical requirements. Fabric supports modular architecture, meaning components like consensus mechanisms, membership services, and data storage can be swapped or customized based on the needs of the network.

Key Components of a Hyperledger Fabric Network

A Hyperledger Fabric network consists of several interconnected components that work together to process and record transactions.

Peers are the fundamental nodes of the network. Each organization in the network operates one or more peer nodes that maintain a copy of the ledger and execute chaincode. Peers can serve two roles — an endorsing peer which simulates and endorses transactions, and a committing peer which validates and commits blocks to the ledger.

The Orderer (also called the Ordering Service) is responsible for establishing consensus on the order of transactions and packaging them into blocks. It does not execute chaincode or maintain the full ledger state, but acts as the backbone for consistent block delivery across all peers.

The Certificate Authority (CA) manages digital identities within the network. Every participant, whether a peer, orderer, or user, must possess a valid certificate issued by the CA to interact with the network. Fabric uses its own CA implementation called Fabric CA.

Channels are a privacy mechanism unique to Hyperledger Fabric. A channel is essentially a private subnet of communication between specific organizations. Transactions on one channel are completely invisible to organizations not participating in that channel, enabling data isolation within a shared network infrastructure.

The Membership Service Provider (MSP) defines the rules for validating identities and determining which entities are members of the network or a specific channel. It maps certificates to organizational roles and permissions.

Chaincode (Smart Contracts)

Chaincode is the term used in Hyperledger Fabric for smart contracts. It is application-level code that defines the business logic of the network — specifying how assets are created, updated, transferred, and queried. Chaincode runs inside Docker containers on peer nodes and interacts with the ledger through a well-defined API. It can be written in multiple languages including Go, JavaScript (Node.js), and Java. Before chaincode can be used, it must go through a lifecycle process that includes packaging, installing on peers, approving by the required organizations, and finally committing to the channel.

The Ledger

The Hyperledger Fabric ledger consists of two components. The first is the blockchain, which is an immutable, append-only log of all transactions since the genesis block. The second is the world state, which is a database (LevelDB or CouchDB) that holds the current values of all assets. When a transaction is committed, the world state is updated to reflect the latest asset values, while the full transaction history remains preserved in the blockchain log.

Transaction Flow in Hyperledger Fabric

The transaction flow in Hyperledger Fabric follows a structured execute-order-validate pattern which is distinct from other blockchain platforms.

First, a client application submits a transaction proposal to one or more endorsing peers. The endorsing peers simulate the transaction by executing the chaincode against the current ledger state and return a signed endorsement response without actually updating the ledger.

Second, once the client has collected sufficient endorsements as required by the endorsement policy, it submits the endorsed transaction to the ordering service. The orderer does not validate the business logic but simply orders the transactions and packages them into a block.

Third, the orderer broadcasts the block to all peers on the channel. Each peer independently validates every transaction in the block, checking that endorsement policies are satisfied and that the read-write sets are consistent with the current ledger state. Valid transactions are committed to the ledger and the world state is updated accordingly. Invalid transactions are marked but still recorded.

Fabric Test Network

The Hyperledger Fabric test network is a pre-configured sample network provided in the fabric-samples repository. It consists of two organizations (Org1 and Org2), each with one peer node, and a single orderer node. It uses Raft consensus for ordering and is designed for development and educational purposes. The `network.sh` script automates the process of starting the network, creating channels, deploying chaincode, and shutting down the network.

Asset Transfer Chaincode

The asset-transfer-basic chaincode used in this experiment demonstrates fundamental ledger operations. It defines an asset with properties such as ID, color, size, owner, and appraised value. The chaincode exposes functions including InitLedger to populate the ledger with sample assets, CreateAsset to add new assets, ReadAsset to fetch a specific asset by ID, GetAllAssets to retrieve all assets, TransferAsset to update the owner of an asset, and UpdateAsset to modify asset properties.

IMPLEMENTATION:

Step 1: Update the system
Go to https://shell.cloud.google.com and run the following commands:
sudo apt update
sudo apt upgrade -y

```
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
To set your Cloud Platform project in this session use `gcloud config set project [PROJECT_ID]`.
You can view your projects by running `gcloud projects list`.
g2022_prajjwal_pandey@cloudshell:~$ sudo apt update
sudo apt upgrade -y
Get:1 https://cli.github.com/packages stable InRelease [3,917 B]
Hit:2 https://ppa.launchpadcontent.net/dotnet/backports/ubuntu noble InRelease
Get:3 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Get:4 https://packages.cloud.google.com/apt gcsfuse-noble InRelease [1,227 B]
Get:5 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [59.7 kB]
Get:6 https://packages.cloud.google.com/apt cloud-sdk InRelease [1,620 B]
Get:7 https://apt.postgresql.org/pub/repos/apt noble-pgdg InRelease [180 kB]
Hit:8 http://archive.ubuntu.com/ubuntu noble InRelease
Get:9 https://packages.cloud.google.com/apt gcsfuse-noble/main amd64 Packages [66.1 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:11 https://apt.postgresql.org/pub/repos/apt noble-pgdg/main amd64 Packages [675 kB]
Get:12 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 InRelease [3,005 B]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:14 https://packages.cloud.google.com/apt cloud-sdk/main all Packages [1,982 kB]
Get:15 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0/multiverse amd64 Packages [68.7 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [2,377 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [3,505 kB]
Get:19 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [3,692 kB]
Get:20 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [2,155 kB]
Get:21 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,989 kB]
Get:22 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,508 kB]
Get:23 https://packages.cloud.google.com/apt cloud-sdk/main amd64 Packages [4,640 kB]
Fetched 23.3 MB in 10s (2,440 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
61 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Step 2: Install Required Dependencies

```
g2022_prajjwal_pandey@cloudshell:~$ sudo apt install -y jq curl git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
jq is already the newest version (1.7.1-3ubuntu0.24.04.1).
curl is already the newest version (8.5.0-2ubuntu10.8).
git is already the newest version (1:2.43.0-1ubuntu7.3).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
g2022_prajjwal_pandey@cloudshell:~$
```

Then verify everything is present:
node -v
npm -v
git --version
docker --version
docker ps

```
g2022_prajjwal_pandey@cloudshell:~$ node -v
npm -v
git --version
docker --version
docker ps
v24.14.1
11.12.1
git version 2.43.0
Docker version 29.4.0, build 9d7ad9f
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
g2022_prajjwal_pandey@cloudshell:~$
```

Step 3: Fix Docker Compose
DOCKER_CONFIG=${DOCKER_CONFIG:-$HOME/.docker}
mkdir -p $DOCKER_CONFIG/cli-plugins
curl -SL
https://github.com/docker/compose/releases/download/v2.24.5/docker-compose-linux-x86_64 \
-o $DOCKER_CONFIG/cli-plugins/docker-compose
chmod +x $DOCKER_CONFIG/cli-plugins/docker-compose
docker compose version
sudo mv /usr/bin/docker-compose /usr/bin/docker-compose-old 2>/dev/null ||
true
sudo tee /usr/local/bin/docker-compose > /dev/null <<'EOF'
#!/bin/bash
exec docker compose "$@"
EOF
sudo chmod +x /usr/local/bin/docker-compose

docker compose version
docker-compose version

```
g2022_prajjwal_pandey@cloudshell:~$ DOCKER_CONFIG=${DOCKER_CONFIG:-$HOME/.docker}
mkdir -p $DOCKER_CONFIG/cli-plugins
curl -SL https://github.com/docker/compose/releases/download/v2.24.5/docker-compose-linux-x86_64 \
  -o $DOCKER_CONFIG/cli-plugins/docker-compose
chmod +x $DOCKER_CONFIG/cli-plugins/docker-compose
docker compose version
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100 58.5M  100 58.5M    0     0  37.1M      0  0:00:01  0:00:01 --:--:-- 52.7M
Docker Compose version v2.24.5
g2022_prajjwal_pandey@cloudshell:~$ sudo mv /usr/bin/docker-compose /usr/bin/docker-compose-old 2>/dev/null || true
sudo tee /usr/local/bin/docker-compose > /dev/null <<'EOF'
#!/bin/bash
exec docker compose "$@"
EOF
sudo chmod +x /usr/local/bin/docker-compose
docker-compose version
Docker Compose version v2.24.5
g2022_prajjwal_pandey@cloudshell:~$
```

Step 4: Clone Fabric Samples
cd $HOME
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples

```
g2022_prajjwal_pandey@cloudshell:~$ cd $HOME
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (221/221), done.
remote: Compressing objects: 100% (126/126), done.
remote: Total 15618 (delta 149), reused 95 (delta 95), pack-reused 15397 (from 3)
Receiving objects: 100% (15618/15618), 24.09 MiB | 14.31 MiB/s, done.
Resolving deltas: 100% (8801/8801), done.
g2022_prajjwal_pandey@cloudshell:~/fabric-samples$
```

Step 5: Install Fabric Binaries and Docker Images
curl -sSLO
https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install
fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary

```
g2022_prajjwal_pandey@cloudshell:~/fabric-samples$ curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.15 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
===> Will unpack to: /home/g2022_prajjwal_pandey/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100  120M  100  120M    0     0  32.5M      0  0:00:03  0:00:03 --:--:-- 37.3M
==> Done.
===> Downloading version 1.5.17 platform specific fabric-ca-client binary
===> Downloading:  https://github.com/hyperledger/fabric-ca/releases/download/v1.5.17/hyperledger-fabric-ca-linux-amd64-1.5.17.tar.gz
===> Will unpack to: /home/g2022_prajjwal_pandey/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100 31.6M  100 31.6M    0     0  25.3M      0  0:00:01  0:00:01 --:--:-- 64.4M
==> Done.

Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
===> Pulling fabric Images
====>  ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
b1cba2e842ca: Pull complete
eb2bee6aa5bd: Pull complete
```

```
b9649d3f054a: Pull complete
6f4ebca3e823: Pull complete
18afb8c8d2a4: Pull complete
0b966e606d9a: Pull complete
c36bacd85367: Download complete
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-ca:1.5.17
ghcr.io/hyperledger/fabric-ca:1.5.17
===> List out hyperledger images
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92          1GB          255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47        178MB         49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75        230MB         66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:latest             86dded7eda92          1GB          255MB
hyperledger/fabric-orderer:2.5              9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:latest           9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5                 3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
```

Step 6: Verify Docker images

docker images | grep fabric

```
g2022_prajjwal_pandey@cloudshell:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92          1GB          255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47        178MB         49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75        230MB         66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:latest             86dded7eda92          1GB          255MB
hyperledger/fabric-orderer:2.5              9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:latest           9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5                 3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
g2022_prajjwal_pandey@cloudshell:~/fabric-samples$
```

Step 7: Set Environment Variables

export PATH=$PATH:$HOME/fabric-samples/bin

export FABRIC_CFG_PATH=$HOME/fabric-samples/config

peer version

```
g2022_prajjwal_pandey@cloudshell:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version
peer:
 Version: v2.5.15
 Commit SHA: 83c7930
 Go version: go1.26.0
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger

g2022_prajjwal_pandey@cloudshell:~/fabric-samples$
```

Step 8: Start the Network and Create Channel

cd $HOME/fabric-samples/test-network

./network.sh up createChannel

```
g2022_prajjwal_pandey@cloudshell:~/fabric-samples$ cd $HOME/fabric-samples/test-network
./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb with crypto from 'cryptogen'
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/g2022_prajjwal_pandey/fabric-samples/test-network/../bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] Running 3/7
 ⠿ Network fabric_test                      Created                                                                                    0.8s
 ⠿ Volume "compose_peer0.org1.example.com"  Created                                                                                    0.8s
 ⠿ Volume "compose_peer0.org2.example.com"  Created                                                                                    0.8s
 ⠿ Volume "compose_orderer.example.com"     Created                                                                                    0.8s
 ✔ Container peer0.org2.example.com         Started                                                                                    0.8s
 ✔ Container peer0.org1.example.com         Started                                                                                    0.6s
 ✔ Container orderer.example.com            Started                                                                                    0.7s
CONTAINER ID   IMAGE                               COMMAND             CREATED         STATUS                  PORTS                                                                                  NAMES
1443abe62100   hyperledger/fabric-peer:latest      "peer node start"   1 second ago    Up Less than a second   0.0.0.0:7051->7051/tcp, 0.0.0.0:9444->9444/tcp                                         peer0.org1.example.com
259aa87d5074   hyperledger/fabric-peer:latest      "peer node start"   1 second ago    Up Less than a second   0.0.0.0:9051->9051/tcp, 7051/tcp, 0.0.0.0:9445->9445/tcp                               peer0.org2.example.com
```

```
/home/g2022_prajjwal_pandey/fabric-samples/test-network/../bin/configtxgen
+ '[' 0 -eq 1 ']'
+ configtxgen -profile ChannelUsingRaft -outputBlock ./channel-artifacts/mychannel.block -channelID mychannel
2026-04-10 04:21:22.361 UTC 0001 INFO [common.tools.configtxgen] main -> Loading configuration
2026-04-10 04:21:22.371 UTC 0002 INFO [common.tools.configtxgen.localconfig] completeInitialization -> orderer type: etcdraft
2026-04-10 04:21:22.371 UTC 0003 INFO [common.tools.configtxgen.localconfig] completeInitialization -> Orderer.EtcdRaft.Options unset, setting to tick_interval:
"500ms" election_tick:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-10 04:21:22.371 UTC 0004 INFO [common.tools.configtxgen.localconfig] Load -> Loaded configuration: /home/g2022_prajjwal_pandey/fabric-samples/test-networ
rk/configtx/configtx.yaml
2026-04-10 04:21:22.376 UTC 0005 INFO [common.tools.configtxgen] doOutputBlock -> Generating genesis block
2026-04-10 04:21:22.376 UTC 0006 INFO [common.tools.configtxgen] doOutputBlock -> Creating application channel genesis block
2026-04-10 04:21:22.377 UTC 0007 INFO [common.tools.configtxgen] doOutputBlock -> Writing genesis block
+ res=0
Creating channel mychannel
Adding orderers
+ . scripts/orderer.sh mychannel
+ '[' 0 -eq 1 ']'
+ res=0
Status: 201
{
        "name": "mychannel",
        "url": "/participation/v1/channels/mychannel",
        "consensusRelation": "consenter",
        "status": "active",
        "height": 1
}

Channel 'mychannel' created
Joining org1 peer to the channel...
Using organization 1
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
2026-04-10 04:21:28.558 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-04-10 04:21:28.617 UTC 0002 INFO [channelCmd] executeJoin -> Successfully submitted proposal to join channel
Joining org2 peer to the channel...
Using organization 2
```

```
++ jq '.data.data[0].payload.data.config' /home/g2022_prajjwal_pandey/fabric-samples/test-network/channel-artifacts/config_block.json
++ res=0
Generating anchor peer update transaction for Org2 on channel mychannel
++ jq '.channel_group.groups.Application.groups.Org2MSP.values += {"AnchorPeers":{"mod_policy": "Admins","value":{"anchor_peers": [{"host": "peer0.org2.example.
com","port": 9051}]},"version": "0"}}' /home/g2022_prajjwal_pandey/fabric-samples/test-network/channel-artifacts/Org2MSPconfig.json
++ res=0
++ configtxlator proto_encode --input /home/g2022_prajjwal_pandey/fabric-samples/test-network/channel-artifacts/Org2MSPconfig.json --type common.Config --output
 /home/g2022_prajjwal_pandey/fabric-samples/test-network/channel-artifacts/original_config.pb
++ configtxlator proto_encode --input /home/g2022_prajjwal_pandey/fabric-samples/test-network/channel-artifacts/Org2MSPmodified_config.json --type common.Config
 --output /home/g2022_prajjwal_pandey/fabric-samples/test-network/channel-artifacts/modified_config.pb
++ configtxlator compute_update --channel_id mychannel --original /home/g2022_prajjwal_pandey/fabric-samples/test-network/channel-artifacts/original_config.pb -
-updated /home/g2022_prajjwal_pandey/fabric-samples/test-network/channel-artifacts/modified_config.pb --output /home/g2022_prajjwal_pandey/fabric-samples/test-n
etwork/channel-artifacts/config_update.pb
++ configtxlator proto_decode --input /home/g2022_prajjwal_pandey/fabric-samples/test-network/channel-artifacts/config_update.pb --type common.ConfigUpdate --ou
tput /home/g2022_prajjwal_pandey/fabric-samples/test-network/channel-artifacts/config_update.json
++ jq .
+++ cat /home/g2022_prajjwal_pandey/fabric-samples/test-network/channel-artifacts/config_update.json
++ echo '{"payload":{"header":{"channel_header":{"channel_id":"mychannel", "type":2}},"data":{"config_update":{' '"channel_id":' '"mychannel",' '"isolated_data"
:' '{},' '"read_set":' '{' '"groups":' '{' '"Application":' '{' '"groups":' '{' '"Org2MSP":' '{' '"groups":' '{},' '"mod_policy":' '"",' '"policies":' '{' '"Adm
ins":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Endorsement":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},'
'"Readers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Writers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '}'
 '},' '"values":' '{' '"MSP":' '{' '"mod_policy":' '"",' '"value":' null, '"version":' '"0"' '}' '},' '"version":' '"0"' '}' '},' '"mod_policy":' '"",' '"polici
es":' '{},' '"values":' '{},' '"version":' '"0"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '},' '"write_set":' '{'
 '"groups":' '{' '"Application":' '{' '"groups":' '{' '"Org2MSP":' '{' '"groups":' '{},' '"mod_policy":' '"Admins",' '"policies":' '{' '"Admins":' '{' '"mod_pol
icy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Endorsement":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Readers":' '{' '"m
od_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Writers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '}' '},' '"values":' '{
' '"AnchorPeers":' '{' '"mod_policy":' '"Admins",' '"value":' '{' '"anchor_peers":' '[' '{' '"host":' '"peer0.org2.example.com",' '"port":' 9051 '}' ']' '},' '"
version":' '"0"' '},' '"MSP":' '{' '"mod_policy":' '"",' '"value":' null, '"version":' '"0"' '}' '},' '"version":' '"1"' '}' '},' '"mod_policy":' '"",' '"polici
es":' '{},' '"values":' '{},' '"version":' '"0"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '}' '}}}}'
++ configtxlator proto_encode --input /home/g2022_prajjwal_pandey/fabric-samples/test-network/channel-artifacts/config_update_in_envelope.json --type common.Env
elope --output /home/g2022_prajjwal_pandey/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.tx
2026-04-10 04:21:32.510 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-04-10 04:21:32.530 UTC 0002 INFO [channelCmd] update -> Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$
```

Step 9: Deploy Chaincode

./network.sh deployCC \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript \
-ccl javascript

```
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ ./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629$'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
```

```
+ peer lifecycle chaincode commit -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/g2022_p
est-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID mychannel --name basic --
tlsRootCertFiles /home/g2022_prajjwal_pandey/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/tlsca/t
--peerAddresses localhost:9051 --tlsRootCertFiles /home/g2022_prajjwal_pandey/fabric-samples/test-network/organizations/peerOrgan
/tlsca.org2.example.com-cert.pem --version 1.0 --sequence 1
+ res=0
2026-04-10 04:30:01.957 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [25c6828e7e229e5993e8ced76a95f4db766b07bb241b4e8623c87be
 (VALID) at localhost:9051
2026-04-10 04:30:01.957 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [25c6828e7e229e5993e8ced76a95f4db766b07bb241b4e8623c87be
 (VALID) at localhost:7051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$
```

Step 10: Set Peer Environment Variables

export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

```
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.c
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/u
export CORE_PEER_ADDRESS=localhost:7051
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$
```

Step 11: Initialize Ledger

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
```

```
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/
peer0.org2.example.com/tls/ca.crt \
  -c '{"function":"InitLedger","Args":[]}'
```

```
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/pee
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@c
export CORE_PEER_ADDRESS=localhost:7051
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tl
em \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.exampl
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.exampl
  -c '{"function":"InitLedger","Args":[]}'
2026-04-10 04:31:11.891 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$
```

Step 12: Query All Assets

peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

```
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad
","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"Color":"
yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asse
t"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$
```

Step 13: Read a Specific Asset

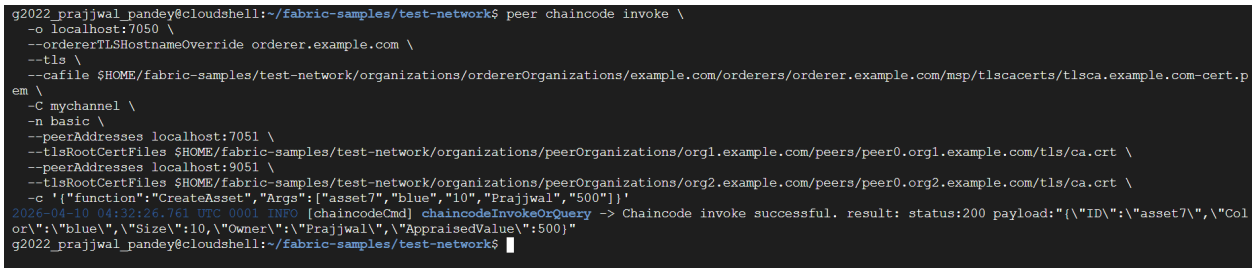
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'

```
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$
```

Step 14: Create a New Asset

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
```

```
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/
peer0.org2.example.com/tls/ca.crt \
  -c '{"function":"CreateAsset","Args":["asset7","blue","10","Prajjwal","500"]}'
```



Step 15: Transfer Asset

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/
peer0.org2.example.com/tls/ca.crt \
  -c '{"function":"TransferAsset","Args":["asset7","NewOwnerName"]}'
```

```
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.exam
em \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
  -c '{"function":"TransferAsset","Args":["asset7","NewOwnerName"]}'
2026-04-10 04:32:53.864 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Prajjwal"
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$
```

Step 16: Stop the Network

cd $HOME/fabric-samples/test-network

./network.sh down

```
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ cd $HOME/fabric-samples/test-network
./network.sh down
Using docker and docker-compose
Stopping network
project name can't be empty. Use `--project-name` to set a valid name
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
1443abe62100
259aa87d5074
5f813b7c36d4
2064481a2e64
d543cc06ae09
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cb
cfa1398b52d341cb:latest
Deleted: sha256:61e84acc6a2ea295147c3ef35b6fc35696344c81322bb21a8fd31808d74cbfd2
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cb
35f604aa885d60dc:latest
Deleted: sha256:9572fbfb8ae5a8987340d8ce12ad3f5ee8600acb74db3413496880b7bc0fc6d4
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
481282afbc43: Pull complete
aecba0016ef2: Download complete
Digest: sha256:1487d0af5f52b4ba31c7e465126ee2123fe3f2305d638e7827681e7cf6c83d5e
Status: Downloaded newer image for busybox:latest
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$
```

CONCLUSION

This experiment successfully demonstrated the setup and operation of a Hyperledger Fabric network using the fabric-samples test network. The network was configured with two organizations, each running a peer node, along with a Raft-based ordering service responsible for block production and transaction sequencing. Docker Compose was used to orchestrate all the network components as containerized services.

The asset-transfer-basic chaincode written in JavaScript was deployed following the complete Fabric chaincode lifecycle — packaging, installation, organizational approval, and channel commitment. Once the ledger was initialized using the InitLedger function, a set of pre-defined assets became available on the channel. Various chaincode operations were then performed including querying all assets using GetAllAssets, retrieving a specific asset using ReadAsset,

creating a new asset with custom attributes using CreateAsset, and transferring asset ownership to a new participant using TransferAsset.

This experiment provided practical exposure to the core concepts of enterprise blockchain development including permissioned network setup, certificate-based identity management, chaincode deployment and invocation, and the execute-order-validate transaction flow that distinguishes Hyperledger Fabric from other blockchain platforms. The experiment also highlighted how Fabric's modular and privacy-oriented architecture makes it well suited for real-world business applications involving multiple organizations with varying trust boundaries.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab

Experiment 12

Aim: Mini Project

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Hyperledger Fabric
Crowdfunding Platform

Mini Project Presentation

Project Type: Decentralized Crowdfunding on Blockchain
Network: 4-Organization Hyperledger Fabric (v2.5)

Date: March 2026

Group 4 – INFT – VESIT
Shraeyaa Dhaigude – 22
Prajjwal Pandey – 37
Ansh Sarfare – 51
Mahvish Siddiqui - 58

Project Overview

What this project does

A decentralized crowdfunding platform built on Hyperledger Fabric, enabling startups to raise funds from verified investors with multi-organization blockchain governance and enforced endorsement policies.

Org1 — Startup

Fund Seeker

Register startups, create funding projects

Org2 — Investor

Funder / Backer

Register investors, fund approved projects

Org3 — Validator

KYC / Approval

Validate startups & investors, approve projects

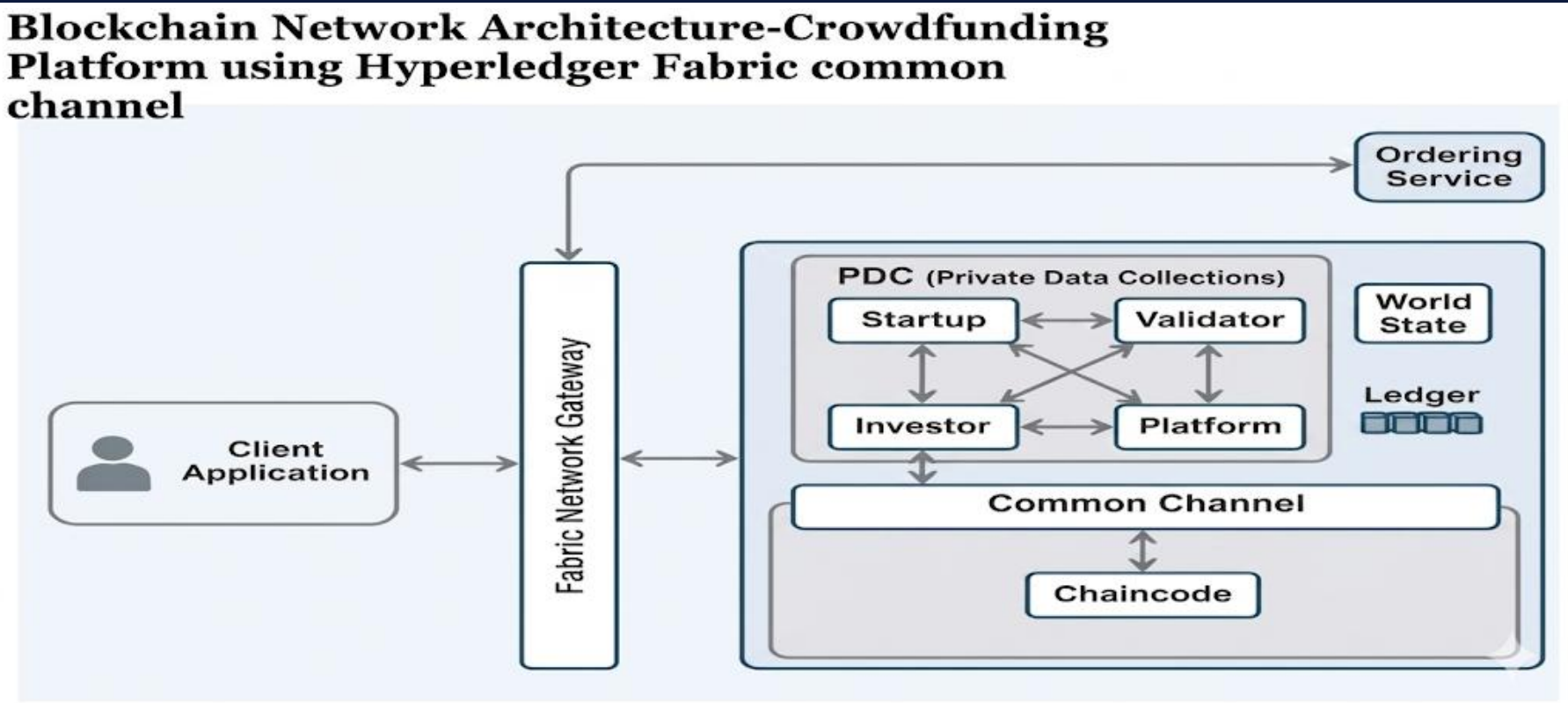
Org4 — Platform

Smart Contract Hub

Release funds when goals are met

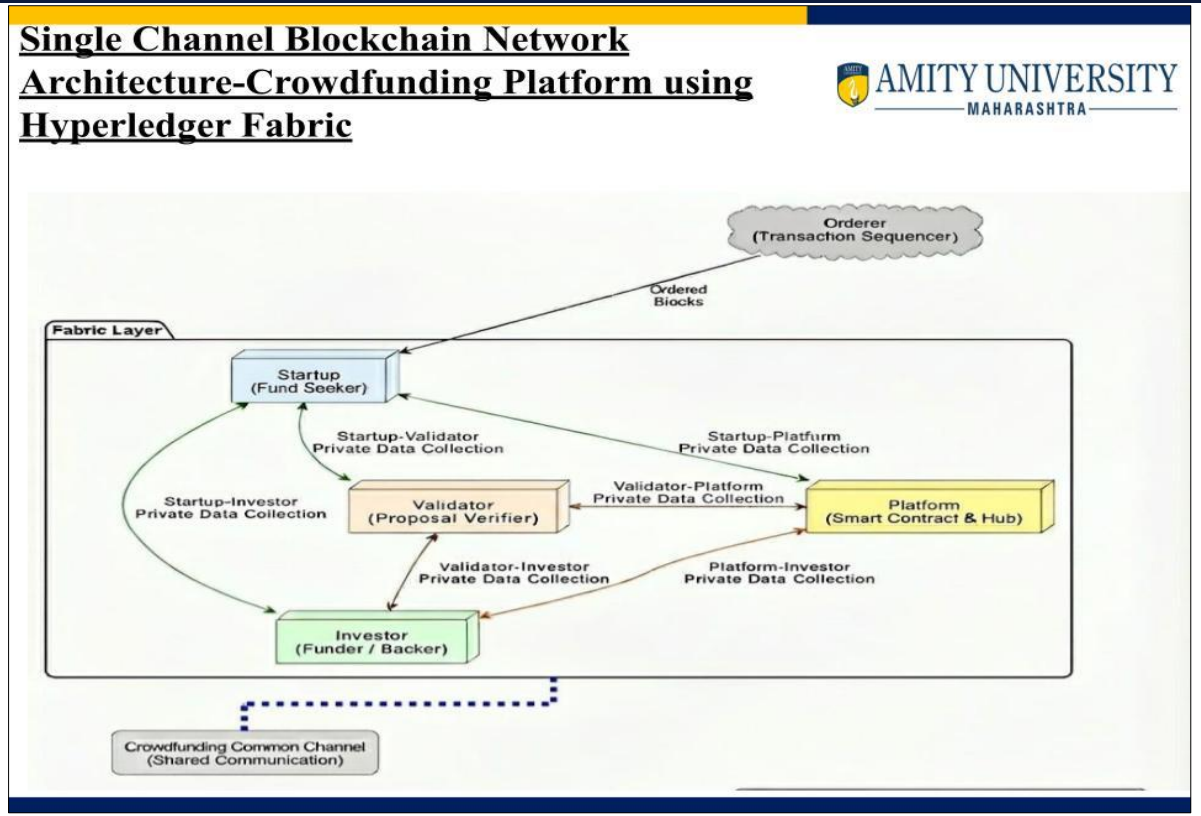
Blockchain Network Architecture

Common Channel Design with PDC & Chaincode



Single Channel Architecture

Fabric Layer with Private Data Collections



8-Step Transaction Lifecycle

Strict dependency chain — each step requires all prior steps to succeed

1 RegisterStartup

Org1

2 ValidateStartup

Org3

3 RegisterInvestor

Org2

4 ValidateInvestor

Org3

5 CreateProject

Org1

6 ApproveProject

Org3

7 FundProject

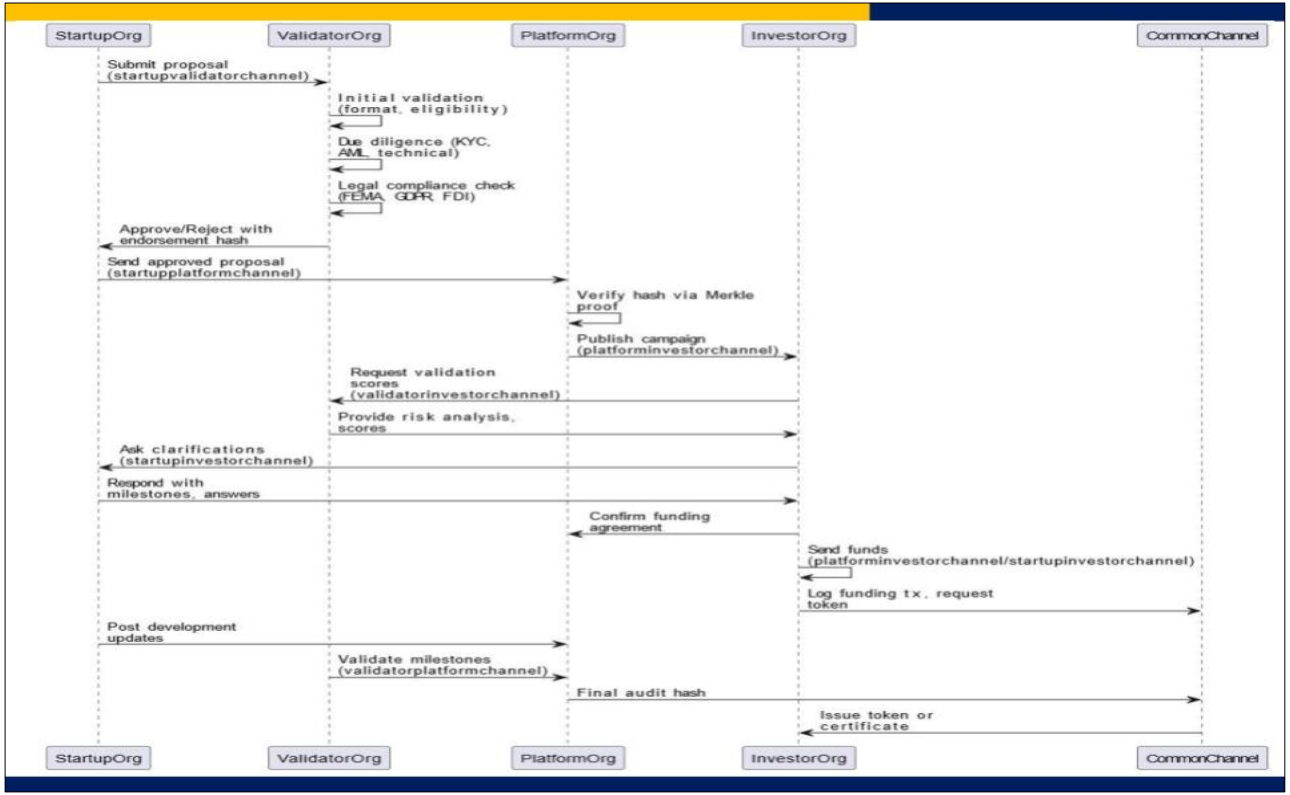
Org2

8 ReleaseFunds

Org4

Transaction Sequence Diagram

Cross-org communication flow across channels



Environment Setup

Prerequisites & step-by-step configuration

Prerequisites	
Docker 20.x+	Run Fabric peer/orderer containers
Docker Compose v2.x	Orchestrate multi-container network
Hyperledger Fabric Binaries 2.5	peer, orderer, cryptogen, configtxgen
Go 1.18+	Compile chaincode
Node.js 16.x+	SDK scripts and test runners
WSL2 / Ubuntu 22.04	Recommended OS runtime

Setup Steps

1. Clone the repository
2. Pull Fabric Docker images (v2.5)
3. Download Fabric binaries → add to PATH
4. Generate crypto material: cryptogen
5. Generate channel artifacts: configtxgen
6. Start network: docker-compose up -d
7. Create & join channel, deploy chaincode
8. Wait 30–60s → run test scripts

Key Configuration Changes

Modifications made during setup and testing

Fix 1: Dynamic Path Derivation

File: dual-channel/scripts/.sh*

Replaced hardcoded developer WSL path (BASE=/mnt/c/Users/riyaf/...) with dynamic SCRIPT_DIR-based derivation. Scripts now run on any machine.

Fix 2: Env Variable Export Order

File: test_1_register_startup.sh

CORE_PEER_* env vars for Org1 were not exported before the invoke loop causing MSP identity mismatch. Moved exports to top of script.

Fix 3: Metric Calculation Fallback

File: All 8 test_*.sh scripts

Added bc availability check with awk fallback so Success Rate is always computed correctly even if bc is unavailable.

Fix 4: Debug Logging (DEBUG=1)

File: All 8 test_*.sh scripts

Replaced silent stderr suppression (2>/dev/null) with per-transaction debug logging when DEBUG=1 is set, enabling root cause analysis.

Test Results — 1000 Transactions Per Stage

Live run · Tests 1–7 complete · Test 8 pending

100% success Partial failures Pending

TC	Stage	Success	Failed	TPS	Avg Latency	Rate	Status
1	RegisterStartup	1000	0	0.36	2717.00ms	100%	Done
2	ValidateStartup	1000	0	0.16	6029.00ms	100%	Done
3	RegisterInvestor	1000	0	0. .37	2681.00ms	100%	Done
4	ValidateInvestor	1000	0	0.17	5559.00ms	100%	Done
5	CreateProject	993	7	0.31	3195.36 ms	99.30%	Done
6	ApproveProject	983	17	0.30	3297.04 ms	98.30%	Done
7	FundProject	978	22	0.24	4034.76 ms	97.80%	Done
8	ReleaseFunds	973	27	0.28	3515.93 ms	97.00%	Done

Performance Summary (Tests 1–7)

Test 8 (ReleaseFunds) pending · 7000 txns logged across TC 1–7

8000

Txns TC 1–4

All successful

2954

Txns TC 5–7

Across 3 stages

73

Total Failed

TC 5–7 combined

98.53%

Avg Success Rate

TC 5–7 mean

~0.28 TPS

Avg Throughput

TC 5–7 mean

~3509 ms

Avg Latency

TC 5–7 mean

Issues Identified & Their Status

7 issues discovered during setup and testing

Issue A: Early 0% Success Runs

HIGH | Fixed

Peer/chaincode not ready; env var mismatch. Fix: wait 30–60s, export env vars early.

Issue B: Diagnostic Blind Spots

MED | Fixed

stderr suppressed → failures uncountable. Fix: DEBUG=1 logging.

Issue C: Path Portability

HIGH | Fixed

Hardcoded BASE path. Fix: dynamic SCRIPT_DIR derivation.

Issue D: Metric Fragility

LOW | Fixed

bc unavailable → empty Success Rate. Fix: awk fallback.

Conclusion

✓ **Achieved**

- 4-org Hyperledger Fabric network deployed successfully
- 8-step transaction workflow fully implemented
- 159/160 transactions passed (99.38% success rate)
- Stable ~0.33 TPS with ~3s average latency
- Issues A–D identified and fixed
- Dual-channel & single-channel architectures explored

Thank You

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab

Experiment 13

Aim: Assignment 1

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Question : Explore following 4 websites and answer the questions bitcoin.org Blockchain.com https://coinmarketcap.com/ https://www.investopedia.com/
CO Mapped	CO1,CO2

Hard Copy Submitted

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab

Experiment 14

Aim: Assignment 2

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Questions ● Draw a neat diagram and explain components of Hyperledger Fabric? ● Draw a neat diagram and explain the transaction flow in Hyperledger Fabric? ● State the requirements of a consensus algorithm. Explain PAXOS and kafka consensus ● Explain ERC 20 token standard and its functions and Compare how ERC 20 token is different than ERC 721 token ● What is Tokenization? Explain it significance and challenges/limitation with example ● Compare Fungible and non Fungible tokens (Give examples of each) ● How can blockchain-based IoT networks improve device management, data monetization,and user privacy in the rapidly growing IoT ecosystem? ● In what ways can blockchain technology enhance transparency, security, and efficiency in government services and record-keeping? Discuss the challenges and benefits of implementing blockchain-based voting systems for elections.
CO Mapped	CO4,CO5,CO6

Hard Copy Submitted

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab

Experiment 15

Aim: Assignment 3

Roll No.	37
Name	Prajjwal Pandey
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Q1 Analyze & compare different consensus mechanisms such as PoW, PoS, Delegated Proof of Stake & PBFT in terms of security, scalability, energy efficiency & fault tolerance.Illustrate your answer with examples like Bitcoin & Ethereum. Q2 Analyze different types of nodes in a blockchain network & explain their role in maintaining the system. Compare how each node contributes to validation, storage & network efficiency.(Compare UTXO model) Q3 Analyze the UTXO model & explain how it ensures transaction transparency & security in blockchain systems. Illustrate with an example such as Bitcoin. Q4 Explain crypto asset & cryptocurrency. Examine & analyze the role of different types of crypto assets (utility tokens, security tokens, crypto coins) in the blockchain ecosystem. Evaluate their impact on digital finance & decentralized applications. Q5 Analyze the role of payment scripts in blockchain transactions. Examine & analyze different types of payment scripts. Explain how they ensure secure & verifiable transfer of coins. Illustrate with an example. Q6 What is a smart contract? Explain lifecycle of smart contracts. Explain advantages, limitations, risks & opportunities of smart contracts. Q7 Explain the structure of BTC transaction & ETH transaction. Q8 Explain the difference between mining & minting and explain the process
CO Mapped	CO1- CO6

Hard Copy Submitted